Published by Red Hat Developer

Developer Portals

Prepare to Perform with Red Hat Developer Hub



Hans-Peter Grahsl, Joshua Wood, and Ryan Jarvinen

Developer Portals

Prepare to Perform with Red Hat Developer Hub

by Hans-Peter Grahsl, Joshua Wood, and Ryan Jarvinen



Table of Contents

Preface ... iii
- Terms used in this book ... iii
- About the example infrastructure ... iv
- About the electronic edition ... iv
- About Red Hat Developer ... v

Chapter 1: A platform for portals ... 1
- The proliferating complexity of simple applications ... 1
  - Hello, [cruel] World ... 2
- Internal developer platforms ... 2
- Backstage: A platform for building developer portals ... 3
  - Backstage history ... 4
  - Backstage today: Project and ecosystem ... 4
  - Red Hat Developer Hub ... 5
- Summary ... 5

Chapter 2: The Software Catalog ... 6
- Catalog overview ... 8
  - Using the Software Catalog ... 8
  - What's in the catalog? ... 9
- Entities ... 10
  - Characteristics of an entity ... 10
  - Purpose and usage ... 10
- The catalog-info file ... 11
  - Basic structure ... 11
  - Key fields ... 11
  - Using catalog-info files ... 12
- Daily life with the catalog ... 13
  - Initial setup and component registration ... 13
  - Day-to-day development ... 13
  - Collaboration and governance ... 13
  - Maintenance and improvement ... 14
  - Scalability and evolution ... 14
- Summary ... 14

Chapter 3: Templates: Application head start ... 15
- The Point of Interest map application ... 15
- Application specifics ... 16
  - Structure and technologies ... 16
  - Architecture ... 16
- Software Templates ... 17
  - Template structure ... 17
- Software Template basics ... 18
- Template registration ... 22
- Catalog inspection for templates ... 24
- Instantiating a template: Scaffolding ... 25

Backend template....25
Proxy and frontend template....42
Summary....48
Chapter 4: Building the POI map....50
Implement the backend....50
Adapt database configuration....50
Write the code....52
Check CI/CD....55
Check backend app status....55
Explore the application's API....57
Add the documentation....59
Update the Software Catalog....65
Implement the proxy and the frontend....70
Write the code....70
Check CI/CD....79
Check frontend app status....80
Add the documentation....81
Update the Software Catalog....82
Summary....82
Chapter 5: Template and component evolution....83
Evolving Software Templates....83
Locate the template and clone the repository....84
Make the necessary template changes....85
Commit and push template updates....86
Updating existing components....86
Register the predefined patch template....87
Apply the predefined patch template....89
Review the opened merge request....92
Merge the auto-generated code changes....93
Inspect the build pipeline....94
Verify all updates for the component....94
Updating existing deployments....97
Register the predefined patch template....97
Apply the predefined patch template....98
Review the opened merge requests....102
Verify the patched deployment for the component....104
Promote the backend application....106
Summary....113
Chapter 6: Embracing the future of development with IDPs....115
Paradigm shift....115
Journey ahead....115
Tomorrow and tomorrow....116
Today....116
Red Hat Developer Hub resources....117
About the authors....118

Preface

This is a developer's guide to developer portals. In the first chapter, you'll learn what an internal developer portal (IDP) is and the problems addressed by this category of software. To practically apply those ideas, you'll examine Red Hat Developer Hub (RHDH), built around the open source Backstage core to implement an enterprise distribution of a platform for creating developer portals.

Subsequent chapters present some of the tools found in an RHDH instance and their representation in the RHDH interface, then show you how to work with those tools to build an application. You will gain insight about developer portals in terms of Backstage and RHDH, which together represent the pioneering implementation of developer portal concepts.

Terms used in this book

You'll see both the names Backstage and Red Hat Developer Hub in this book, which uses RHDH as a convenient Backstage instance. We don't exactly use the terms interchangeably. Directions and examples will usually refer to RHDH, but concepts and terms that come directly from the upstream Backstage core are named as such.

For example, you might go to the RHDH home page, but read references to core concepts like Backstage Software Templates. This illustrates how Red Hat Developer Hub is a distribution of Backstage, providing all the core Backstage features, layered beneath essential plug-ins and configuration tended by Red Hat in the RHDH product.

The examples in this book could be followed stepwise on a generic Backstage instance with little adaptation. But not *any* generic Backstage. You need a Backstage instance configured with the plug-ins for the implementing services on which the examples depend.

About the example infrastructure

At work, where things finally run won't be entirely your problem as a developer. But to practically apply the ideas in this book and conduct its exercises, you'll need infrastructure.

Internal developer portals (IDPs) like Backstage, and Red Hat Developer Hub (RHDH) in turn, connect work together and coordinate work intelligibly. A developer portal shows you the important work, but it does not do the work. An IDP deployment coordinates dozens of infrastructure pieces and processes and often hundreds of projects and services. The exercises in this book aim to simulate useful processes and DevOps/GitOps practices, and they employ several supporting services to show the actual value of IDPs as a central point of view and entry.

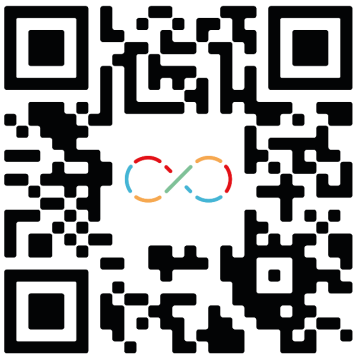
Bookmark the following URL, where you'll find continually updated guidance for getting access to the required infrastructure, from RHDH itself to OpenShift, Argo CD, Tekton, and the other supporting services this book's exercises harness in order to illustrate something like the real world: https://red.ht/dev-portals-book-infra

About the electronic edition

Print is easiest to read and it is portable without electric power or fragile plastic. But printed tech books can't quickly change to reflect the rapid changes in the software they cover.

Print copies of *Developer Portals: Prepare to Perform with Red Hat Developer Hub* include continuing access to the electronic edition, which is updated as errata and software advances require.

The always-different electronic version's location is always the same: https://red.ht/developer-hub-book



Scan the QR code to access the digital edition.

About Red Hat Developer

Building and delivering modern, innovative apps and services is more complicated and fast-moving than ever. Red Hat Developer has the technical tools and expertise to help you succeed.

The Red Hat Developer program provides many member benefits, including a no-cost subscription for individuals and access to products like Red Hat Enterprise Linux and Red Hat OpenShift. Visit our website to learn more.

Chapter 1: A platform for portals

This is (secretly) yet another book about DevOps.

As a cultural initiative, DevOps has probably made software more reliable and its creation more rapid. Its best understood practices have benefited their progenitors, publicizers, and most successful adopters. Developer engagement with deployment concerns engenders a systems thinking that improves application architecture, but DevOps declares for developers an unbounded array of new concepts, terms, and concerns.

This increase in cognitive load can make it feel like the early returns from DevOps practices such as automation and "infrastructure as code" and its progression toward GitOps are diminishing and progress slowing. More than 80% of developers say they have some kind of DevOps responsibility. The more layers between code editor and cloud deployment a developer has to master, the harder it is to onboard new teammates and the greater the chances of their burning out in the face of endless switching into less and less familiar contexts.

The proliferating complexity of simple applications

Think about writing "Hello, World." A few years ago, you'd grab your favorite editor or IDE, write some code, build it with your compiler or run it through your runtime. Then you'd hand it off to whoever controlled production when you pushed your locally-tested changes to the development branch. Your code got built and tested and deployed and had network traffic routed to it, backing services connected, deployment specifics configured, but little of that was within your control or even your line of sight.

DevOps implies a host of new concerns, systems, services, and tools.

Microservices add complexity along another axis. There are many pieces of any microservices application architecture. What is the component you need to work with in the first place? Who is responsible for a given microservice? How do you use its API facilities? What APIs and other components does it depend on? Where is its documentation?

Hello, [cruel] World

Writing "Hello, World" hasn't gotten any harder. But running it has. Hello, World today means a running container, built by an automated pipeline (triggered by a source code commit) into a container image, stored in some registry, with an attendant manifest declaring how it should be brought to life by a container orchestrator.

There is a lot of stuff that is not your application.

So much YAML

Build configuration, CI/CD, container, repository, Deployment, StatefulSet, Ingress, Route...

You want this stuff—or you should. And your company definitely wants it. It has a lot of benefits, even though we're focusing on some of the drawbacks right now. But complexity in doing simple things will threaten to impose diminishing returns. In computing, the answer to complexity is always abstraction.

You're supposed to be an expert in Java, Go, Python, or whatever your language and framework–not an expert on the details of your new team's AWS and OpenShift environments, configurations, and prerequisites.

Trade it all for just a little more

Internal developer portals (IDPs) aim to provide abstraction of development processes, offering a surface for capturing a team's tools, libraries, source boilerplate, and configuration and centralizing access to them. Ironically, in the IDP ecosystem examined here, simplifying all that YAML involves... a little more YAML.

Internal developer platforms

An internal developer portal is a set of tools, services, and practices intended to improve the productivity and collaboration of software development teams by abstracting and automating the surrounding complexity of the development and deployment process. An IDP provides developers with a unified environment for managing application components, services, and

the development life cycle, including code, infrastructure, and deployment. IDPs address several key challenges in the software development process, including:

- **Complexity:** Modern software development involves many tools, services, and processes. IDPs centralize access to these resources.
- **Collaboration:** IDPs promote collaboration by offering shared resources, standardized workflows, and a consistent experience in a development team.
- **Visibility:** Developers can't always see what other teams are working on. IDPs make large deployment environments more transparent.
- **Productivity:** By automating repetitive tasks and providing self-service capabilities, IDPs allow developers to focus on writing code and delivering value.

Backstage: A platform for building developer portals

Backstage is an open source IDP platform created by Spotify, originally to serve their internal development. Backstage organizes the developer portal around five central functions:

- **Software Catalog:** A central repository for the services and components used in a team or organization, enabling discovery and reuse. Catalog entities index the disparate resources that go into an application, from source code to build services to orchestrated deployments.
- **Plug-in ecosystem:** Backstage supports custom plug-ins and a growing ecosystem of integration and automation plug-ins. Plug-ins integrate Backstage with infrastructure services and software components, import patterns and mechanisms, and allow teams to customize Backstage to integrate their own systems, tools, and methods.
- **Standardized workflows:** Designed to capture effective patterns, tools, processing used by development teams and present them as established paths to development, Backstage helps enforce best practices and standards.
- **TechDocs:** TechDocs conventionalize the mechanics of creating and publishing documentation but someone still has to write it. You can write documentation easily in Markdown or other familiar markups and manage it like code in your repos to see it compiled and published in an automated way.
- **Search:** Search lets others find components, services, and documentation without clicking around endlessly.

Backstage aims to enhance developer productivity with a single place for development activities.

Backstage history

Backstage started at Spotify in 2016-17. Spotify developers maintain a huge number of microservices, components, and tools.

Backstage addressed the complexity and obscurity attendant in an ever-expanding set of services and tools—and ever-expanding teams creating and using them. The main idea was a unified interface for discovering and interacting with all of the resources that comprise an application. Not just source code, but integration and deployment configuration, source control, standard versions, languages, and libraries; execution status; rebuild and redeploy automations—everything. Even docs.

In 2020, Spotify released Backstage under the open source Apache license, allowing other organizations to use the platform and contribute to its improvement. Developers and companies adopted the project in response, many of them, including Red Hat, contributing new features, abilities, and fixes to the upstream code base. A Backstage ecosystem has grown around the core software as well, with plug-ins and integrations developed to extend its functionality.

In 2022, the Cloud Native Computing Foundation (CNCF) accepted Backstage as an incubating project. This solidified Backstage's position as a principal implementation of IDP concepts and standardized the project’s governance within the CNCF landscape.

Backstage today: Project and ecosystem

Backstage is an open source platform designed as the foundation of an internal developer portal. Earlier, you saw the three key software expressions of Backstage’s purpose. Those features are how Backstage abstracts complexity and provides paved roads to standard tools and practices. When resources are documented, discoverable, searchable, and their dependencies and ownership clear, teams benefit in several ways.

Backstage promotes collaboration by providing visibility into what different teams are working on. Transparency helps prevent duplication of efforts, encourages reuse of services and components, and fosters a sense of community among development teams.

Backstage can automate tasks such as creating new services or components, provisioning infrastructure, and generating documentation. Automation reduces manual overhead and lets developers focus on writing code.

Teams can customize Backstage with features and plug-ins to fit their specific requirements. Since Backstage has a growing community of contributors and adopters, new features, integrations, and improvements appear frequently.

Red Hat Developer Hub

Akin to familiar Red Hat flagship products like Red Hat Enterprise Linux and Red Hat OpenShift, Red Hat Developer Hub (RHDH) is an enterprise distribution with an open source project at its core. RHDH harnesses the Backstage core code together with enterprise features, curated plug-ins, day-two configuration, and ready-made software templates for common development scenarios.

Like Backstage, Red Hat Developer Hub is a platform for building developer portals. There is always platform engineering work, with or without that formal title. Identifying patterns and standards and refining them in line with organization goals is a key part of getting value out of your site's developer portal. RHDH makes it easier for you to get started with the examples in the book by avoiding a lot of secondary decisions about integrations, configuration, and features that you'd need to make if you started from scratch with upstream Backstage.

Summary

Every team and every project evolves a tailored development environment. This collection of tools, services and configuration is often maintained by convention and transmitted by osmosis.

Internal developer portals help teams curate, manage and replicate these environments. Backstage is an open source CNCF project for building developer portals, and for encapsulating tools, services, documentation and best practices in "golden paths" to ease onboarding and everyday work. Red Hat Developer Hub is Red Hat's enterprise IDP platform, curating Backstage core and the ecosystem around it.

Chapter 2: The Software Catalog

Before you can do much work, you have to find your tools and materials. The Developer Portal's Software Catalog, shown in Figure 2-1, provides a searchable index of APIs, services, and software components along with their documentation, ownership and access metadata, and Templates for their repeated instantiation.

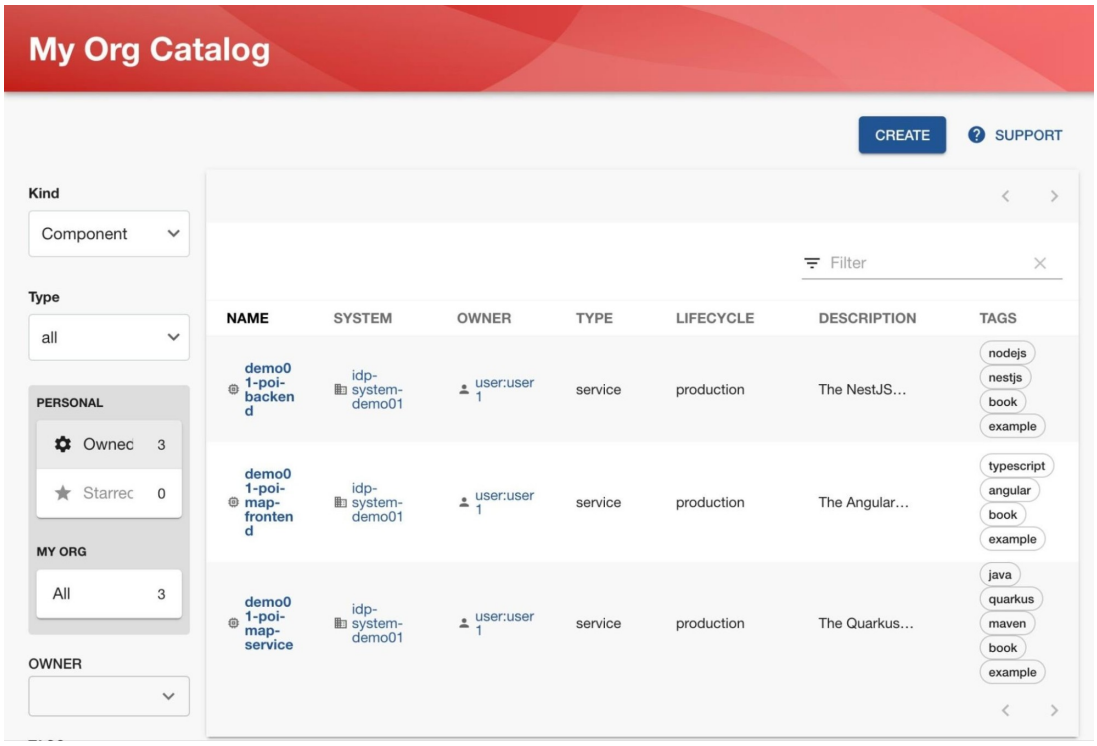


Figure 2-1: The Software Catalog.

Catalog overview

The catalog provides a central, organized location where development teams can register and document the services and components they use in their software projects. This includes information about applications, APIs, libraries, databases, and more.

The Software Catalog is a (human) service discovery platform that lets developers search for and browse existing services and resources. This makes it easier to find and reuse existing components, reducing redundancy and promoting collaboration.

The catalog conventionalizes ways to attach metadata, documentation, and other relevant information to each service or component entry. This might include descriptions, ownership information, usage guidelines, and documentation, making it easier for others to understand and work with these resources.

Notice that metadata includes ownership and responsibility for each service or component in the catalog. Making this information obvious makes communication more efficient because it is clear who is accountable and who to contact for any entity in the catalog.

Backstage, and Developer Hub in turn, is a switchboard for integration with other tools and services, such as version control systems, CI/CD pipelines, and monitoring. Integration means the catalog reflects the current state of the real world, and you can comprehend that state without bouncing between different user interfaces for each integrated service.

Organizations can customize the Software Catalog and most of Developer Hub. This allows you to define what types of services or components should be included, how information is structured, and what access rules regulate who can see or modify components and sections of the portal.

The catalog serves as a centralized system for tracking software components, making it easier for development teams to manage and discover information about the software within their organization.

Using the Software Catalog

First, you need to set up Developer Hub (or upstream Backstage) and its Software Catalog in your environment. This involves deploying an instance of the developer portal and configuring the Software Catalog to connect to your source control and other tools.

Components are added to the catalog through YAML configuration files conventionally named `catalog-info.yaml`. These files contain metadata about the component, including its name, owner, and type, and are usually stored in the same repository as the component.

Once components are added, you can use the Developer Hub interface to explore the catalog. You can search for components, view detailed information, and access linked resources such as documentation, source code, and operational dashboards. This book focuses on this daily development usage of a Developer Portal and its Software Catalog.

Owners and teams can update the metadata and documentation as components evolve, ensuring the catalog remains reliable and up to date.

What's in the catalog?

The Software Catalog can contain a wide variety of items, broadly categorized into the following:

- **Services:** Backend services, microservices, or server-side applications.
- **Websites:** Frontend applications, web apps, or static websites.
- **Libraries:** Shared code libraries or modules that can be used by other software components.
- **Documentation:** Standalone documentation sites or repositories containing documentation related to software components.
- **APIs:** RESTful APIs, GraphQL APIs, or any other API endpoints.
- **Data pipelines:** ETL jobs, data processing pipelines, and other data-intensive workflows.
- **Components:** Any other software components that don't neatly fit into the above categories but are part of the organization's development and delivery landscape.

The Software Catalog is designed to improve software development and operational efficiency by providing a centralized, searchable, and documented catalog of all the software components in an organization. Its ability to integrate with various tools and provide insights into the health and status of components is what makes it valuable. The preceding list is not exhaustive; teams can create new *Kinds* to define arbitrary resources. For example, other Kinds might include clusters managed as deployment targets for different stages of application lifecycle or different geographies, Playbooks of Ansible automations that deploy infrastructure components, or anything else related to a team's development activities.

Entities

An **Entity** is a unit of configuration or a resource that is part of the software ecosystem in an organization. Entities are the fundamental building blocks within the Catalog, serving as abstractions for various types of software-related items, such as services, websites, libraries, APIs, and documentation. Each entity is described by a YAML file, usually named `catalog-info.yaml`, which defines the entity's characteristics, relationships, ownership, and other details.

Characteristics of an entity

- **Type and Kind:** Entities are categorized by their `type` and `kind`, which specify the general category and the specific nature of the entity, respectively. For example, a service might be represented as an entity with the kind `Component` and the type `service`.
- **Metadata:** This includes essential information such as the entity's name, description, tags, and other identifiers that help in cataloging and discovering the entity within the catalog.
- **Specification** (spec): The spec field provides detailed information about the entity, including its life cycle status (e.g., experimental, production), and ownership (which team or individual is responsible for the entity), and any specific attributes relevant to its kind.
- **Annotations and labels:** Entities can be annotated with additional metadata and labels that offer more context, link to external resources, or integrate with other tools and services. You can use annotations to link an API documentation, code repository, or monitoring dashboard directly to the entity.
- **Relations:** Entities can be related to one another, illustrating dependencies, ownership, or other relationships. These relations help in understanding the ecosystem's structure and how different entities interact.

Purpose and usage

The primary purpose of defining entities in the Software Catalog is to provide a structured and searchable inventory of all software components within an organization. This inventory aids in visibility, governance, and operational efficiency by creating a comprehensive,

integrated view of an organization's software landscape, streamlining workflows, and enhancing the developer experience.

The catalog-info file

The `catalog-info.yaml` file is the primary mechanism through which components are described and added to the catalog. This YAML file contains metadata about a software component, allowing Developer Hub to understand, categorize, and display information about the component. The structure and contents of the file enable integration of components into the Developer Hub ecosystem. The next section describes how the `catalog-info.yaml` file describes components.

Basic structure

The `catalog-info.yaml` file includes several key pieces of information, structured in YAML format. Here is a simplified example to illustrate the basic pieces:

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: example-service
  description: "An example service."
  tags:
    - java
    - quarkus
spec:
  type: service
  lifecycle: experimental
```

Key fields

- `apiVersion`: Specifies the version of the Backstage API that the file is compatible with. This helps ensure that the file structure aligns with what Backstage and Backstage distributions like Developer Hub expect.
- `kind`: Describes the type of component. Common kinds include `Component`, `API`, and `Resource`, among others. This field helps the catalog understand how to treat and display the entity.
- `metadata`: Contains essential information about the component, such as:
    - `name`: A unique identifier for the component within the catalog.

    - `description`: A brief description of what the component does.
    - `tags`: A list of keywords or tags associated with the component, aiding in search and categorization.
- `spec`: Provides detailed specifications about the component, including:
    - `type`: A more specific classification of the component, such as service, website, library, etc.
    - `lifecycle`: Indicates the life cycle stage of the component (e.g., `experimental`, `production`, `deprecated`).
    - `owner`: Specifies the team or individual responsible for the component. This is crucial for governance, maintenance, and communication purposes.

Additional fields

The `catalog-info.yaml` file can also include a variety of other fields to provide more detailed information about the component, integrate with other tools, or link to external resources. Some examples include:

- `annotations`: Used to attach additional metadata or links to external resources, such as documentation, source code repositories, or operational dashboards.
- `relations`: Defines relationships between different entities in the catalog, such as dependencies between services or ownership relations.
- `providesApis`: For components that provide APIs, this field expresses the service side of the relation to consumers of that API, enumerating the API endpoints provided by a component for the API list, describe, and exercise features of the developer portal.
- Consumers that rely on endpoints of an API describe that side of the relation with the `consumesApi` field.

Using catalog-info files

By convention, developers include a `catalog-info.yaml` file in the root directory of their component's source code repository. When the repository is registered with Developer Hub, the platform reads this file to ingest and display information about the component in the Software Catalog. This lets team members discover, understand, and interact with various software components across the organization through a unified interface.

By standardizing the way components are described and integrated into the Software Catalog, the `catalog-info.yaml` file facilitates better software inventory management, enhances discoverability, and promotes more connected and efficient development.

Daily life with the catalog

You use the Software Catalog as a central hub for managing, discovering, and interacting with software components. The next sections describe how you'll typically engage with the Software Catalog throughout different stages of development.

Initial setup and component registration

Developers begin by **registering their software components** with the catalog. As discussed earlier, this involves creating the `catalog-info.yaml` file in the root of a component's repository. This file is then submitted to the catalog, either manually through the UI or automatically via discovery mechanisms set up by the team.

Day-to-day development

Discovery and browsing: Once components are registered, you can use the catalog to discover existing components, APIs, and tools. This aids in reusing code, understanding dependencies, and learning about other teams' projects. The search functionality, filtering, and tagging systems help you quickly find relevant components.

Understanding components: For each component, the catalog provides detailed information, including its ownership, life cycle status (e.g., production, deprecated), and dependencies. You can easily access the component's repository, related documentation, and operational dashboards from the catalog.

Integrating and using APIs: When building or updating services, you can use the catalog to find and integrate APIs. The catalog provides endpoints, documentation, and even testing tools for listed APIs, simplifying the integration process.

Collaboration and governance

Ownership and responsibilities: The catalog clearly lists the ownership of each component, making it easier to identify and contact the responsible teams for collaboration, support, or contributions. This transparency fosters a sense of ownership and accountability.

Compliance and best practices: Teams can enforce standards, best practices, and compliance requirements through the catalog. Templates can be provided for creating new services or components, ensuring consistency and adherence to organizational policies.

Maintenance and improvement

Monitoring and health checks: Developer Hub and the Software Catalog integrate with monitoring and CI/CD tools, allowing developers to see the health, performance, and build status of their components directly within the catalog. This centralized overview aids in proactive maintenance and troubleshooting.

Documentation and resources: Developers can access and contribute to the documentation of components, improving knowledge sharing and collaboration. The Software Catalog often serves as a gateway to a component's documentation, decision records, and related resources.

Scalability and evolution

As the organization grows, the catalog supports the **scaling of software development processes** by streamlining component registration, discovery, and management. It helps maintain a clear overview of the software landscape, even as the number of components increases.

Summary

The Software Catalog encapsulates increasingly diverse software ecosystems, allowing you to focus more on building features and fixing bugs, and less on navigating a maze of infrastructure and supporting services.

Chapter 3: Templates: Application head start

Software Templates are a Backstage feature for flexibly defining a starting point for some arbitrary kit. A template might represent a place to begin work on a new application built with some standard language or framework—for example, potentially including executables, runtimes or compilers of standard versions, boilerplate code, build configurations, and the like.

You'll investigate templates by adding to your running Red Hat Developer Hub portal a pair of templates that represent a starting point for working on a web application. Then, you'll instantiate from those templates with the portal's Create function to generate all the scaffolding for your new application: source code boilerplate in a Git repo, pipelines to build it, and manifests to deploy it on Kubernetes or Red Hat OpenShift.

The Point of Interest map application

The exercises in this book center on a world map application (Figure 3-1) that you can pan, zoom, and click to investigate arbitrary points of interest.

You'll construct the Point of Interest (POI) application with a set of templates that define and deploy foundation components, then use Red Hat Developer Hub's Software Catalog, API index, and other facilities to implement the application and extend it with new features.



Figure 3-1: The Point of Interest map application user interface.

Application specifics

Before we begin, let's take a look at the application's technologies and architecture.

Structure and technologies

The POI map application has four parts:

- A relational database that stores the point of interest records: **PostgreSQL**.
- A backend application exposing a REST API to serve the points of interest from the database, written in **TypeScript** with **NestJS**.
- A proxy application that sits as a facade between the backend and frontend to delegate web client requests to the backend's REST API accordingly, written in **Java** with **Quarkus**.
- A Single Page Application (SPA) to provide the user interface where the map is displayed, written in **TypeScript** with **Angular.**

Architecture

While the backend, proxy, and frontend parts of the map application could be individually built and separately deployed, it makes sense to simplify the architecture a bit. The SPA frontend is embedded with the API client component, served with the Quarkus HTTP server methods. This gives the POI map application an architecture of three primary pieces: the frontend/API client machinery, the NestJS POI API backend, and the database. See Figure 3-2.

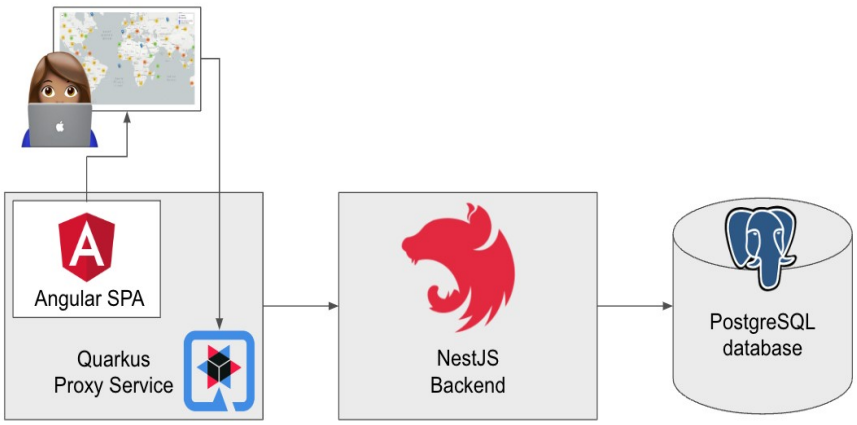


Figure 3-2: POI application architecture.

Software Templates

Template structure

A Software Template is stored in its own source control repository, then registered in a given RHDH instance's software catalog.

The two templates defining the foundations of the POI map application use the same folder structure:

```
my-template (1)
├── manifests (2)
├── skeleton (3)
└── template.yaml (4)
```

where:

1. `my-template` is the top-level directory containing all template resources;
2. The `manifests` subdirectory holds the YAML files, Helm charts, and other declarations related to the deployment of the application;
3. The `skeleton` subdirectory holds the basic source code structure of the application;
4. `template.yaml` is a YAML file defining a Backstage catalog entity of the kind Template.

The scaffolding to support the three-part architecture of the map application is split between two templates, as shown in Figure 3-3. The first template deploys the backend application

together with its database. The second template creates the framework for the frontend and API client.

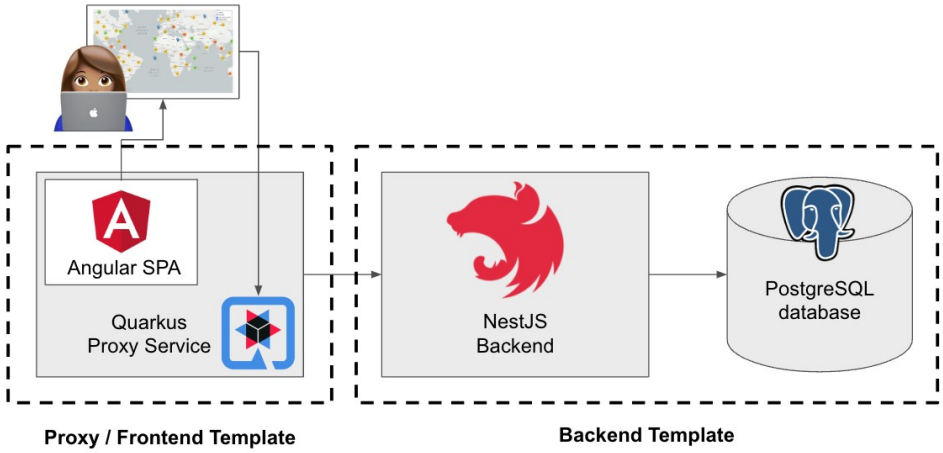


Figure 3-3: Application architecture expressed in two templates.

Software Template basics

As you saw in the preceding template directory listing, the `template.yml` file at the root of the template directory defines a Backstage Software Template. A template manifest is structured around a few major YAML stanzas.

First, the metadata block describes the Template and defines its owner and the Backstage entity type created by instantiating it–in this case, an entity of type `service`.

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: a-simple-template
  title: A simple template
  description: This simple template is used for learning purposes.
spec:
  owner: rhdeveloper-book-authors
  type: service
```

Next, the `parameters` array declares the elements of the configuration form presented when someone selects a template and creates a new instance of the entities defined in it. The `parameters` array describes configuration that varies between deployments of a template's resources so that you can change it as appropriate for a new instance.

Given the example `parameters` stanza shown here, each new instance of this template will expect you to enter or choose a number of configuration items, including the source

repository where template resources should be created, and the RHDH user who owns the entities created from the template. While platform engineering topics like template creation are beyond the scope of this book, it is helpful to see how a template author defines the template instance configuration form. Notice in the `Name` property that parameters can have a default value. The `Owner` property defines a selection pop-up menu and the items on it.

```
  parameters:
  - title: Enter some (required *) component parameters
    required:
      - name
    properties:
      name:
        title: Name
        type: string
        description: unique name for this component
        default: my-component-123
      owner:
        title: Owner
        type: string
        description: owner of this component
        ui:field: EntityPicker
        ui:options:
          catalogFilter:
            kind: [User]
  - title: Choose a repository location
    required:
      - repoUrl
    properties:
      repoUrl:
        title: Repository Location
        type: string
        ui:field: RepoUrlPicker
        ui:options:
          allowedHosts:
            - github.com
            - gitlab.com
```

The preceding YAML snippet renders as a form in RHDH's web UI, as shown in Figure 3-4. You can specify the component's name and owner in the first form section.

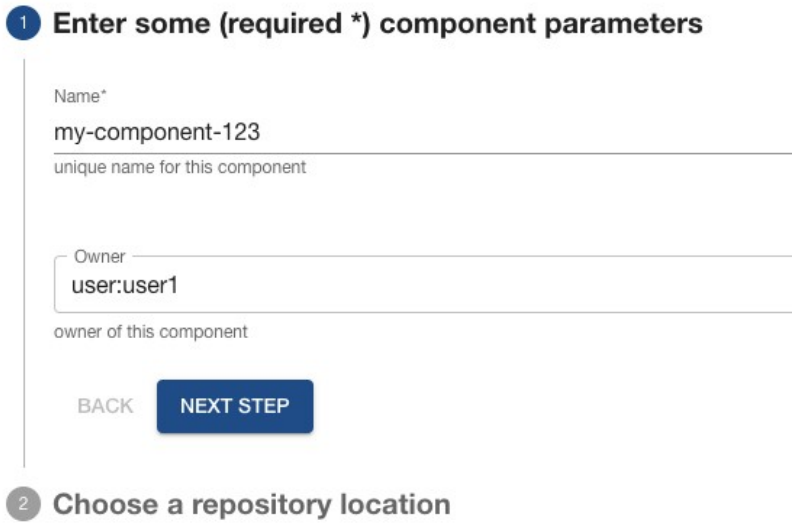


Figure 3-4: Template Parameters form wizard, section 1.

In the second form (Figure 3-5), the template gathers where to scaffold the entity's source code repository. In this template, **host** can be either **github.com** or **gitlab.com**, selected from a drop-down menu. The user or organization owner and the repository name are also configured in this section.

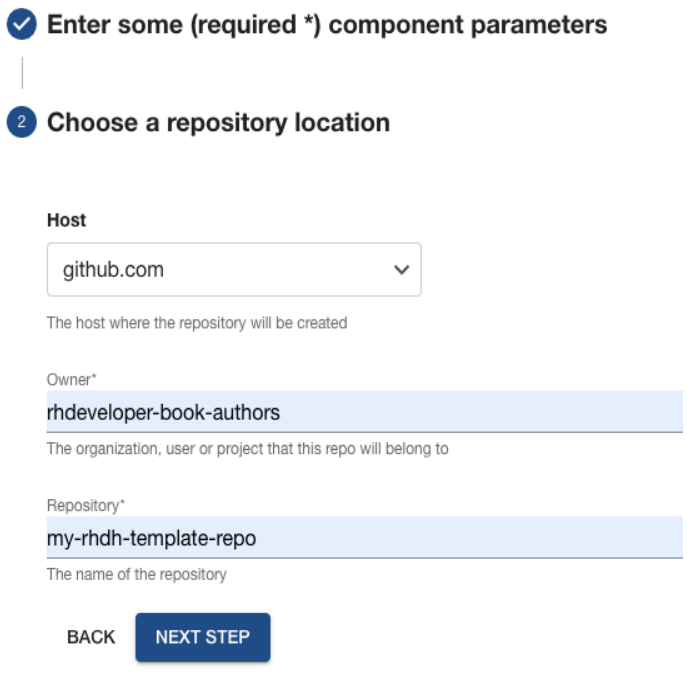


Figure 3-5: Template Parameters form wizard, section 2.

The third section is where the templating magic actually happens. The `steps` array defines the sequential actions that are executed by the scaffolder backend in RHDH:

```
  steps:
    - id: templateSource
      name: Generating the source code component
      action: fetch:template
      input:
        url: ./skeleton
        targetPath: ./source
        values:
          name: ${{ parameters.name }}
    - id: publishSource
      name: Publishing to the source code repository
      action: publish:github
      input:
        sourcePath: ./source
        description: Source code repository for component ${{ parameters.name }}
        repoUrl: ${{ parameters.repoUrl }}
        defaultBranch: main
        repoVisibility: public
    - id: registerComponent
      name: Register component into the catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps.publishSource.output.repoContentsUrl }}
        catalogInfoPath: '/catalog-info.yaml'
```

This excerpted `steps` array defines three step actions:

- `templateSource → fetch:template` downloads source boilerplate from the template's origin repository, expands variables declared in the parameters, replacing their tokens with the appropriate values within files and filenames, and finally places the result in the configured repository, or optionally in a subdirectory specified by the `targetPath` variable.
- `publishSource → publish:github` initializes a Git repository based on the templated source from step 1 and pushes it to GitHub. There are similar actions available for other Git hosting services; for instance, GitLab or BitBucket.

- `registerComponent → catalog:register` registers entities into the software catalog based on a catalog description file. Conventionally named `catalog-info.yaml`, this file is read from the Git repository created in step 2.

Template registration

You need to tell the portal about a new template in order to work with it. In an organization with an established Developer Hub or Backstage instance, daily development probably won't involve registering new templates as often as it will involve creating new entities from provided templates and monitoring and working with entities already created and indexed in the Software Catalog. But you'll have to register the two POI application templates on your new and mostly empty RHDH instance, so here is a look at how template registration works.

There are two ways to add templates and make them available to the portal's Create functions. First, RHDH inherits `app-config.yaml`, Backstage's main portal configuration file. This file declares configuration for the life of a running portal instance, including references to template source URLs. The portal must be restarted to change this static configuration.

Templates can instead be added dynamically through the Register existing component item found in the Create view. Again, you inform the portal about new templates by reference to their source URL.

You can use the dynamic approach just described to add the two templates the POI application will be built from. In the GitHub organization hosting the template repository for this book, you will find detailed instructions on how to prepare the necessary templates for your RHDH environment. The README explains what to do before you can proceed with the following instructions:

1. Go to **+ Create → Register** existing component and copy the full HTTP(s) URL to the `template.yaml` file in the `nestjs-with-postgres` folder of your forked Git repository hosting all book templates.. Paste the URL into the form's (1) **Select URL** field (see Figure 3-6).
2. Clicking the **ANALYZE** button shows that two entities will be added into the software catalog (Figure 3-7). One entity is the location, the HTTP URL from where the template was loaded, and the other entity is the template itself.

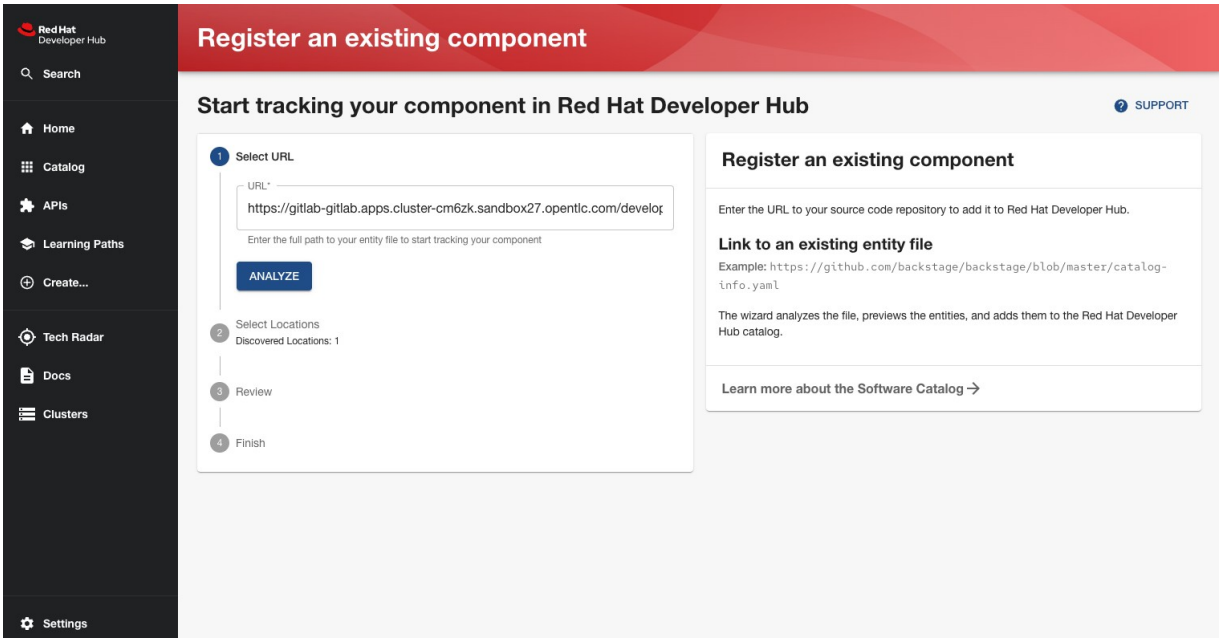


Figure 3-6: Template registration into the Software Catalog, step 1.

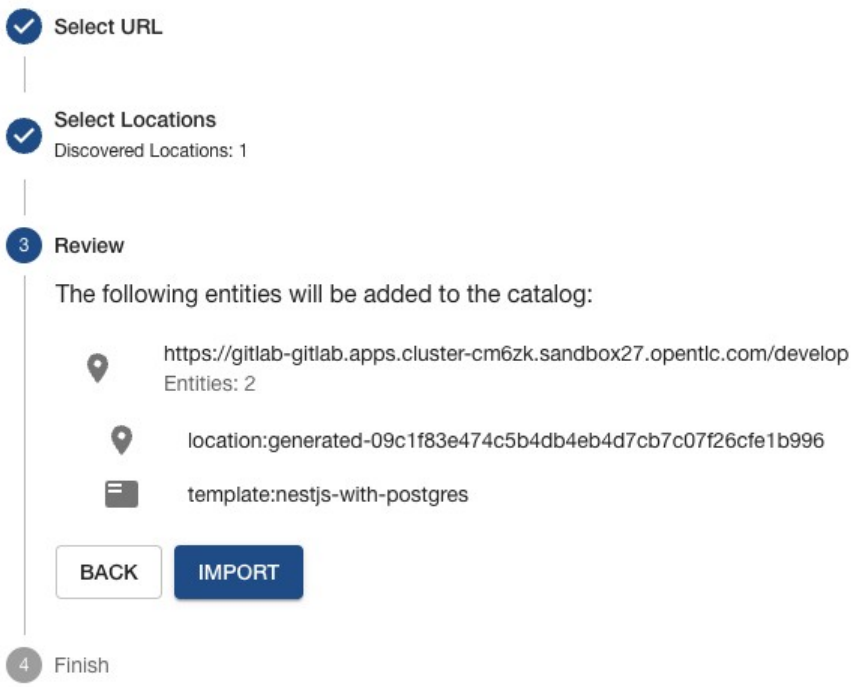


Figure 3-7: Template registration into the Software Catalog, step 2.

3. Click the **IMPORT** button to confirm. Figure 3-8 shows the entities have been added to the catalog.

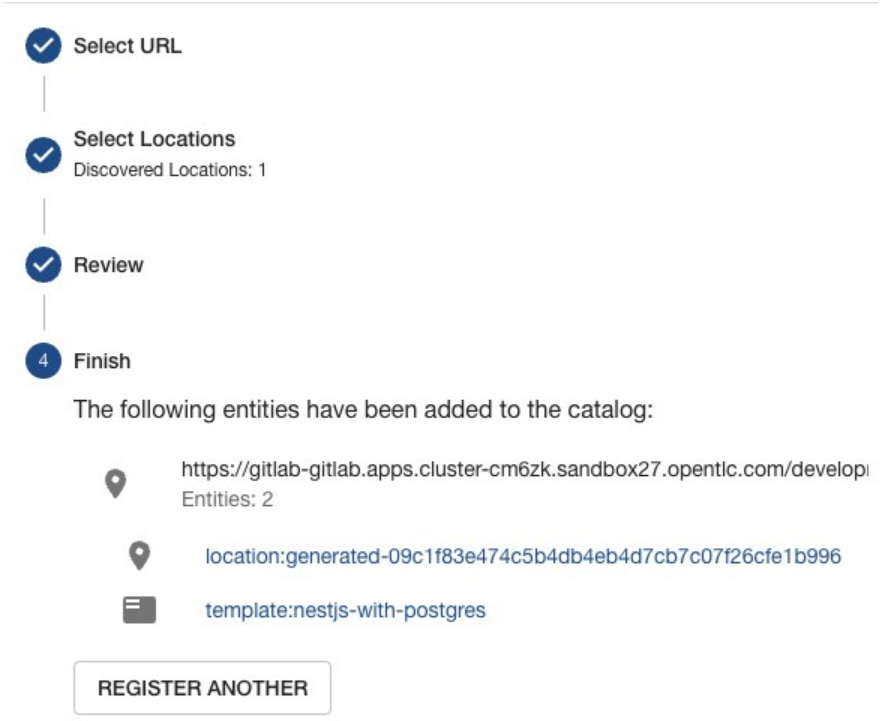


Figure 3-8: Template registration into the Software Catalog, step 3.

4. Click the **REGISTER ANOTHER** button. This brings you the same registration form wizard where you can register the second template. Copy the HTTP URL of the `template.yaml` file in the `quarkus-with-angular` folder of the Git repository into the form's (1) **Select URL** field and continue as you did for the first template.

Catalog inspection for templates

Go to the software catalog and filter for **Kind: Template** to see the two new templates (Figure 3-9).

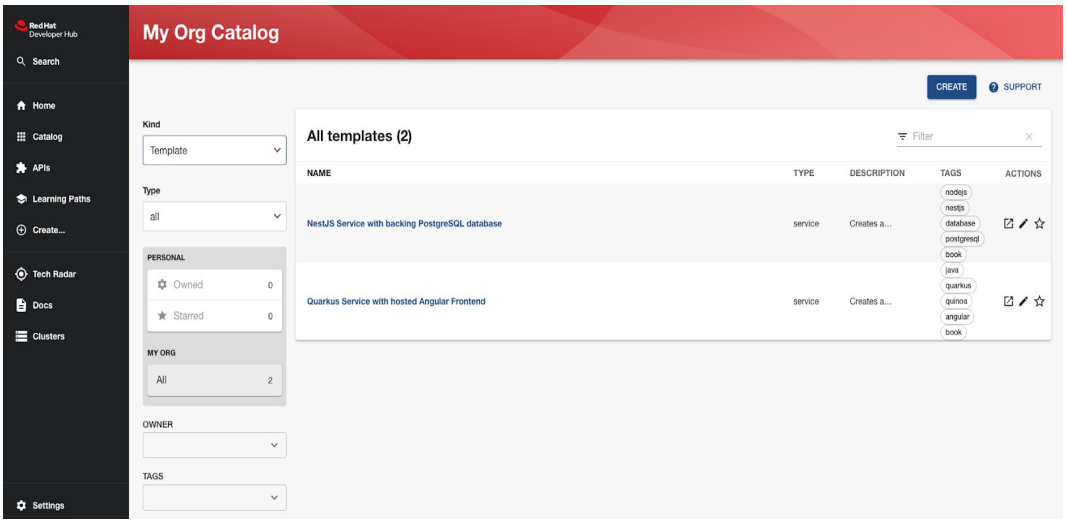


Figure 3-9: Viewing templates in the Software Catalog.

Instantiating a template: Scaffolding

With the two map application templates available to the portal's Create functions, you're informed about how Software Templates work and how to add templates to your RHDH portal. From here, you're ready to start where most developers would: using a template to get the basics in place for building a new application.

Backend template

Begin by creating an instance from the `nestjs-with-postgres` template. This template scaffolds the source repository for the backend service, builds the code in it, and deploys the result along with the database server on which the backend relies. Go to **+ Create** and select the template **NestJS Service with backing PostgreSQL database** by clicking the **CHOOSE** button in the lower-right corner of the template tile (Figure 3-10).

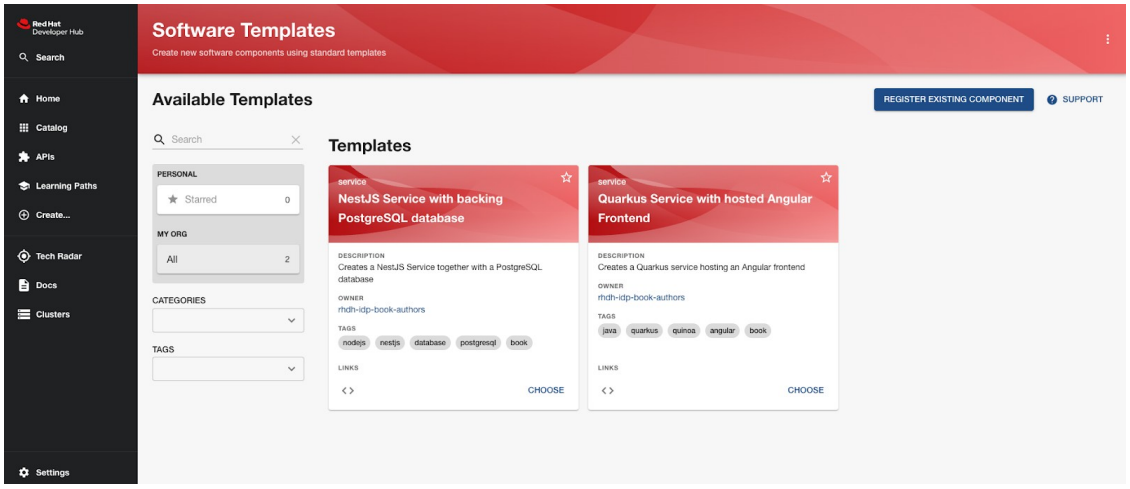


Figure 3-10: Portal Create view with template tiles.

Template form wizard

This brings you to the template's form wizard (Figure 3-11) where you can configure certain elements of the template. In the first section of this form, you define the GitLab location where the resulting source code and GitOps repositories should be stored.

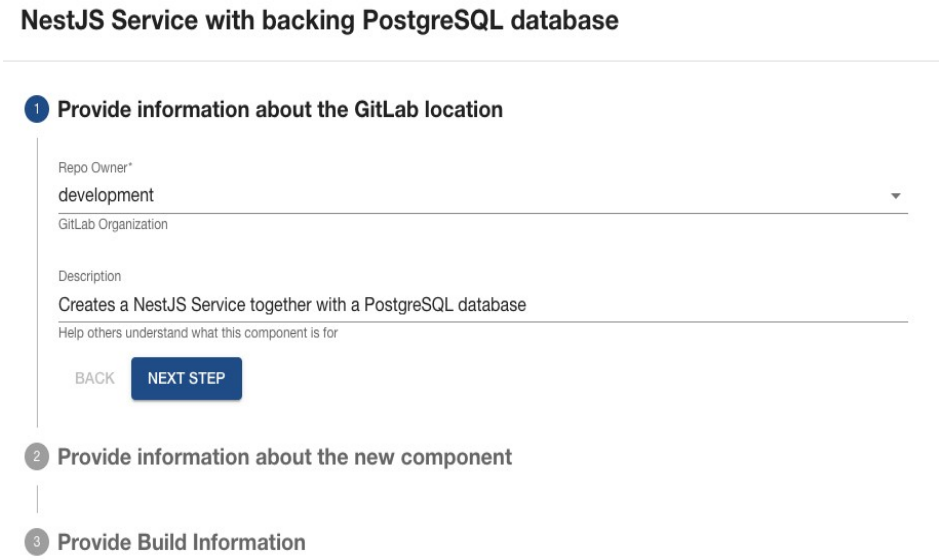


Figure 3-11: NestJS Service template configuration form, first section.

In the second section (Figure 3-12), you specify the who, what, and where of the application resources to be scaffolded from the template. The cluster ID and namespace where the running application should be deployed, what application ID names it, and the user who owns it. You'll see this metadata reflected in descriptions, data about, and links between entities in the Software Catalog after you submit the Create forms and scaffolding is complete.

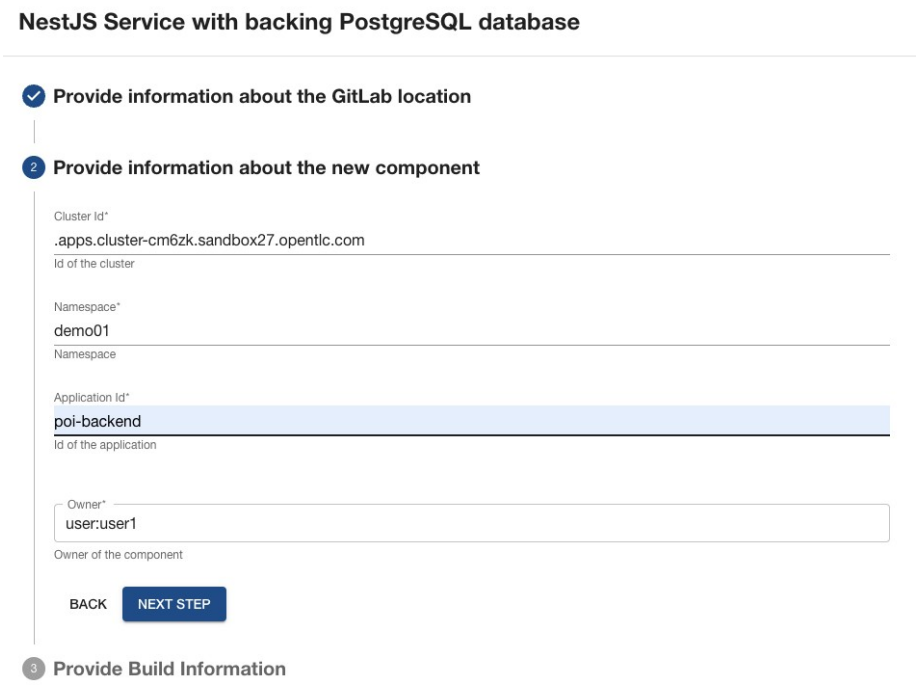


Figure 3-12: NestJS Service template configuration form, second section.

The third form section (Figure 3-13) specifies the container image registry where the application containers should be stored and available to the cluster orchestrator (OpenShift in the book's exercises). You can also optionally set a tag for the container image. The default, `latest`, is acceptable for purposes of building the example application, although it is not usually a best practice for production.

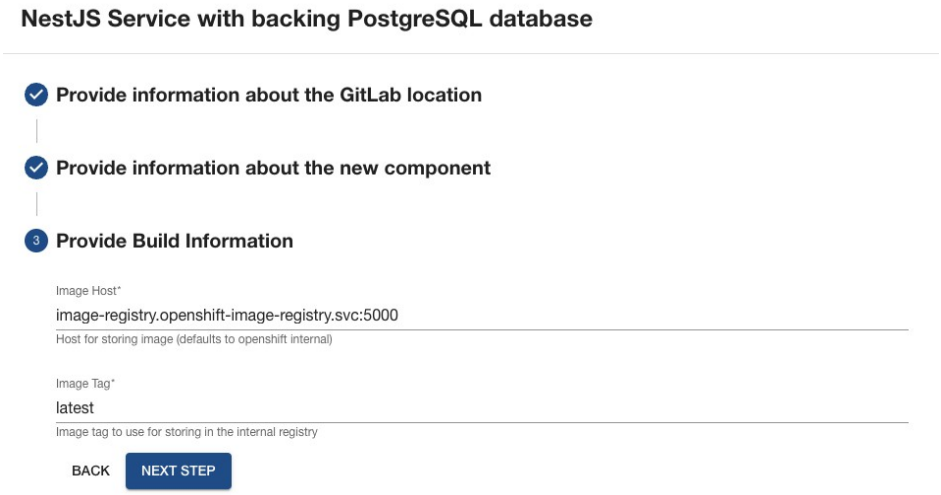


Figure 3-13: NestJS Service template configuration form, third section.

Clicking the **NEXT STEP** button shows you a summary of all the entered form fields for a final review, as shown in Figure 3-14. Click **CREATE** to kick off the process of scaffolding application resources from the template.

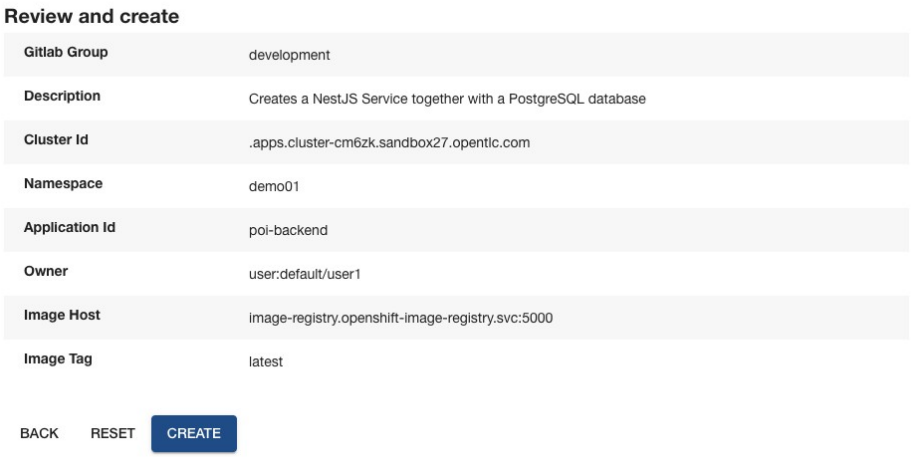


Figure 3-14: NestJS Service template configuration form, final review.

Template scaffolding process

The NestJS Service backend template is composed of six sequential steps (Figure 3-15), each of which represents either a built-in or a custom scaffolder action available in your running RHDH portal:

1. `fetch:template`: Fetches the template from its location and recursively walks through all source folders and files (see `skeleton` subfolder at the origin). In each file, the scaffolder checks if it finds variables and needs to perform parameter replacements based on the settings that have been entered upfront in the form wizard. Find more details about the templating syntax in the Backstage documentation.
2. `publish:gitlab`: All processed source files resulting from the templating process in step 1 are then published into a source code repository for this component according to the GitLab settings.
3. `fetch:template`: Similar to step 1, it will fetch the template contents from its location and recurse through the manifest files (see `manifest` subfolder at the origin) to potentially perform parameter replacements in the files' contents based on the settings that have been entered.
4. `publish:gitlab`: All processed manifest files resulting from the scaffolding process in step 3 are then published into a GitOps repository for this component according to the GitLab settings.

5. `catalog:register`: Using the information found in the scaffolded source code repository's `catalog-info.yaml` file, the component is registered in the software catalog. To learn more about the descriptor format, check out the official docs page.
6. `argocd:create-resources`: Instructs Argo CD to take action and start processing what has been scaffolded and published into the GitOps repository in steps 3 and 4, respectively.

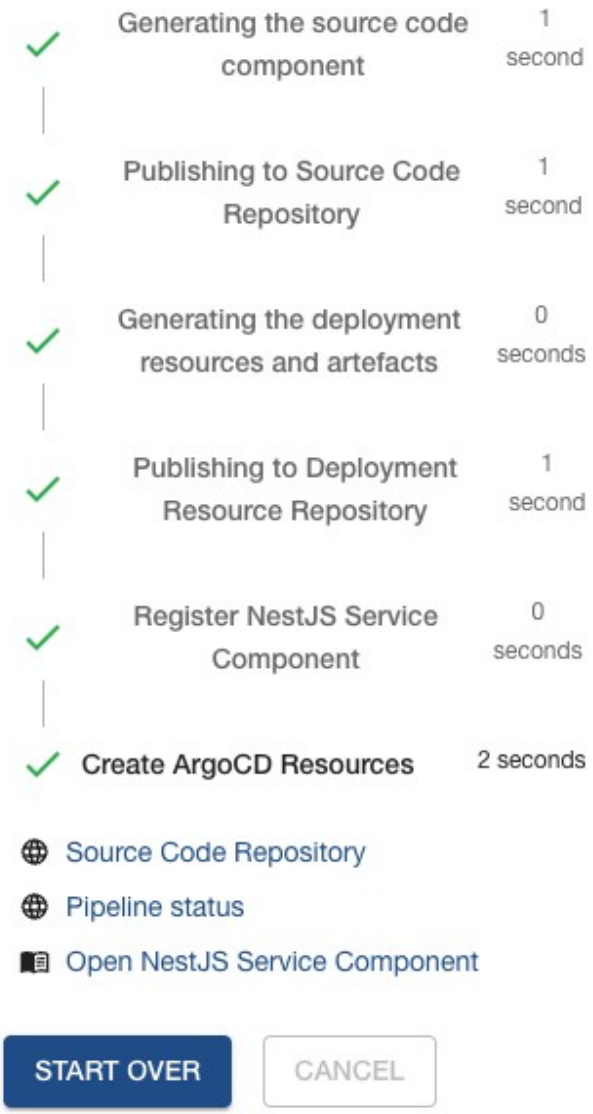
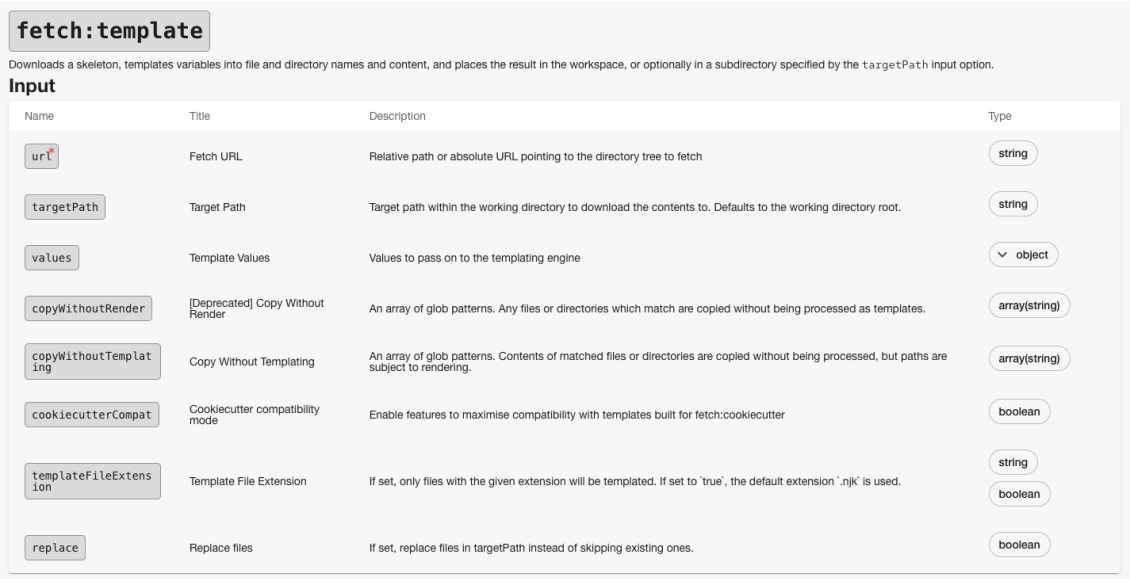


Figure 3-15: NestJS Service template scaffolding.

Template steps can use built-in actions and custom actions written for a team's specific needs. You can view which actions are installed in your RHDH instance by visiting `http(s)://<YOUR_BACKSTAGE_BASE_URL>/create/actions`. This page shows the

instance's available actions, including a brief description of each and the parameters it accepts (Figure 3-16).



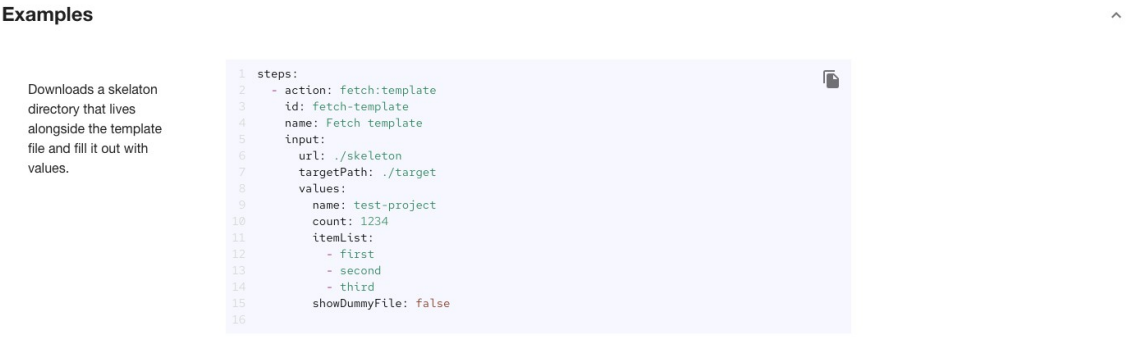
fetch:template

Downloads a skeleton, templates variables into file and directory names and content, and places the result in the workspace, or optionally in a subdirectory specified by the `targetPath` input option.

Input

Name	Title	Description	Type
url *	Fetch URL	Relative path or absolute URL pointing to the directory tree to fetch	string
targetPath	Target Path	Target path within the working directory to download the contents to. Defaults to the working directory root.	string
values	Template Values	Values to pass on to the templating engine	object
copyWithoutRender	[Deprecated] Copy Without Render	An array of glob patterns. Any files or directories which match are copied without being processed as templates.	array(string)
copyWithoutTemplating	Copy Without Templating	An array of glob patterns. Contents of matched files or directories are copied without being processed, but paths are subject to rendering.	array(string)
cookiecutterCompat	Cookiecutter compatibility mode	Enable features to maximise compatibility with templates built for fetch:cookiecutter	boolean
templateFileExtension	Template File Extension	If set, only files with the given extension will be templated. If set to `true`, the default extension `.njk` is used.	string boolean
replace	Replace files	If set, replace files in targetPath instead of skipping existing ones.	boolean

Figure 3-16: Describing the fetch-template action and its parameters.



Examples

Downloads a skelaton directory that lives alongside the template file and fill it out with values.

```
steps:
  - action: fetch:template
    id: fetch-template
    name: Fetch template
    input:
      url: ./skeleton
      targetPath: ./target
      values:
        name: test-project
        count: 1234
        itemList:
          - first
          - second
          - third
        showDummyFile: false
```

Figure 3-17: Example usage for the fetch-template action

There's a lot going on when you click **Create** and kick off the process of scaffolding from a template. If you imagine yourself encountering a portal where this template is already available, the value of the developer portal comes into clearer focus. Think of bouncing between service UIs and auth systems to manually perform all the steps automated by the template actions, from source control in GitLab to GitOps processes to continually build from source with Argo CD and Tekton Pipelines. You can see a depiction of the services harnessed together by the template scaffolding process in Figure 3-18.

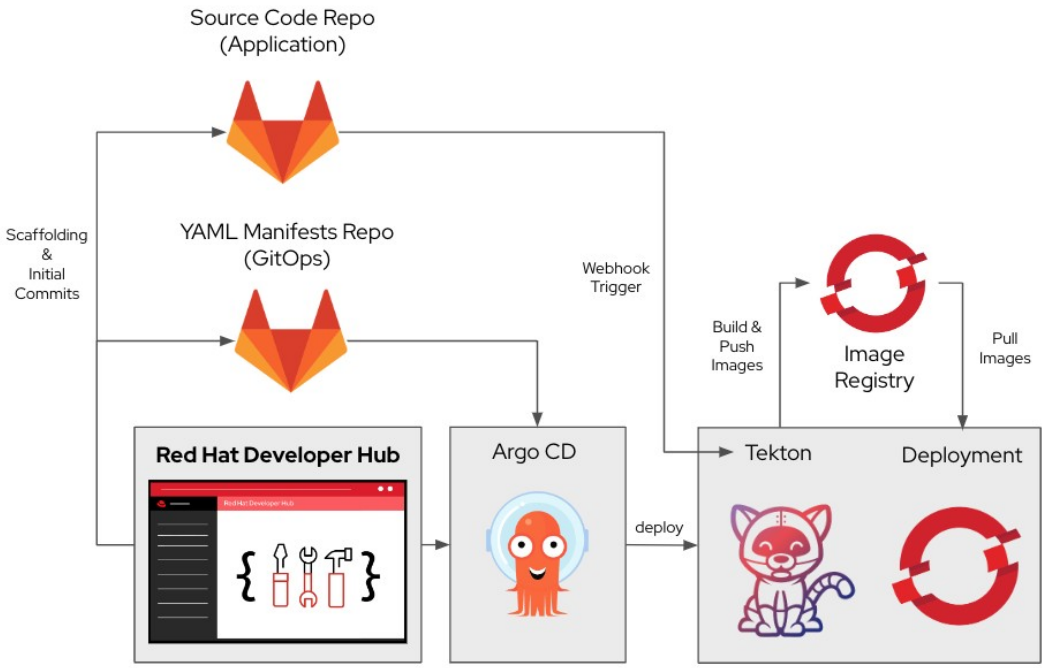


Figure 3-18: RHDH GitOps automation: Portal, GitLab, Argo CD, Tekton, OpenShift.

RHDH first reads the template contents from GitLab and then writes the scaffolded source code as well as the resulting GitOps-related repository to GitLab. Then the portal instructs Argo CD to create all the specified resources in the target Kubernetes cluster and namespace. The POI map example templates generate manifests in the form of Helm deployment charts that declare the following:

- A CI pipeline in Tekton, and a webhook event listener that is triggered on every commit to the source code repository (see the `helm/build` folder of the related GitOps repository).
- Everything needed to deploy the backend application, which in this simple case is a Kubernetes Deployment, Service, and Route, along with the database (see the `helm/app` folder of the related GitOps repository).

Template results

Continuing with the itemization of the steps automated by RHDH in its process of scaffolding resources from the backend template, take a look at the GitOps resources the scaffolder puts in place:

- Two new repositories, `demo01-poi-backend` and `demo01-poi-backend-gitops`, dividing application source from CI/CD automation resources (Figure 3-19).

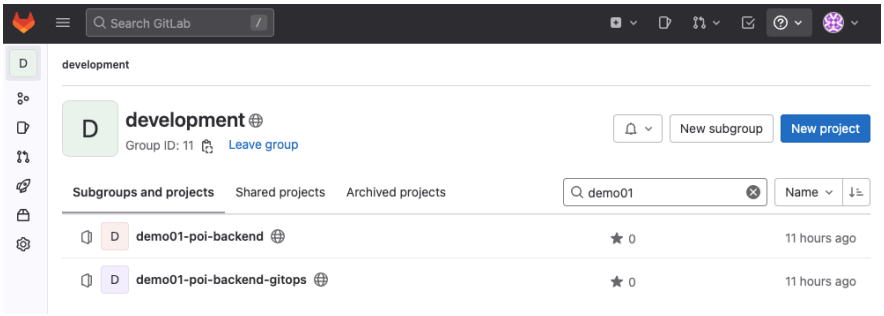


Figure 3-19: GitLab repositories for the POI backend.

- Three Argo CD "applications" (bootstrap, build pipeline, and actual application, as shown in Figure 3-20) for the build and deployment automation that form the basis of GitOps practices. These Argo applications are among the resources declared in the files beneath the `*-gitops` repo.

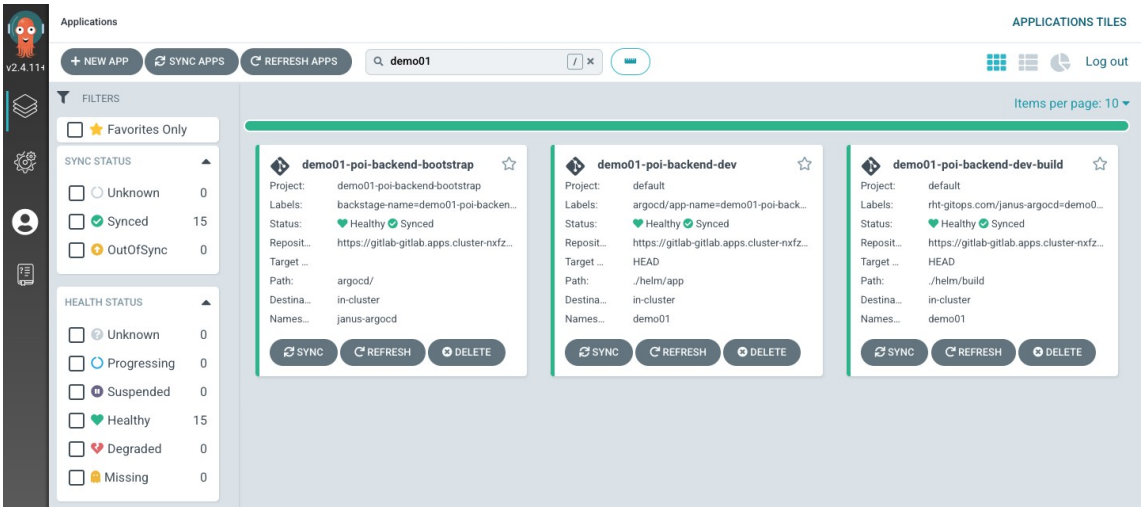


Figure 3-20: Argo CD applications for the POI backend.

- The deployed application, comprising the NestJS service and its PostgreSQL database. The build pipeline and the webhook that triggers rebuilds and redeployments when source code changes are committed also run on the deployment target cluster, seen in the OpenShift web console in Figure 3-21.

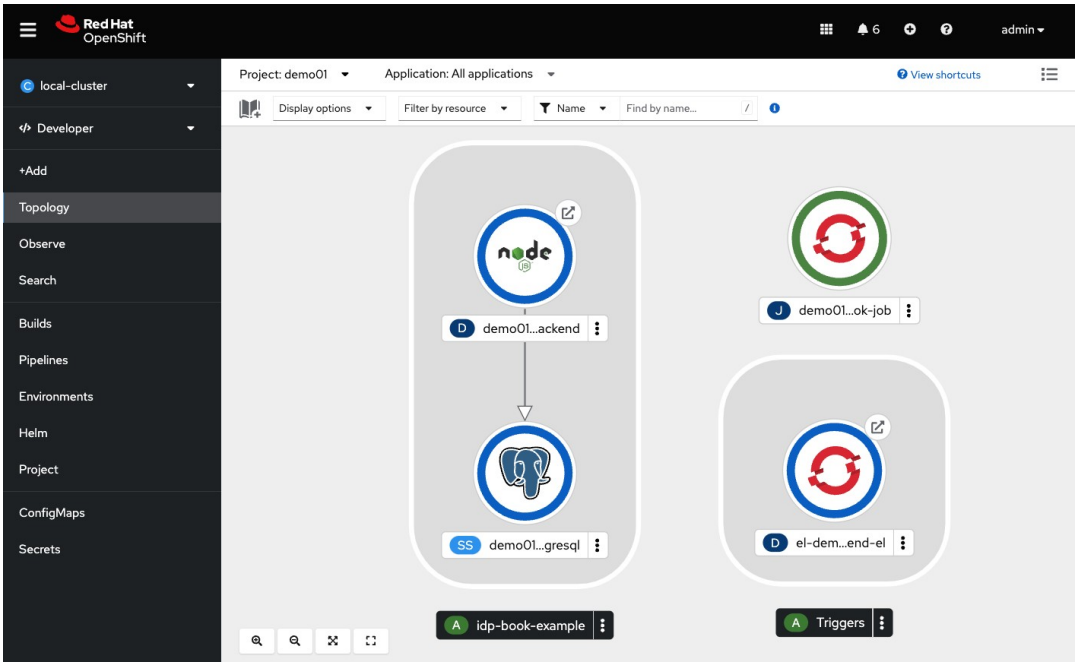


Figure 3-21: OpenShift Topology view of the POI backend.

POI backend in the Software Catalog

The template scaffolding process added an entry to the Software Catalog for the POI backend, `demo01-poi-backend`. Switch to the Catalog View in the RHDH left navigation to see it (Figure 3-22).

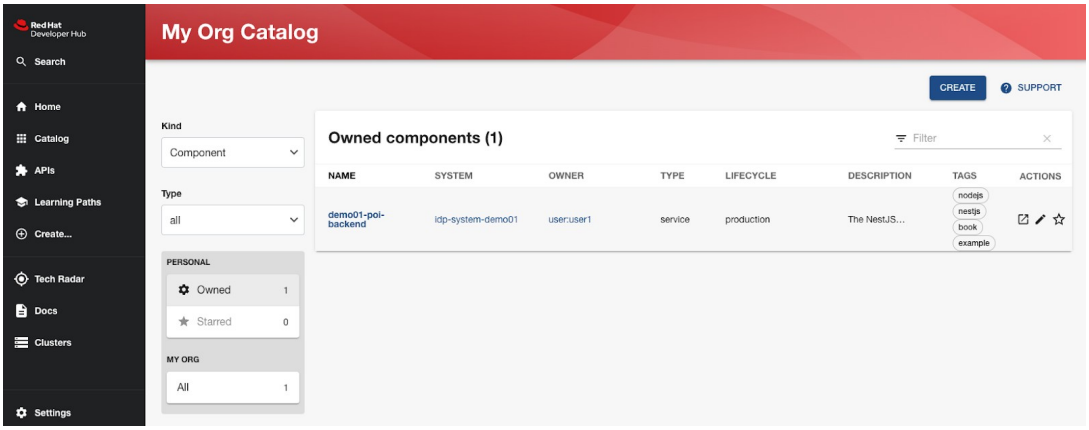


Figure 3-22: POI backend entity in the Software Catalog.

You can inspect a component by clicking on its name to open the component overview (Figure 3-23).

> **Note:** The available tabs in the Component view depend on the configuration of the RHDH instance.

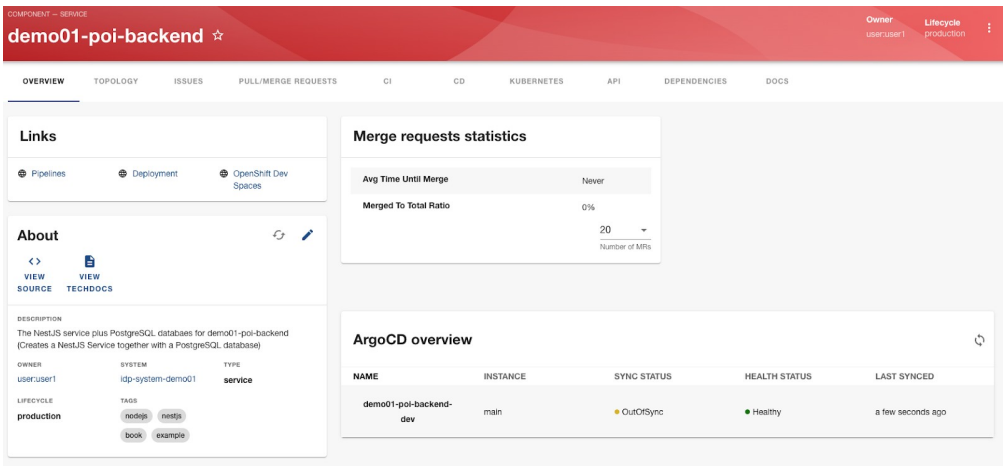


Figure 3-23: POI backend component Overview tab.

In the next sections, you will briefly visit the different tabs from that component detail view to figure out what you can learn about this registered catalog component that represents the backend service of the POI map application.

The Component screens in RHDH represent everything known about this application, derived from component metadata, information from plug-in integrations with infrastructure services, like Argo CD, Tekton, and the OpenShift (or Kubernetes) cluster where executables run.

Overview tab

The Overview tab displays a few tiles, including an About section with direct access to the source code repository (**View source**) and technical documentation (**View TechDocs**). In the upper-right corner of a Component Overview's About tile you can:

- Edit the underlying `catalog-info.yaml` contents in the corresponding Git repository to make changes to this catalog component.
- Trigger the portal to reread the component's `catalog-info.yaml` and update the component with the new configuration and metadata.

The available information displayed in the details view and its different tabs is directly and largely based on the component's `catalog-info.yaml` file. To give a simple example, the links tile holds custom component links which are found in the links section of this component's YAML definition:

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: demo01-poi-backend
```

```
  description: Creates a NestJS Service together with a PostgreSQL database
  annotations:
    argocd/app-name: demo01-poi-backend-dev
    backstage.io/kubernetes-id: demo01-poi-backend
    backstage.io/kubernetes-namespace: demo01
    backstage.io/techdocs-ref: dir:.
    gitlab.com/project-slug: development/demo01-poi-backend
    janus-idp.io/tekton-enabled: 'true'
  tags:
    - nodejs
    - nestjs
    - book
    - example
  links:
    - url: https://console-openshift-console.apps.cluster-
nxfzm.sandbox2909.opentlc.com/dev-pipelines/ns/demo01/
      title: Pipelines
      icon: web
    - url: https://console-openshift-console.apps.cluster-
nxfzm.sandbox2909.opentlc.com/k8s/ns/demo01/deployments/demo01-poi-backend
      title: Deployment
      icon: web
    - url:
https://devspaces.apps.cluster-nxfzm.sandbox2909.opentlc.com/#https://gitlab-
gitlab.apps.cluster-nxfzm.sandbox2909.opentlc.com/development/demo01-poi-backend?
che-editor=che-incubator/che-code/latest&devfilePath=.devfile-vscode.yaml
      title: OpenShift Dev Spaces
      icon: web
spec:
  type: service
  lifecycle: production
  owner: "user:default/user1"
  system: idp-system-demo01
  providesApis:
    - demo01-poi-backend-api
  dependsOn:
    - resource:default/demo01-poi-backend-db
```

Towards the end of this component definition you can see information about the owner, relationships regarding the logical grouping of components into a system, but also whether or not there are provided or consumed APIs for this component (`providesApis`), and if

dependencies to other components and/or infrastructure resources (e.g., databases, messaging queues, caches) exist (dependsOn).

Topology tab

The Topology plug-in provides a tab in the Component view showing the component's resources on a deployment target OpenShift or Kubernetes cluster (Figure 3-24). These include the usual application resources in Kubernetes API terms, such as Deployment, Job, Daemonset, Statefulset, CronJob, and Pod.

When you click on the POI backend deployment, a side pane slides in from the right to show more details. You can even retrieve logs from the container running in the pod, directly in the portal's Component view.

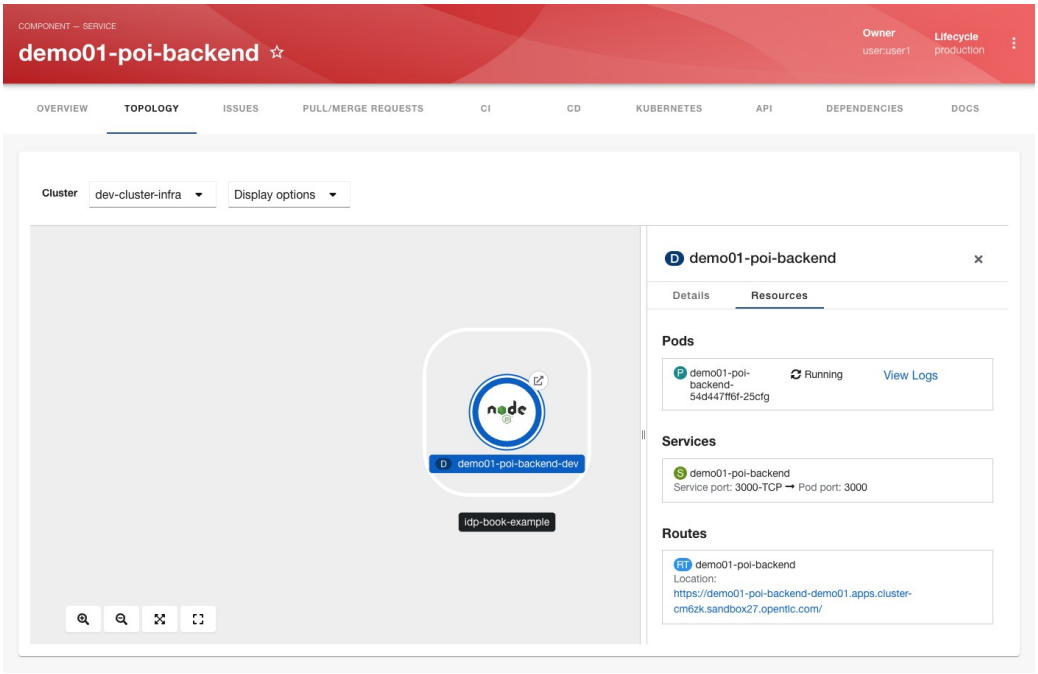


Figure 3-24: POI backend component Topology tab.

Find more information about how to install, configure and use the Topology plug-in in the Red Hat Plug-ins for Backstage documentation.

Issues and Pull / Merge Requests tabs

As a developer, you need to be aware of project issues as well as incoming changes from other developers. You can see this activity in your portal's Component views. Based on plug-ins, RHDH can integrate with various Git hosting services such as GitLab, GitHub, and others to retrieve and display issues (Figure 3-25) and pull/merge requests (Figure 3-26), respectively. Because the component is freshly scaffolded and just registered in the catalog, unsurprisingly,

both these tabs are currently empty. You will revisit them during the development of the application specific code.

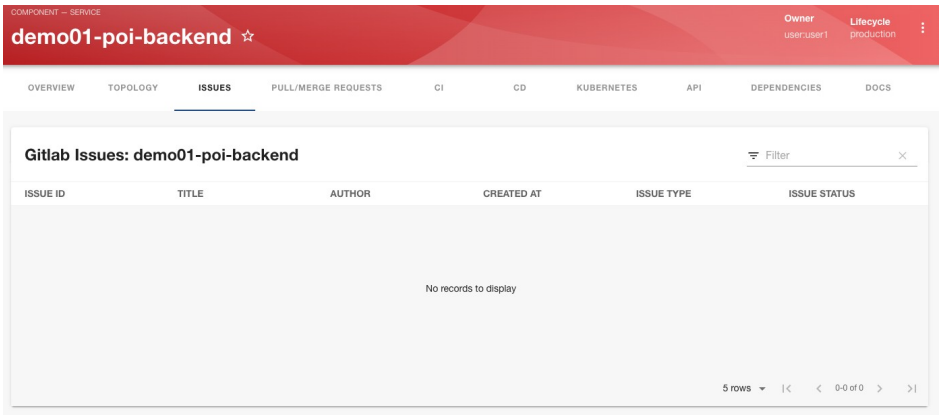


Figure 3-25: POI backend component Issues tab.

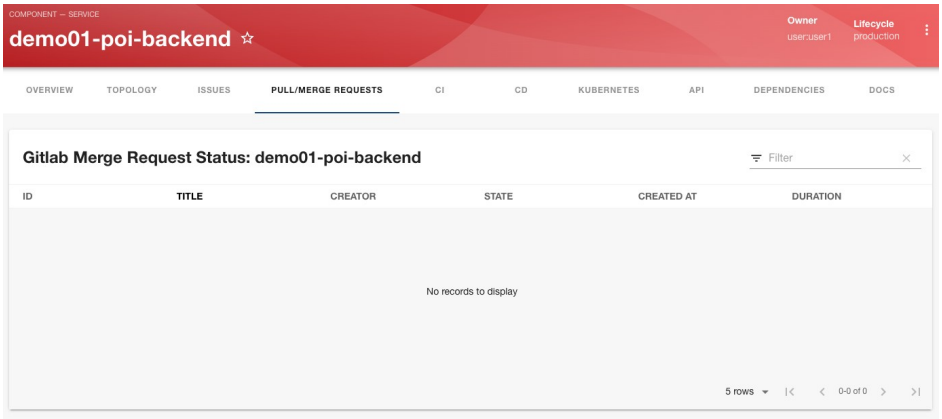


Figure 3-26: POI backend component Pull/Merge Requests tab.

CI tab

In this tab, you can explore the build pipelines for the component in question. This view isn't limited to just one type of continuous integration (CI), but if applicable, can conveniently display multiple CI-related activities for the same component. In your example, and as shown in Figure 3-27, there are two different pipelines, namely:

- A Tekton pipeline used to create the container image for the backend service.

- A GitLab pipeline that is used for building and publishing the technical documentation for the component.

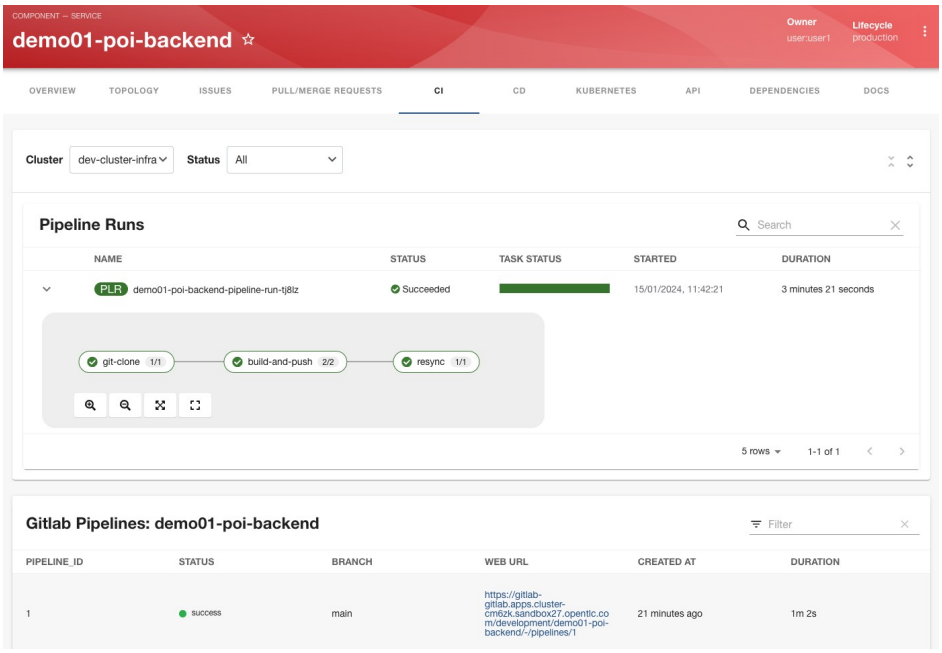


Figure 3-27: POI backend component, CI tab.

A click on any of the listed pipeline runs shows the separate pipeline steps/stages and their respective outcomes. It's also possible to retrieve the logs for each step/stage individually by clicking on it.

Find more information about how to install, configure and use this plug-in in the Red Hat Plug-ins for Backstage documentation.

CD tab

In this example, the CD related information is retrieved from Argo CD by means of another plug-in. What's shown is a basic tabular history of the Argo CD application managing the component's deployment (Figure 3-28).

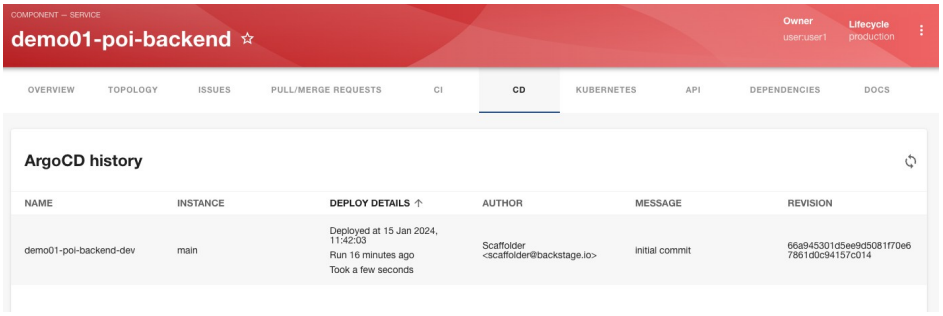


Figure 3-28: POI backend component CD tab.

Each component's Overview Tab (see Figure 3-29) also shows an Argo CD status tile that relates the sync and health status and the last synced timestamp.

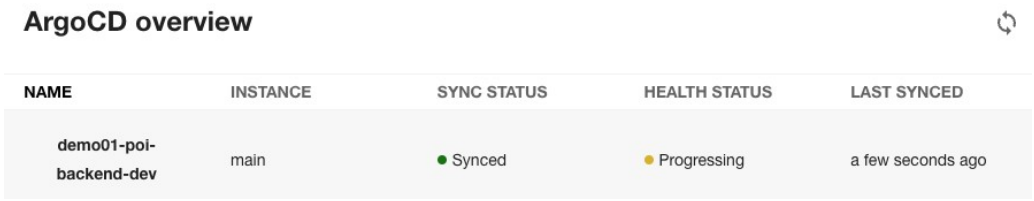


Figure 3-29: The POI backend's Argo CD tile in the Component Overview tab.

Find more information about how to install, configure and use this plug-in in the Red Hat Plug-ins for Backstage documentation.

Kubernetes tab

In this tab (Figure 3-30), you can inspect the various pods underpinning the catalog component that are running in the target Kubernetes cluster, including some workload-related details. This is very handy, in particular, when there are any issues or errors with some of these pods.

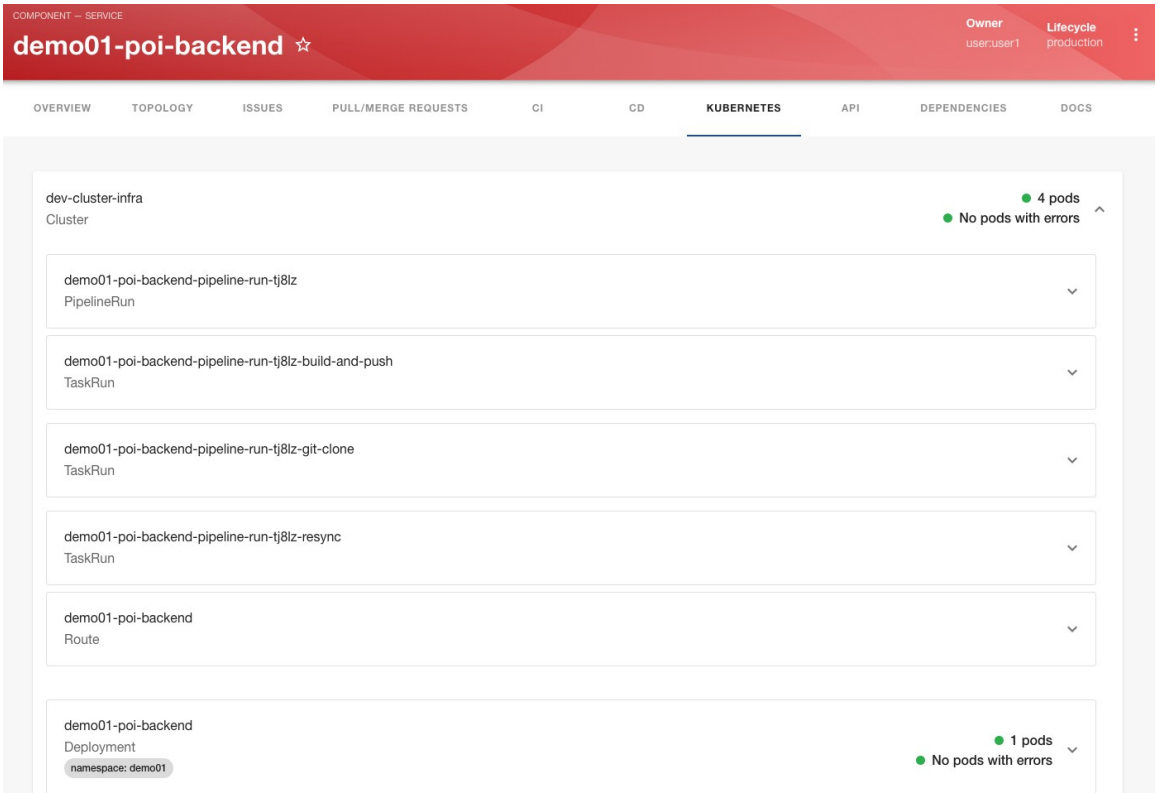


Figure 3-30: POI backend component Kubernetes tab.

API tab

Whether or not components either consume APIs from other components and/or provide APIs themselves, including API ownership information as well as system relationships if applicable, is all shown in the API tab (Figure 3-31). The scaffolded backend application provides an API that you can further investigate by clicking on its name. However, as this is currently a "Hello, World" REST endpoint, a more detailed discussion concerning API-related RHDH features follows at a later stage. After the actual API for the POI backend service has been implemented/added, we will revisit this view.

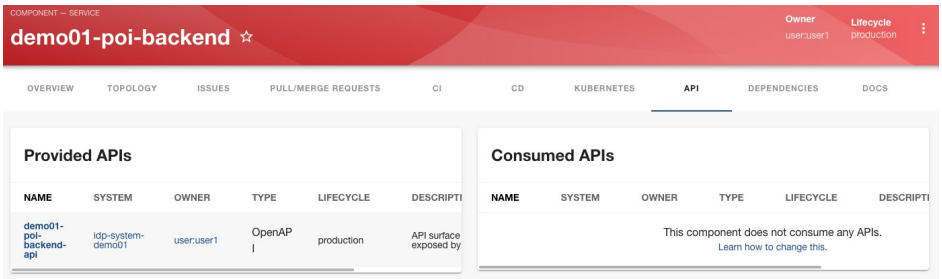


Figure 3-31: POI backend component API tab.

Dependencies tab

Components very rarely live in isolation; instead, they are often logically grouped to form a superordinate system. In addition, components can directly depend on other components or resources such databases, caches, messaging infrastructure, and the like. Thankfully, the dependencies tab provides insights into these aspects, with helpful diagrams illustrating more complex component hierarchies and/or relationships between components and resources (Figure 3-32).

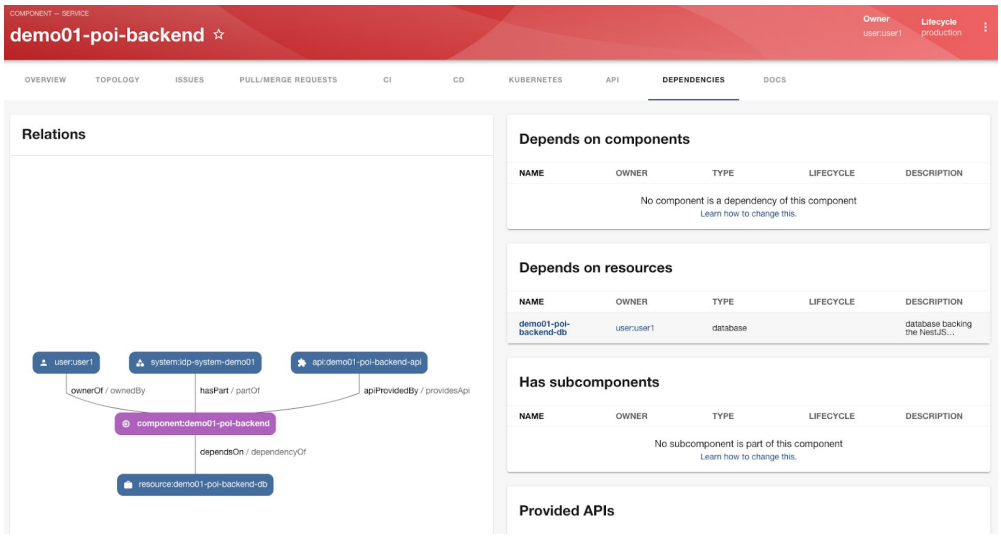


Figure 3-32: POI backend component Dependencies tab.

For the registered backend component you can see at a first glance:

- who owns it (`user1`);
- which resource—in this case, database—it depends on (`demo01-poi-backend-db`);
- that it provides an API (`demo01-poi-backend-api`);
- and that this component is part of a system (`idp-system-demo01`).

As more components will be added by means of applying further templates and by properly maintaining all these relationships in the respective `catalog-info.yaml` files during the development phase, such diagrams will grow and thus become more valuable in making sense of larger and more complex systems.

Docs tab

Having technical documentation for registered catalog components is vital. The core idea is to live a "docs-like-code" approach. Under the covers, the default way to write documentation is based on Markdown and the documentation-related files are co-located in the same

repository as the component's source code. The Docs tab (Figure 3-33) shows the latest available version of a component's rendered HTML documentation, which has been generated and published as part of the configured CI pipeline.

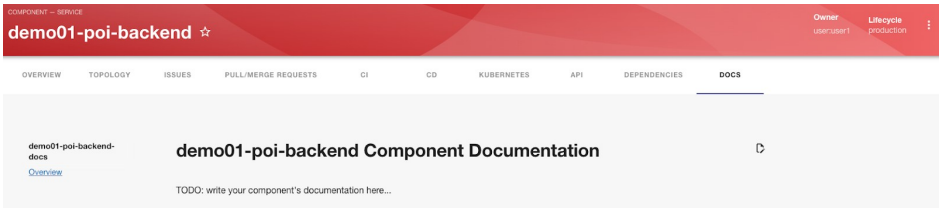


Figure 3-33: POI backend component Docs tab.

The backend component has a working TechDocs setup as configured and scaffolded during the templating phase. Two really handy features, which we'll examine more closely later during the application development phase, are:

- **Modifying the underlying Markdown file:** By clicking the edit icon in the upper right, the user is redirected to the specific Markdown file in the component's source code repository. This is where changes can directly be made. Thanks to the configured CI pipeline, any changes are triggering a new pipeline run. Once this is finished, a refresh of the documentation page in the Docs tab will show the updated contents immediately.
- **Opening a documentation-related issue:** By highlighting text on the documentation page and clicking **Open GitLab / GitHub / ... issue**, you are redirected to the Git repository issue creation page. This allows for easy creation of an issue to signal problems with the documentation to the maintainers/owners of the catalog entity.

Proxy and frontend template

Now that the backend component's source code and GitOps repositories are in place, you continue by applying the second custom template—the `quarkus-with-angular` template, which addresses the proxy and frontend parts of the POI map application. Go to **+ Create** and select the template **Quarkus Service with hosted Angular Frontend** by clicking the **CHOOSE** button in the lower-right corner of the corresponding template tile (Figure 3-34).

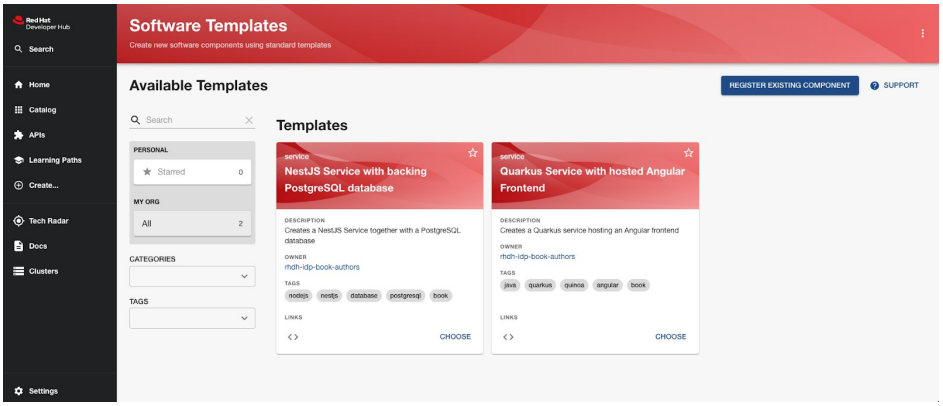


Figure 3-34: Portal Create view with template tiles.

Template form wizard

This brings you into the template's form wizard where you can parameterize certain elements of the template in question. You've been here before when scaffolding the backend component from the first template. What you are going to do here is strikingly similar.

In the first section of this form (Figure 3-35), you define information about the GitLab location used for publishing the resulting source code and GitOps repositories—leave the defaults as-is.

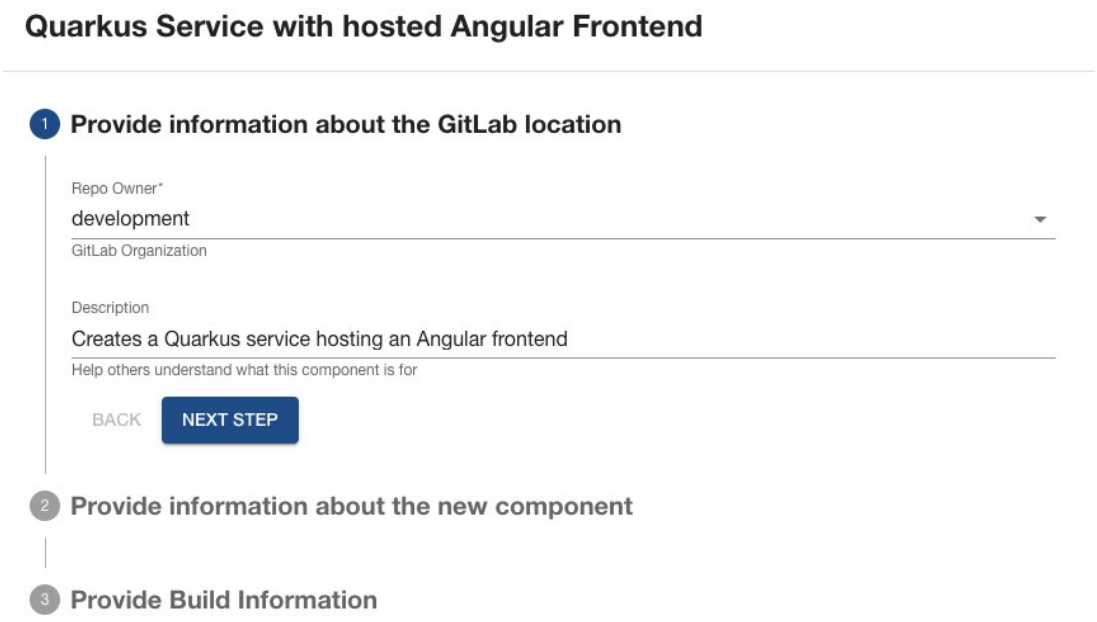


Figure 3-35: Quarkus Service with Angular template configuration form, first section.

In the second section (Figure 3-36), you specify important settings, namely the cluster ID, namespace, application ID, and owner for our new software component. Based on this

information it's clear into which Kubernetes cluster and namespace this component is eventually going to be deployed with the entered application ID as its name. The selected user defines the ownership for this software component.

> **Note:** It's important to make sure that you use the same namespace and cluster ID as for the backend template you applied earlier (see Template form wizard on page 26).

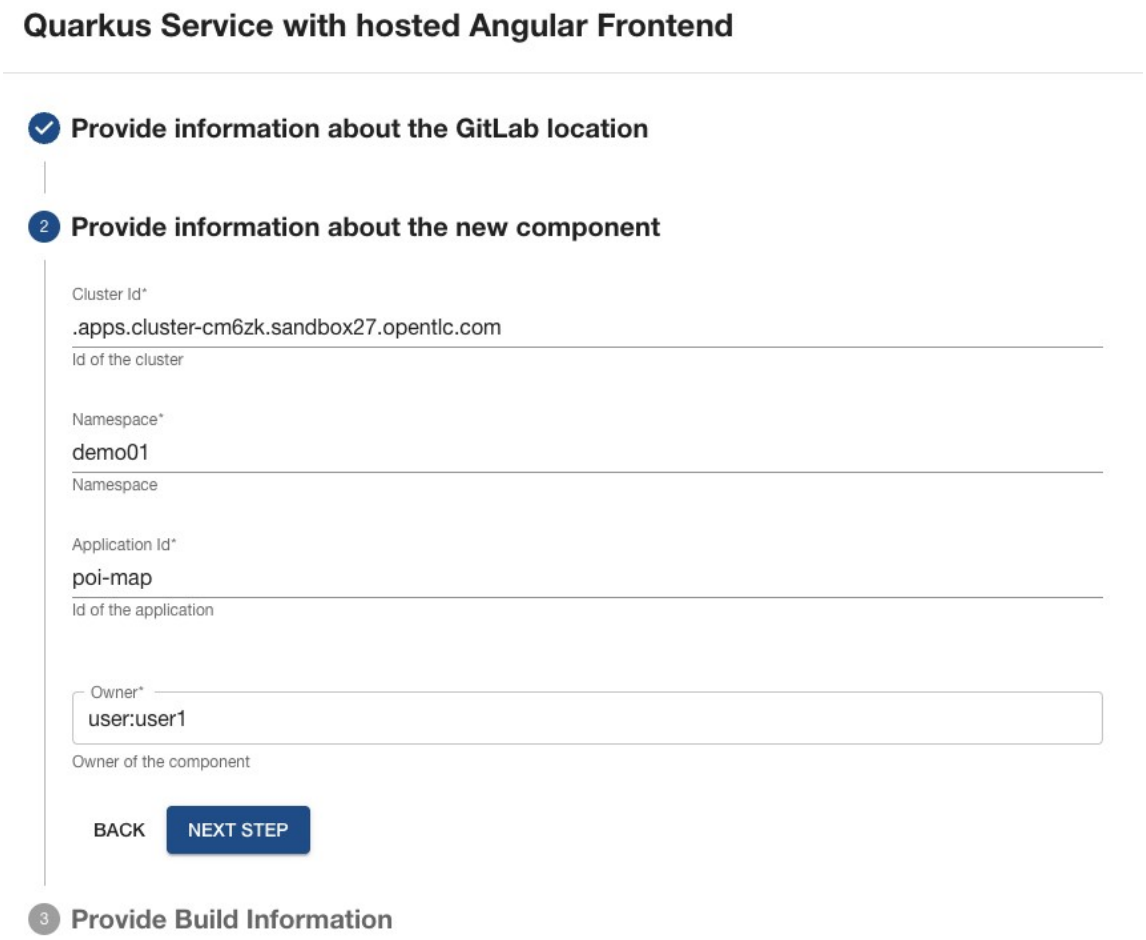


Figure 3-36: Quarkus Service with Angular template configuration form, second section.

In the third step (Figure 3-37), it's defined which image registry to use for pushing the container image to. You can also choose a custom tag that will be used during the CI process to tag the container image with.

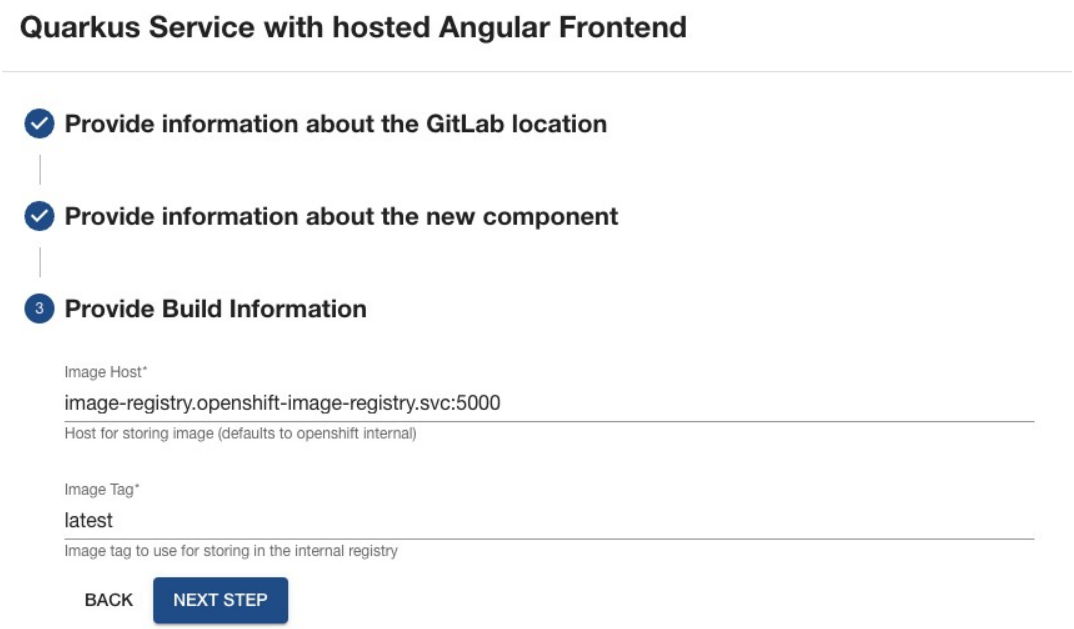


Figure 3-37: Quarkus Service with Angular template configuration form, third section.

Clicking **NEXT STEP** shows a summary of all the entered form fields for a final review (Figure 3-38). Finally, hit **CREATE** to kick off the actual templating process (Figure 3-39).

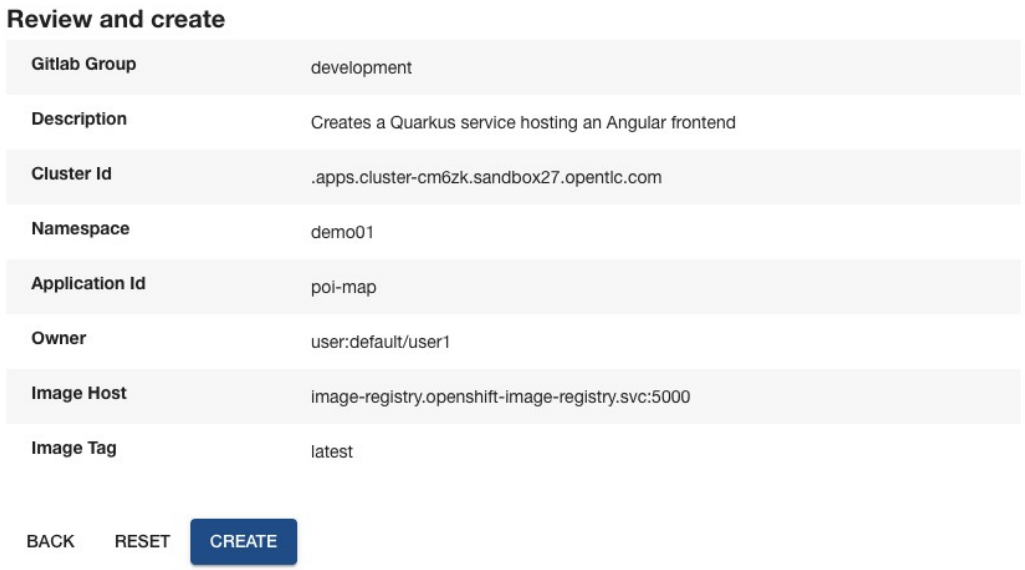


Figure 3-38: Quarkus Service with Angular template configuration form, final review.

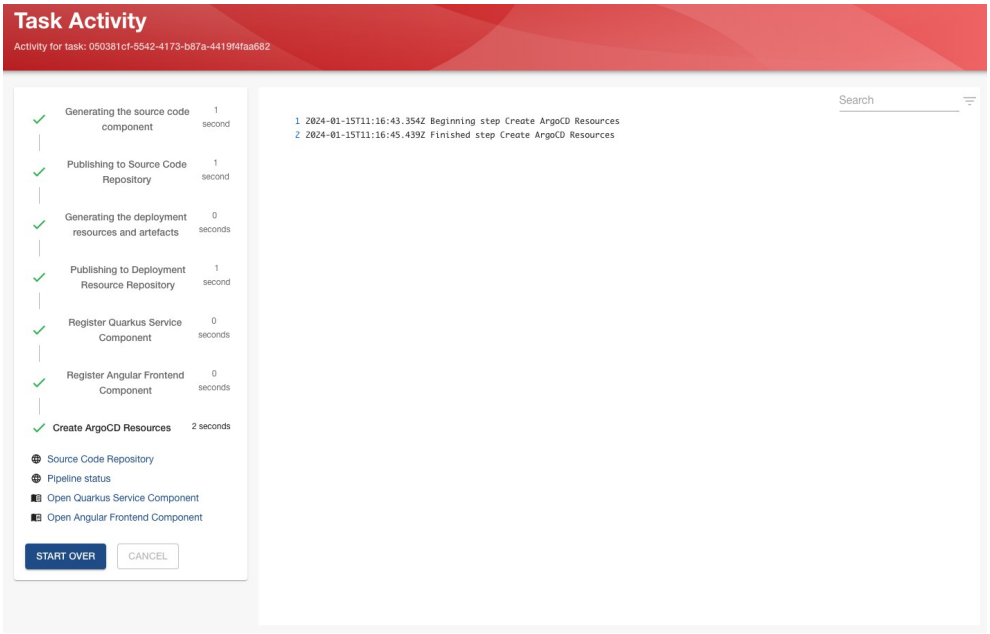


Figure 3-39: Quarkus Service with Angular template scaffolding results.

Template scaffolding process

Insights into what's happening during the scaffolding process under the covers have been provided in detail for the backend template already. Feel free to revisit the Template scaffolding process section on page 28 if necessary.

Template results

As a result of applying this template, you end up with the following resources created on your behalf:

- In GitLab you have two new repositories, namely `demo01-poi-map` and `demo01-poi-map-gitops` (Figure 3-40).

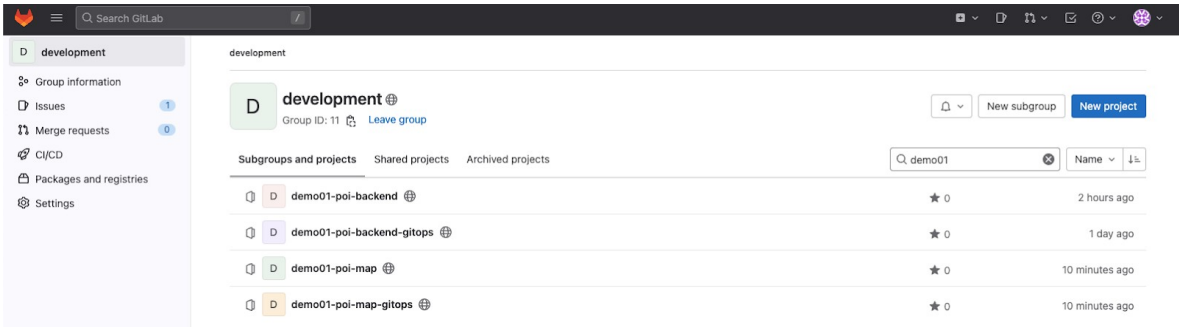


Figure 3-40: GitLab repositories for the POI frontend.

- In Argo CD, there are 3 new “applications” (bootstrap, build pipeline, actual application) for GitOps, as shown in Figure 3-41.

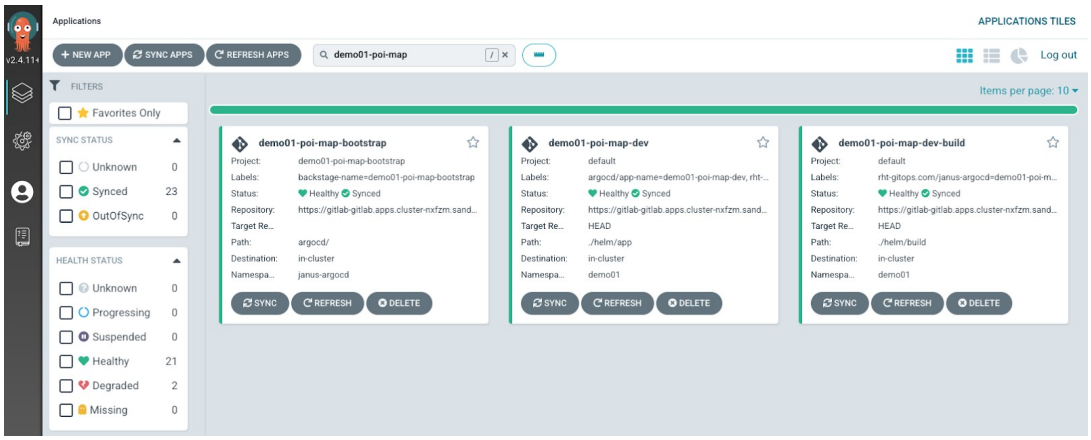


Figure 3-41: Argo CD applications for the POI frontend.

- In Kubernetes, there is the deployed Quarkus proxy application, which also serves the Angular single page application frontend. Additionally, the build pipeline and the webhook related resources have been set up. See Figure 3-42.

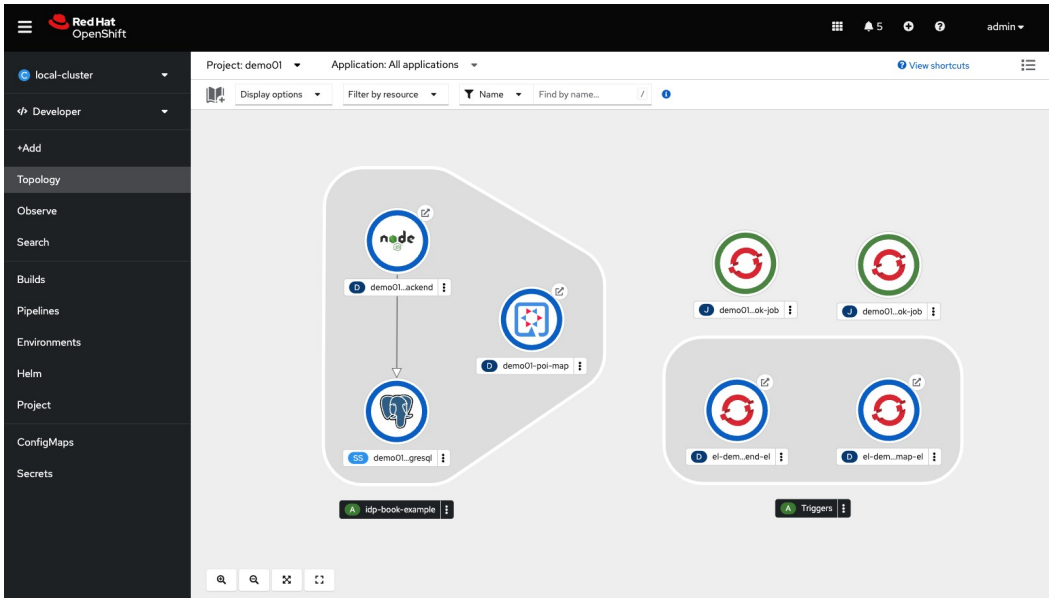


Figure 3-42: OpenShift Topology view with the POI backend and frontend.

POI proxy service and frontend in the Software Catalog

In contrast to the backend template, the major difference with this template is the fact that you end up with two separately registered catalog components. This is intentional, and results from the “monorepo” approach that has been chosen when creating the template for this proxy service that embeds the frontend application.

Switch to the Catalog View in Developer Hub to find and explore your two new software catalog components, which are `demo01-poi-map-frontend` (Angular Single Page Application) and `demo01-poi-map-service` (Quarkus proxy), as shown in Figure 3-43.

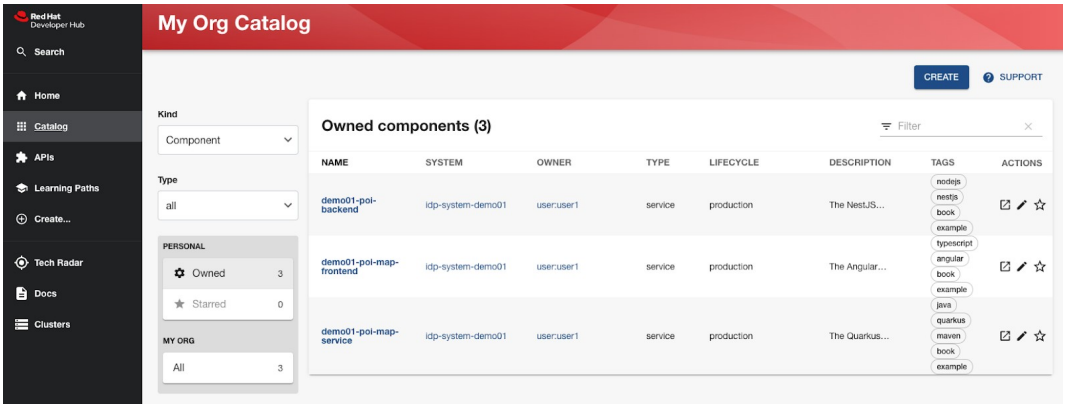


Figure 3-43: POI proxy service and frontend entities in the Software Catalog.

You can inspect various details and important information about both these components by clicking on their respective names in the tabular listing to open a specific component's detail view.

Just to clarify, of course it would work to treat both these components completely separately. However, we've chosen this architecture for the proxy service and the frontend application paired with the monorepo approach in order to show RHDH's flexibility of working with templates and software catalog components in different ways.

> **Note:** Keep in mind that the actual possibilities, the available tabs, and the tiles anywhere in that detail view primarily depend on the configuration of the RHDH instance, the installed plug-ins as well as any component view customizations which might or might not be in place for your environment.

Because you already visited the different tabs for a component's detail view earlier while exploring the `demo01-poi-backend` component (see Error: Reference source not foundPOI backend in the Software Catalog on page 33) of the POI map application, this should all look very familiar to you at that stage.

Summary

You've explored the basics of how Software Templates are defined, then used two example templates to deploy the foundation atop which you'll implement the POI map application. In your running portal, the Software Catalog lists three new entities indexing all of the resources

of your development project, from source code to the executing components of the application.

In the next chapter, you'll add code to the scaffolding provided by the templates to create a functional POI map.

Chapter 4: Building the POI map

You've used a couple of Software Templates to put everything in place to get a new application going. Now you can implement your application features with actual code. Although you've peered inside a template and registered it in your portal, often your usual work would begin nearer to this point, making changes to existing entities indexed in your Software Catalog.

Implement the backend

You scaffolded the POI backend from the `nestjs-with-postgres` template in the last chapter. This included a skeletal source code repository. So far, the repo contains only a simple "Hello, World" NestJS application.

Adapt database configuration

Because the backend service will use a database table named `poi` with a specific table structure, you need to provide a SQL DDL (Data Definition Language) snippet to configure Helm's provisioning of the PostgreSQL database.

Open the backend's GitOps repository, `demo01-poi-backend-gitops`, in GitLab. As the change to be made is a rather trivial one, you might take a shortcut here rather than going through a full development cycle including feature branches and merge requests. Edit the Helm chart's `values.yaml` file directly in GitLab to add the following DDL snippet:

```
    initdb:
      scripts:
```

```
    db_init.sql: |
      -- CreateTable
      CREATE TABLE "Poi" (
          "id" BIGSERIAL NOT NULL,
          "name" TEXT NOT NULL,
          "description" TEXT,
          "latitude" DOUBLE PRECISION NOT NULL,
          "longitude" DOUBLE PRECISION NOT NULL,
          "type" TEXT NOT NULL,

          CONSTRAINT "Poi_pkey" PRIMARY KEY ("id")
      );
```

To do so, browse to the file `helm/app/values.yaml` in the GitLab repository `demo01-poi-backend-gitops` and append the preceding snippet (Figure 4-1). This will ensure that the database table your backend service needs gets created during the database bootstrapping phase. With the DDL in place, click the **Commit changes** button to confirm your change.

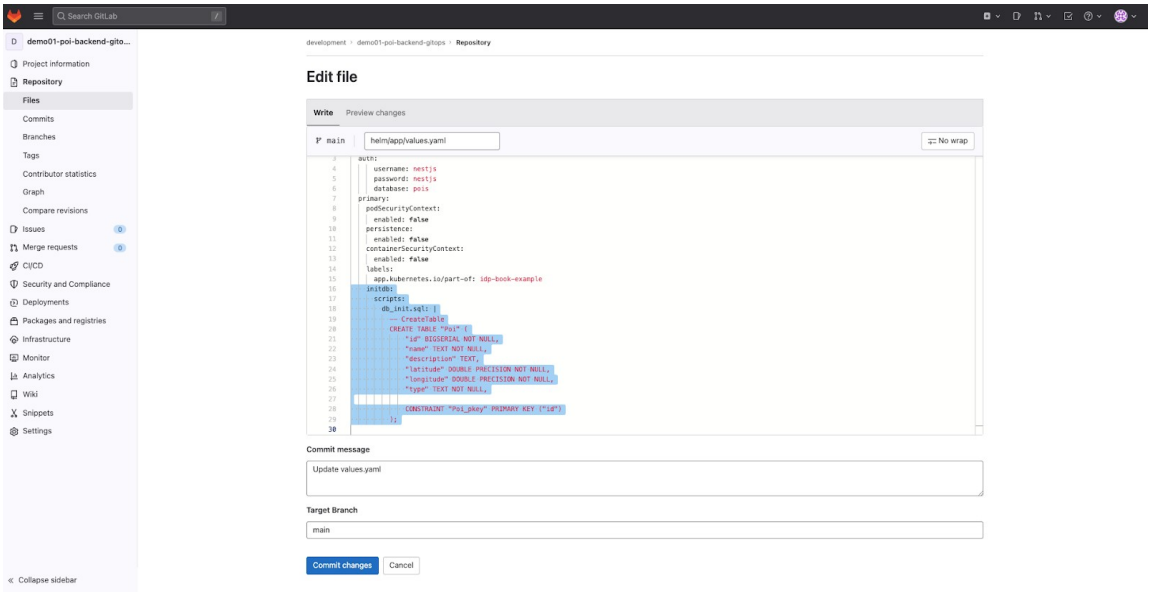


Figure 4-1: Edit Helm chart's values.yaml in GitLab to add SQL init script.

This change in the GitOps repository will eventually trigger another build pipeline run and consequently also lead to a redeployment of the postgres database instance by Argo CD.

Write the code

Click **Catalog** in the RHDH left-hand navigation and select the `demo01-poi-backend` component. In the component's details view, jump to the corresponding GitLab source code repository by clicking the **View Source** link in the About tile of the Overview tab. From the GitLab page of the repository (Figure 4-2), use the **Clone** button to create a local clone for developing the application code.

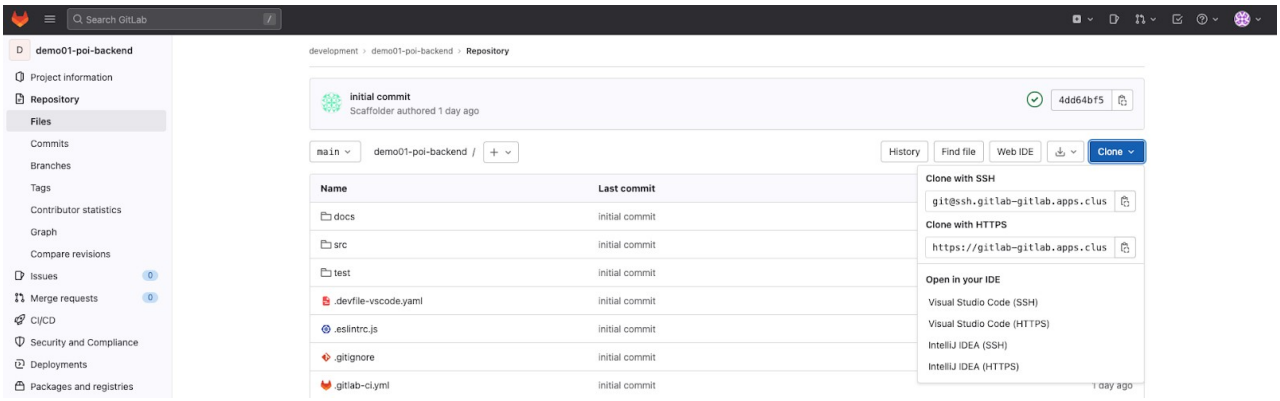


Figure 4-2: POI backend repository in GitLab.

After cloning this repository, open the NestJS project in the IDE of your choice. At this point, you could start to develop the code for the application in question.

We provide an implementation for this backend service in a repository in the `developer-hub-books` GitHub organization.

You are going to merge the sample code into the repository of your scaffolded backend application. From within the root folder of your cloned Git repository, run the following commands at a terminal prompt in your IDE or OS:

1. Add the sample code repository as another Git remote called `solution`:

```
git remote add solution
https://github.com/developer-hub-books/rhdh-book1-sample-code-poi-map.git
```

2. Fetch the contents from this new Git remote: `git fetch solution`
3. Create a new Git branch named `app-dev`: `git checkout -b app-dev`
4. Merge the sample code into this new Git branch:

```
git merge --allow-unrelated-histories --strategy-option theirs -m "merge
provided sample code" solution/main
```

5. From the project repository's root folder, open the file `openapi.yaml`, scroll down to line 97 in this file, and **specify the proper server URL** (see TODO section in the following code snippet) for this OpenAPI spec.

```
servers:
  #------------------------------------------------------------------
  # TODO:
  #  Add the secure https route URL pointing to your backend service into
the setting
  # below. You can find the route URL in RHDH by inspecting the component's
details in
  # the topology view. If you used the suggested namespace ('demo01') and
component id
 # ('poi-backend') it should look like this
  # https://demo01-poi-backend-demo01.<cluster_id_here>
  #   Beware that this must be a secure HTTP URL that starts with
'https://'
  - url: <HTTPS_ROUTE_URL_TO_YOUR_BACKEND_SERVICE_HERE>
    description: (RHDH component deployment)
  #------------------------------------------------------------------
```

   It's important that this server URL matches the respective HTTPS route URL that was created during the initial deployment of the backend service. Save this file after applying the modification.

6. Stage all changes and commit your work to the `app-dev` branch:
   `git commit -am "implement poi backend"`
7. Push this branch to the `origin` remote (i.e., your POI backend repository):
   `git push origin app-dev`
8. In the GitLab web UI, create a new merge request for your `app-dev` branch (Figure 4-3).

Once the merge request has been created, you can switch to the component's details view in RHDH and select the **Merge Requests** tab, which should now show the merge request you just created (Figure 4-4). Back in GitLab, you can merge this merge request (Figure 4-5).

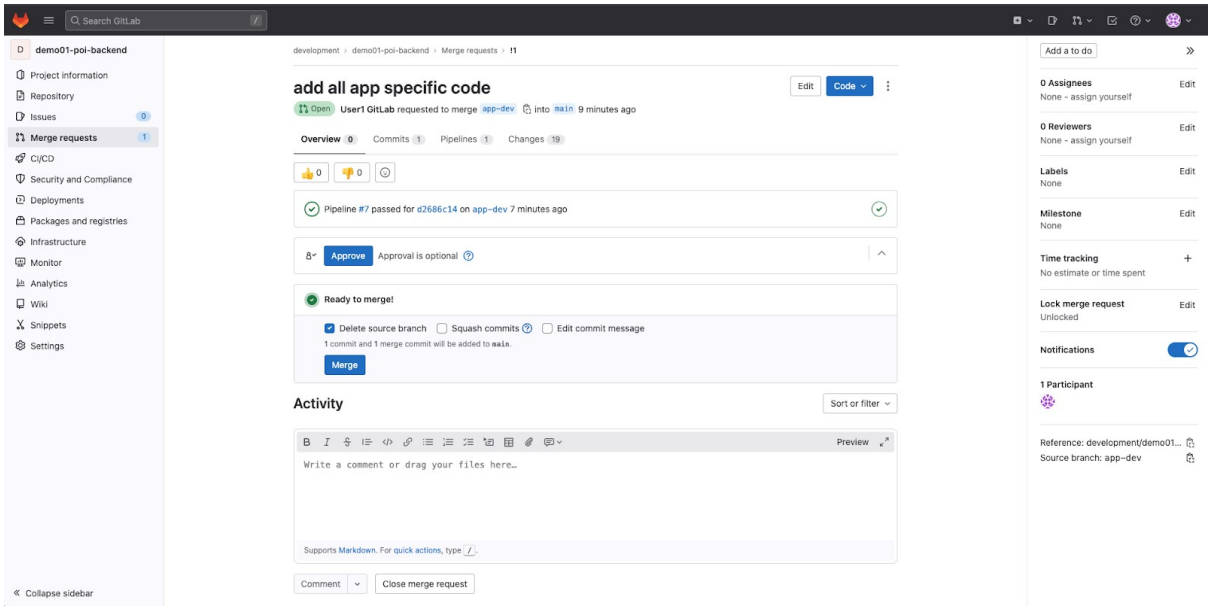


Figure 4-3: GitLab open merge request for app-dev branch.

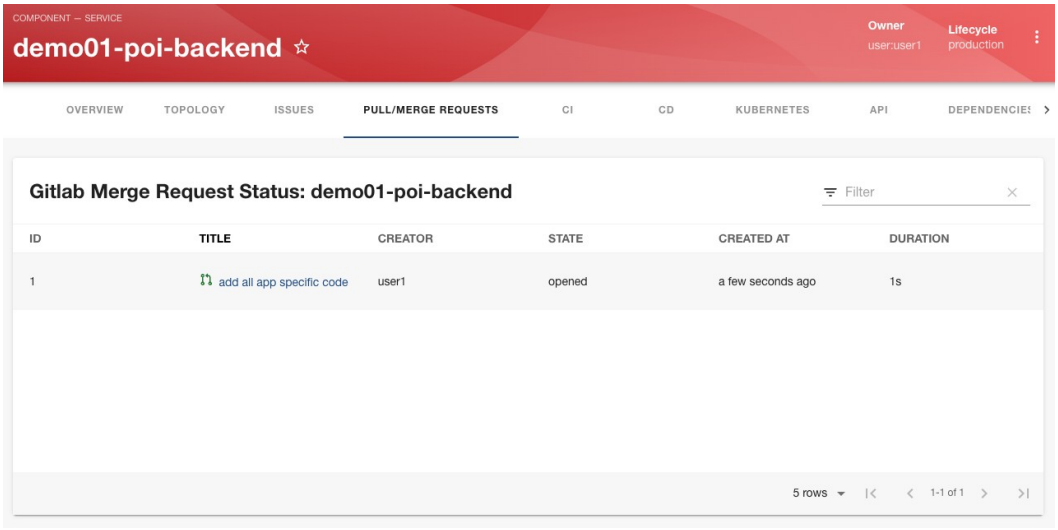


Figure 4-4: POI backend component view Pull/Merge Requests tab in RHDH.

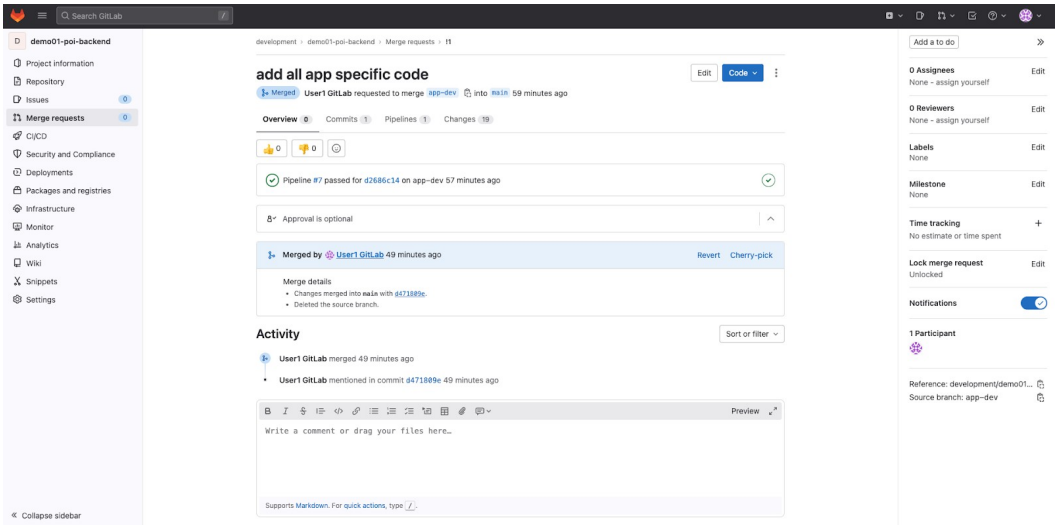


Figure 4-5: GitLab-merged merge request for app-dev branch.

Check CI/CD

The merged code will trigger the configured build pipeline via a webhook. After a minute or so, the code changes are available in the freshly built container image for your backend service (Figure 4-6).

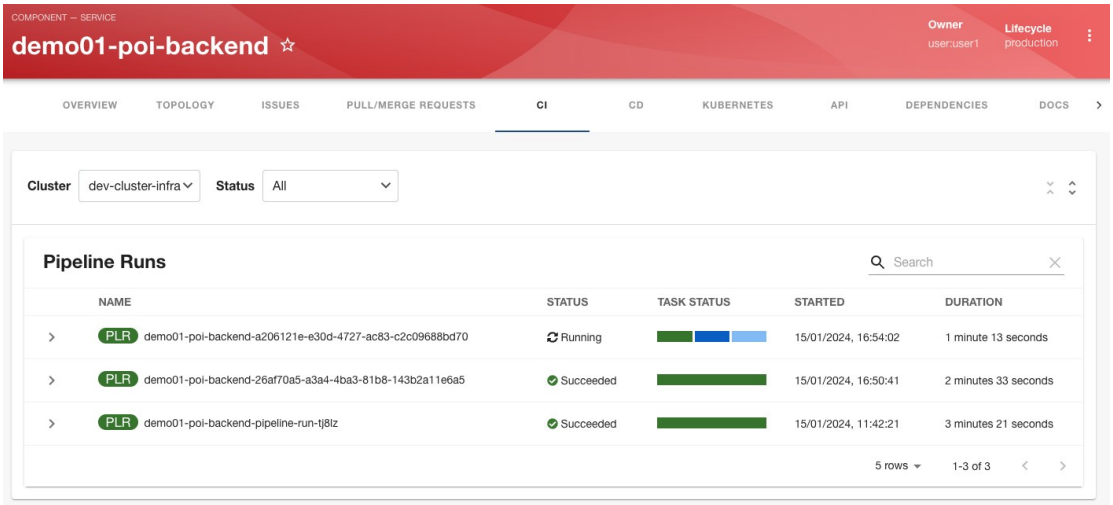


Figure 4-6: POI backend component view Tekton tab in RHDH.

Check backend app status

With the necessary application changes done, check if your backend service is running correctly. Open the `demo01-poi-backend` in the RHDH catalog component view, switch to the **Topology** tab, click the deployment, and select **Resources** from the right-side pane (Figure 4-7).

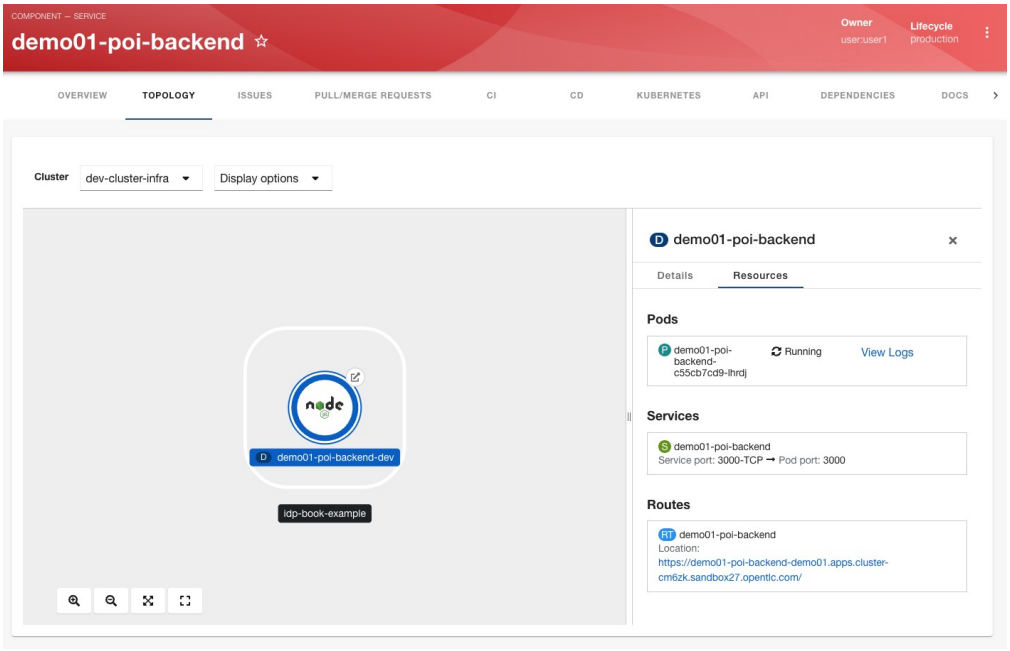


Figure 4-7: POI backend component Topology tab deployment resources.

The pod appears to be running fine. You can explore the logs by clicking **View Logs** next to the running pod information. You should see a log output similar to the one in Figure 4-8.



Figure 4-8: POI backend component pod logs view.

This indicates that the backend service should be up and running without any problems. After closing the logs, you can click the **Routes** link, which will open a new browser tab. Because the backend service isn't serving anything on the "/" path, the error message shown in the new tab is expected:

```
{"message":"Cannot GET /","error":"Not Found","statusCode":404}
```

By appending `/ws/info` to the end of the current URL, you should see the following response:

```
{"id":"poi-backend","displayName":"National Parks","coordinates":
{"lat":0,"lng":0},"zoom":3}
```

If you change the end of the URL to the path `/poi/find/all`, you should see a response with plenty of JSON output for all the loaded points of interest from the database: national parks across the world.

Explore the application's API

Your NestJS backend service offers a Swagger UI based on the underlying OpenAPI specification. You can inspect the exposed REST API by switching to the API tab in the catalog component's detail view and then clicking on the one entry, `demo01-poi-backend-api`, in the Provided APIs table, as shown in Figure 4-9.

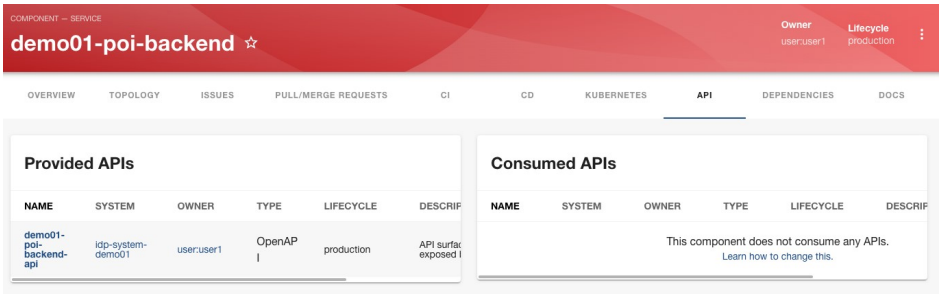


Figure 4-9: POI backend component API tab.

In the API view, there is a Links tile in the Overview tab (Figure 4-10) that has two entries:

- **Swagger UI:** A direct link to the Swagger UI as served by the running backend service.
- **API Spec:** A direct link to this API's underlying `openapi.yaml`, which resides in the component's source code repository.

Clicking the Swagger UI link opens a new browser tab to inspect and experiment with the exposed API methods from the Swagger web UI (see Figure 4-11).

Clicking the **API Spec** link opens the GitLab repository showing the `openapi.yaml` file (Figure 4-12).

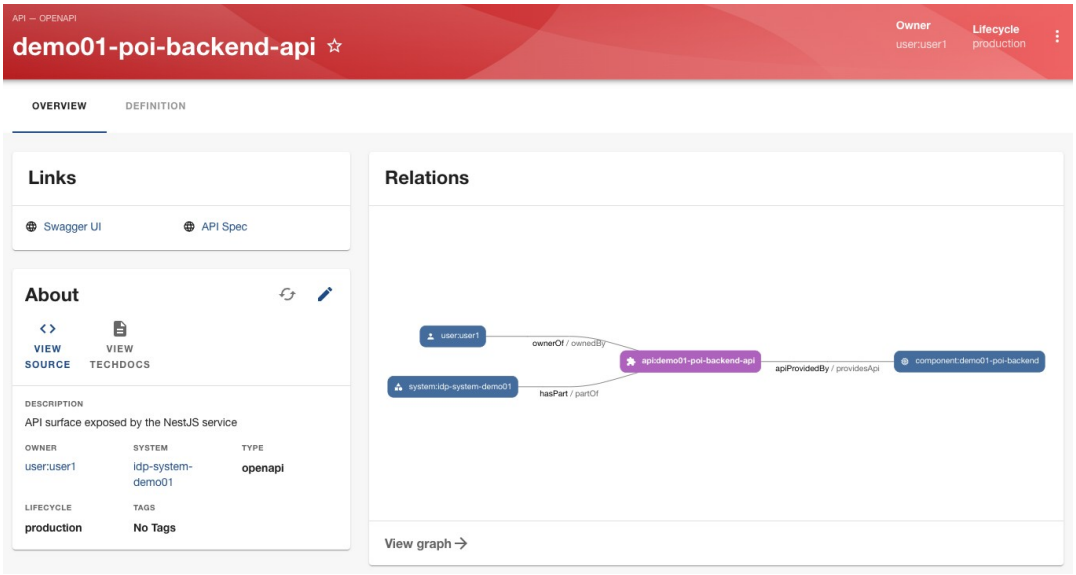


Figure 4-10: POI backend API Overview tab.

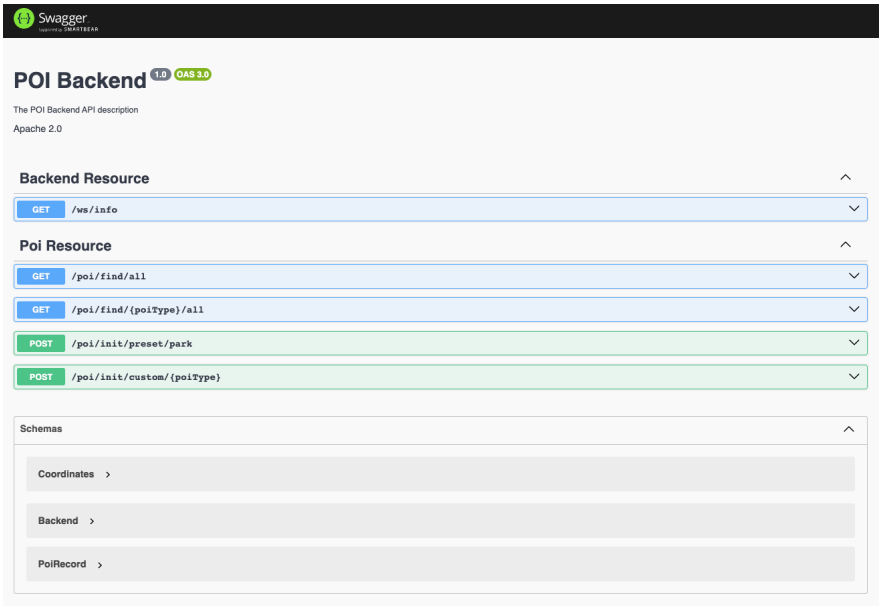


Figure 4-11: POI backend API Swagger UI.

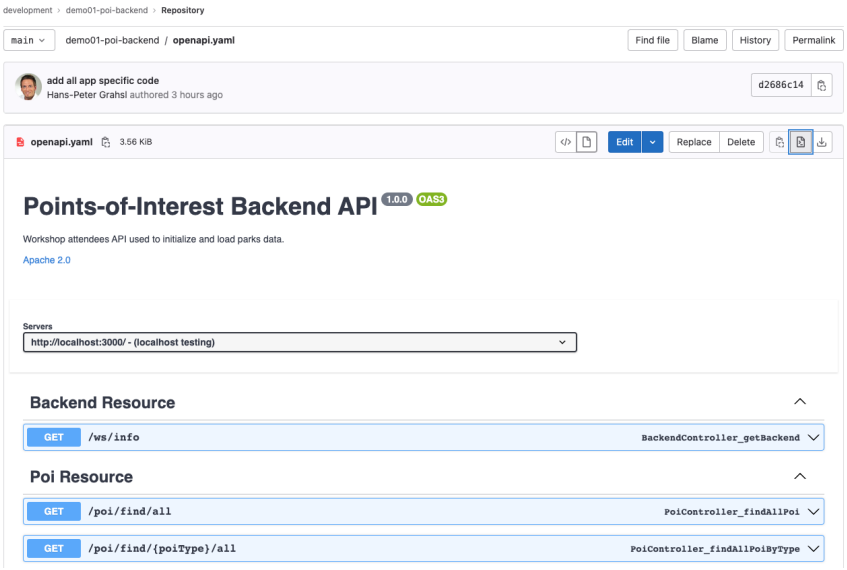


Figure 4-12: GitLab Swagger UI for OpenAPI spec of POI backend.

You can also view the file contents (Figure 4-13) rather than the rendered Swagger UI in GitLab by clicking **Open Raw** in the upper-right corner.

```
---
openapi: 3.0.0
paths:
  /ws/info:
    get:
      operationId: BackendController_getBackend
      parameters: []
      responses:
        '200':
          description: get information about this registered backend
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Backend'
      tags:
        - Backend Resource
  /poi/find/all:
    get:
      operationId: PoiController_findAllPoi
      parameters: []
      responses:
        '200':
          description: get all points-of-interest irrespective of the specific poi type
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/PoiRecord'
      tags: &ref_0
        - Poi Resource
```

Figure 4-13: GitLab raw file view for openapi.yaml definition of POI backend.

Add the documentation

Now that the application is up and running, you can shift your focus to another important aspect: documentation. You learned in the Applying Templates section that everything necessary to follow a “docs-like-code” approach is preconfigured and in place from the beginning. This means you can fully focus on writing documentation itself. No need to worry or explicitly care about generating and publishing documentation updates based on changed documentation sources.

To see how convenient it is to add new documentation to your backend component, open the `demo01-poi-backend` component detail view in the RHDH catalog and click on the **View TechDocs** link in the About tile of the component's Overview tab. This opens the current version of the backend service documentation (Figure 4-14).

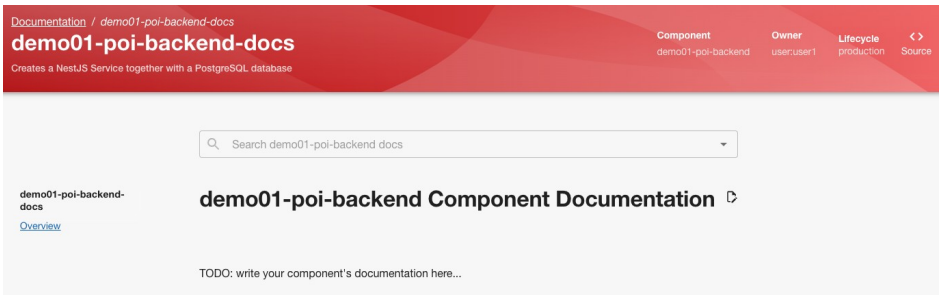


Figure 4-14: POI backend component tech docs.

It's immediately apparent that what you are reading still reflects the documentation as originally scaffolded during the templating phase of this component. You can fix that right away and write some useful documentation by clicking the **Edit this page** icon in the upper-right corner.

This brings you to the source code repository, directly into GitLab's edit mode for the underlying Markdown file of this very documentation page. You might want to come up with some words on your own, or you can use the following exemplary Markdown:

```
# POI Backend Component Documentation

The POI backend component represents a web service written in [TypeScript]
(https://www.typescriptlang.org/) with [NestJS](https://nestjs.com/) that serves
points of interest data records from a [PostgreSQL](https://www.postgresql.org/)
database.
```

Copy and paste this into GitLab's editor for the `docs/index.md` file as shown in Figure 4-15 and confirm the change by clicking the **Commit changes** button.

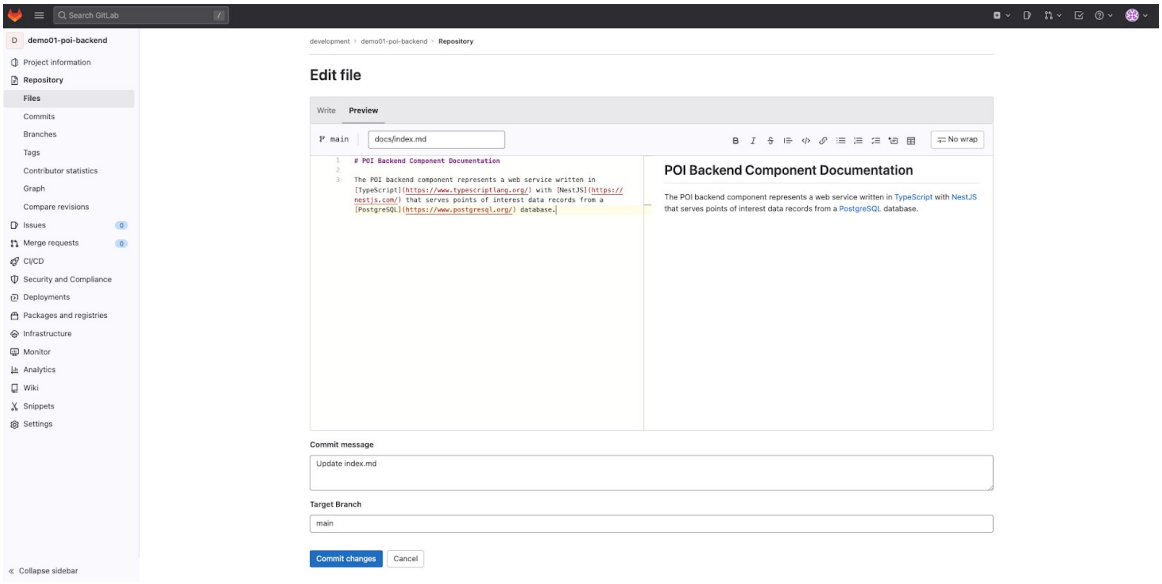


Figure 4-15: GitLab edit file view for docs/index.md.

This code change will trigger a GitLab CI pipeline run (Figure 4-16), which will generate and publish the updated documentation. Check the pipeline and give it a few moments to run.

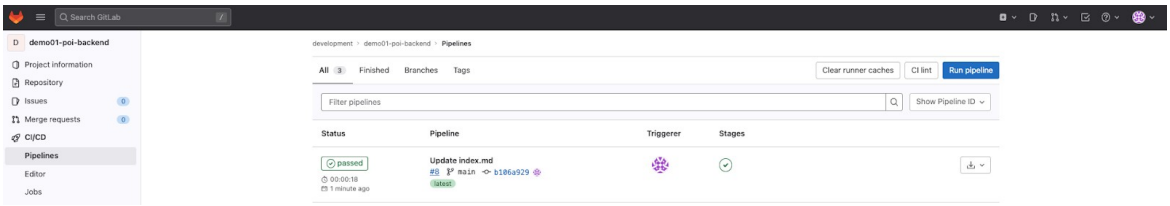


Figure 4-16: GitLab CI pipeline run due to TechDocs changes.

Once the pipeline has successfully finished, switch back to the browser tab showing the RHDH component view. Reload the page in order to see the rendered HTML view with the new documentation based on the update you just committed (Figure 4-17).

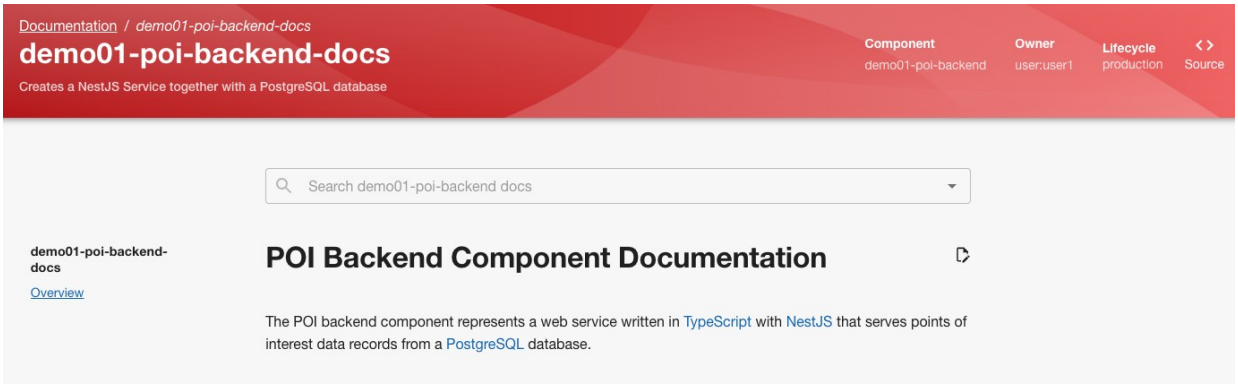


Figure 4-17: POI backend component updated TechDocs.

If you want to create multiple files, introduce a folder hierarchy, or add images and illustrations to your documentation, we recommend that you write the documentation locally in your Markdown editor or IDE of choice. This allows you to create a separate branch and also rely on merge requests, including reviews for everything you wrote, similar to the workflow used in the Write the code section earlier (see page 52) for implementing the backend component.

Another nice TechDocs feature in RHDH is the ability to raise documentation-related issues as you're reading, right from the respective docs page in question. All you need to do is highlight the text on the page and wait a moment for a tooltip labeled **Open new GitLab issue** to appear (Figure 4-18).

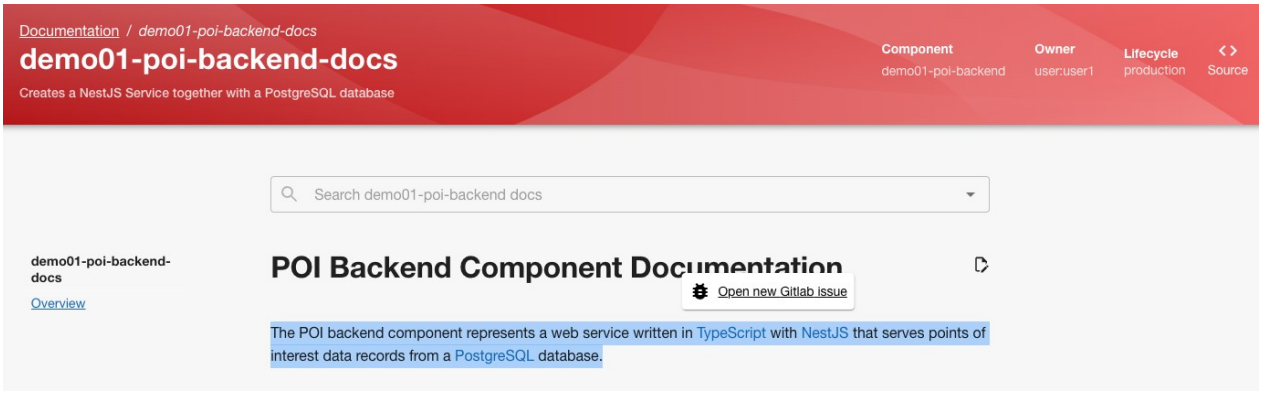


Figure 4-18: Opening a new TechDocs issue.

Clicking the link in the tooltip will take you directly to a GitLab issue creation page. Users can then report issues they encounter as they try to make sense of the existing documentation. It's pretty intuitive to use; you state the problem right below the "The comment on the text" section, as shown in Figure 4-19.

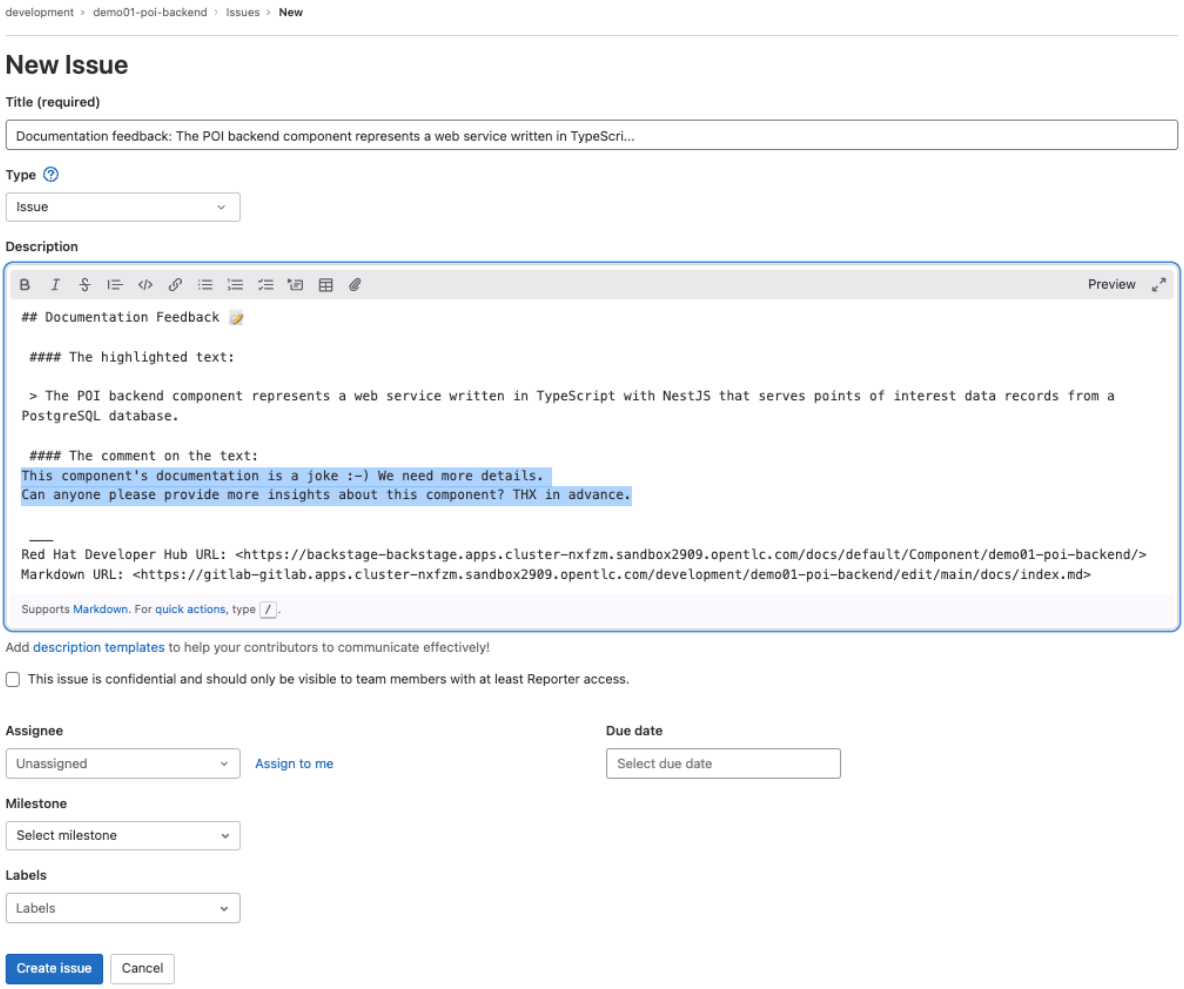


Figure 4-19: GitLab create new TechDocs issue for POI backend component.

When you are done, click **Create issue** at the bottom of the page. The result is shown in Figure 4-20.

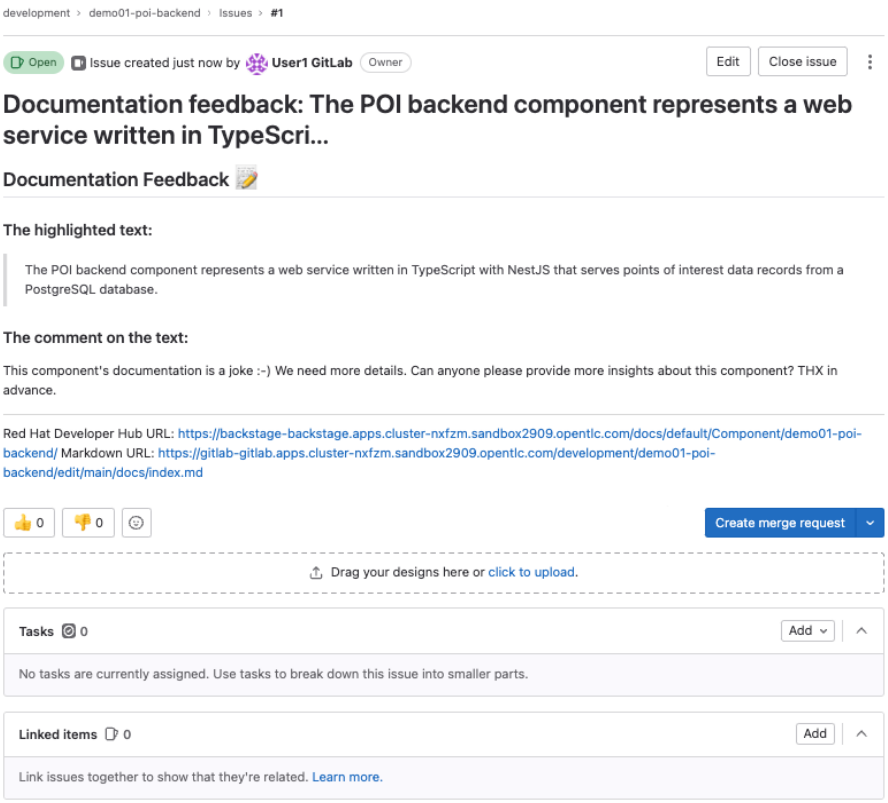


Figure 4-20: GitLab open TechDocs issue for POI backend component.

Switching to the RHDH component view for the `demo01-poi-backend` component and selecting the Issues tab, we can of course see this raised documentation-related issue accordingly (Figure 4-21).

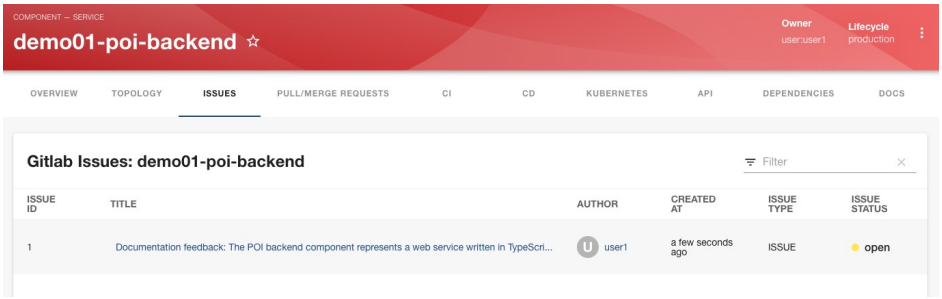


Figure 4-21: POI Backend component Issues tab with open TechDocs issue.

In summary, TechDocs in Red Hat Developer Hub takes away a lot of the usual pain and hassle related to technical documentation and is supposed to just work, provided it has been configured once upfront for RHDH and is properly integrated into the respective software templates.

Update the Software Catalog

After developing the application specific code and writing some documentation, you should also update relevant sections of the `catalog-info.yaml` for the `demo01-poi-backend` component. For this simple service, most of the catalog YAML definition is fine as originally scaffolded. However, it contains a few default descriptions across the entities, namely for the Component, the API, and the Resource. Modify the descriptions for the Component, the API, and Resource to something more meaningful that fits this `demo01-poi-backend` component. For instance, you might want to change these as follows:

- Component description: `NestJS backend service for the POI map application`
- API description: `API provided by the NestJS backend service of the POI map application to load and store POI records from the database`
- Resource description: `Database storing the POI records for the NestJS backend service of the POI map application`

Go to the `demo01-poi-backend` component's catalog detail view, select the **Overview** tab and click the **Edit Metadata** icon in the upper-right corner of the About tile (Figure 4-22).

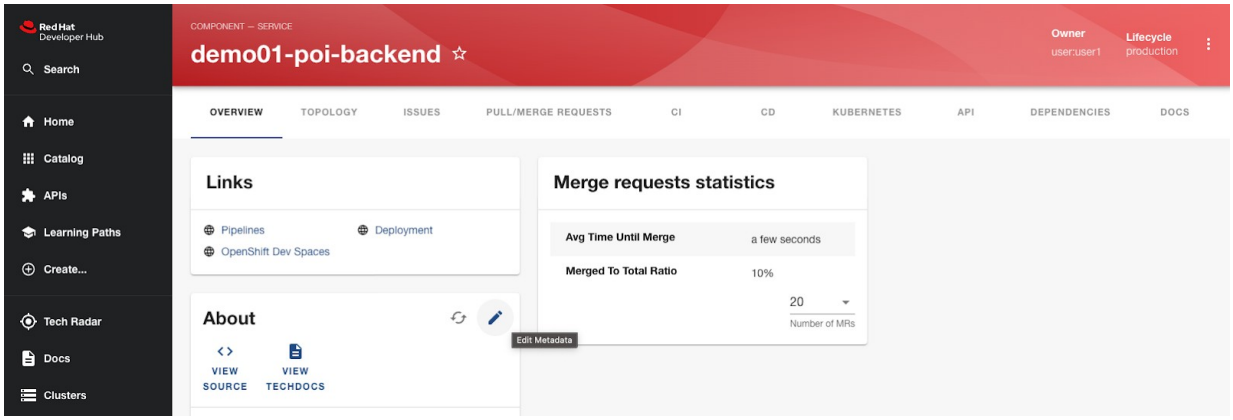


Figure 4-22: POI backend component Overview tab edit metadata.

This opens the `catalog-info.yaml` file in GitLab's edit mode, where you can directly modify the three descriptions in the YAML definition as shown in the following example. Most relevant in the context of this example are the three description fields marked in bold:

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
```

```
  name: demo01-poi-backend
  description: NestJS backend service for the POI map application
  annotations:
    argocd/app-name: demo01-poi-backend-dev
    backstage.io/kubernetes-id: demo01-poi-backend
    backstage.io/kubernetes-namespace: demo01
    backstage.io/techdocs-ref: dir:.
    gitlab.com/project-slug: development/demo01-poi-backend
    janus-idp.io/tekton-enabled: 'true'
  tags:
    - nodejs
    - nestjs
    - book
    - example
  links:
    - url: https://console-openshift-console.apps.cluster-
nxfzm.sandbox2909.opentlc.com/dev-pipelines/ns/demo01/
      title: Pipelines
      icon: web
    - url: https://console-openshift-console.apps.cluster-
nxfzm.sandbox2909.opentlc.com/k8s/ns/demo01/deployments/demo01-poi-backend
      title: Deployment
      icon: web
    - url:
https://devspaces.apps.cluster-nxfzm.sandbox2909.opentlc.com/#https://gitlab-
gitlab.apps.cluster-nxfzm.sandbox2909.opentlc.com/development/demo01-poi-backend?
che-editor=che-incubator/che-code/latest&devfilePath=.devfile-vscode.yaml
      title: OpenShift Dev Spaces
      icon: web
spec:
  type: service
  lifecycle: production
  owner: "user:default/user1"
  system: idp-system-demo01
  providesApis:
    - demo01-poi-backend-api
  dependsOn:
    - resource:default/demo01-poi-backend-db
---
apiVersion: backstage.io/v1alpha1
kind: System
```

```
metadata:
  name: idp-system-demo01
  tags:
    - rhdh
    - book
spec:
  owner: "user:default/user1"
---
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: demo01-poi-backend-api
  description: API provided by the NestJS backend service of the POI map
application to load and store POI records from the database
  links:
    - url: http://demo01-poi-backend-demo01.apps.cluster-
nxfzm.sandbox2909.opentlc.com/swagger
      title: Swagger UI
      icon: web
    - url:
https://gitlab-gitlab.apps.cluster-nxfzm.sandbox2909.opentlc.com/development/
demo01-poi-backend/-/blob/main/openapi.yaml
      title: API Spec
      icon: code
spec:
  type: openapi
  lifecycle: production
  owner: "user:default/user1"
  system: idp-system-demo01
  definition:
    $text: ./openapi.yaml
---
apiVersion: backstage.io/v1alpha1
kind: Resource
metadata:
  name: demo01-poi-backend-db
  description: database storing the POI records for the NestJS backend service of
the POI map application
spec:
  type: database
  owner: "user:default/user1"
```

```
  system: idp-system-demo01
```

Confirm these metadata changes by clicking **Commit changes** at the bottom (Figure 4-23).

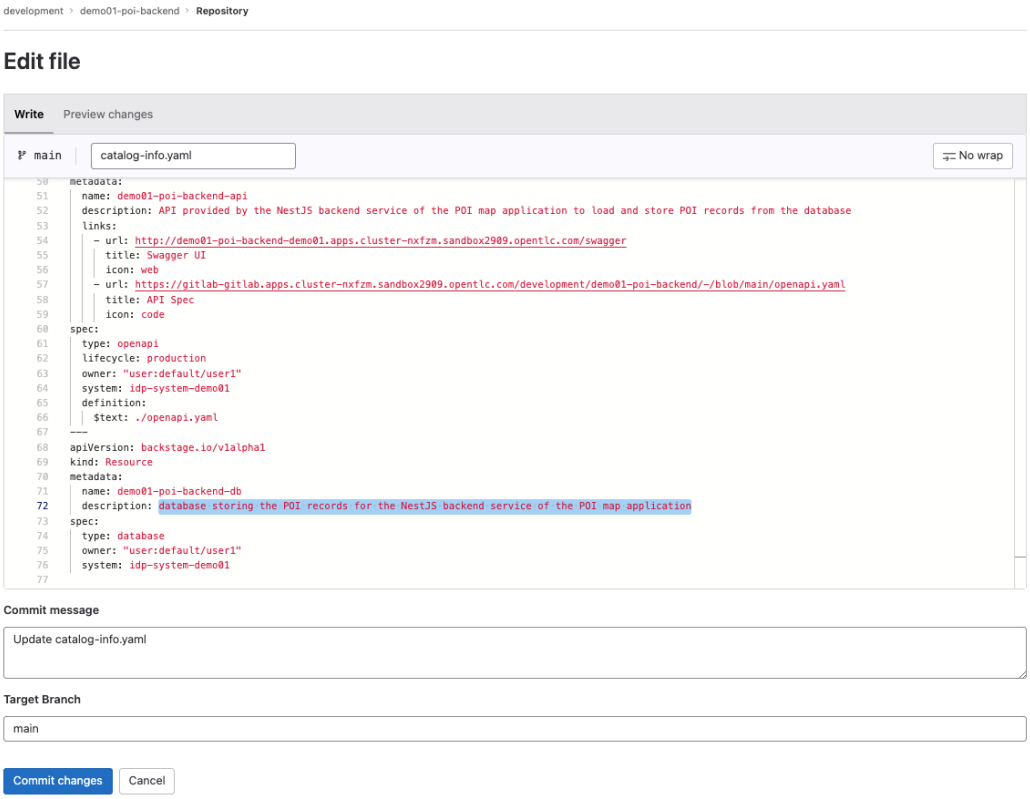


Figure 4-23: GitLab edit file view for catalog-info.yaml of POI backend component.

If you now go back to RHDH into the `demo01-poi-backend` component's detail view, select the **Overview** tab, and take a look at the **About** tile, it might still show the previous component description. The reason is that RHDH, based on configuration settings, will periodically refresh such component changes by syncing the respective files from the GitLab repository into the software catalog. In case you are impatient, you can click the **Sync** icon in the upper-right of the About tile to actively schedule a refresh (Figure 4-24).

Eventually, whether you just waited for a while or actively scheduled a refresh, you will see the three description changes that have been done in the underlying `catalog-info.yaml` in the respective RHDH catalog view and component tabs (Figure 4-25).

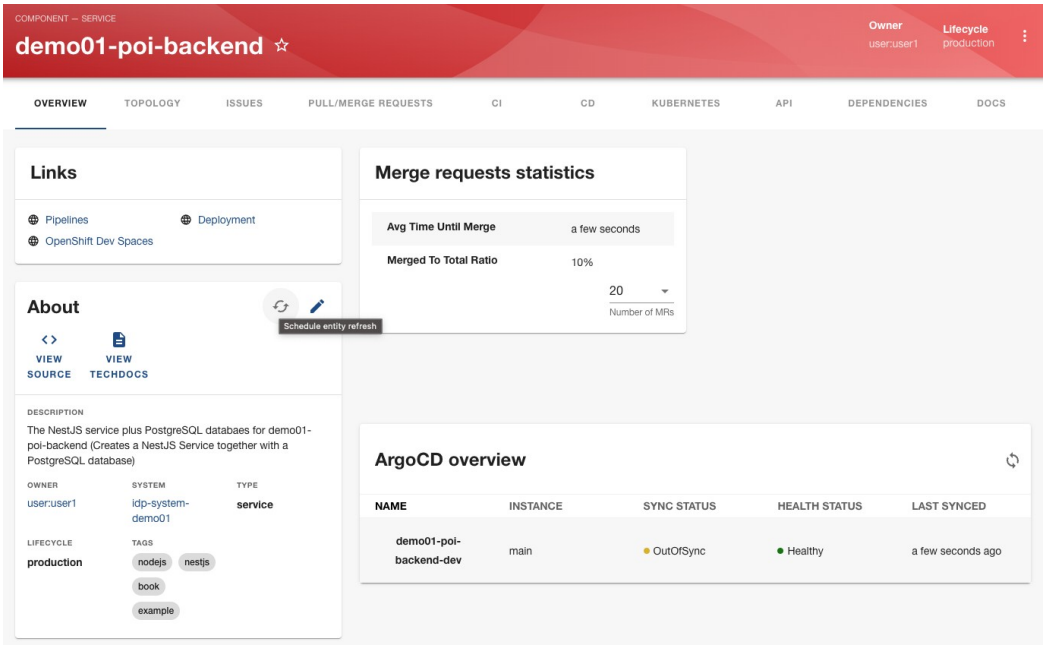


Figure 4-24: POI backend component Overview tab schedule entity refresh.

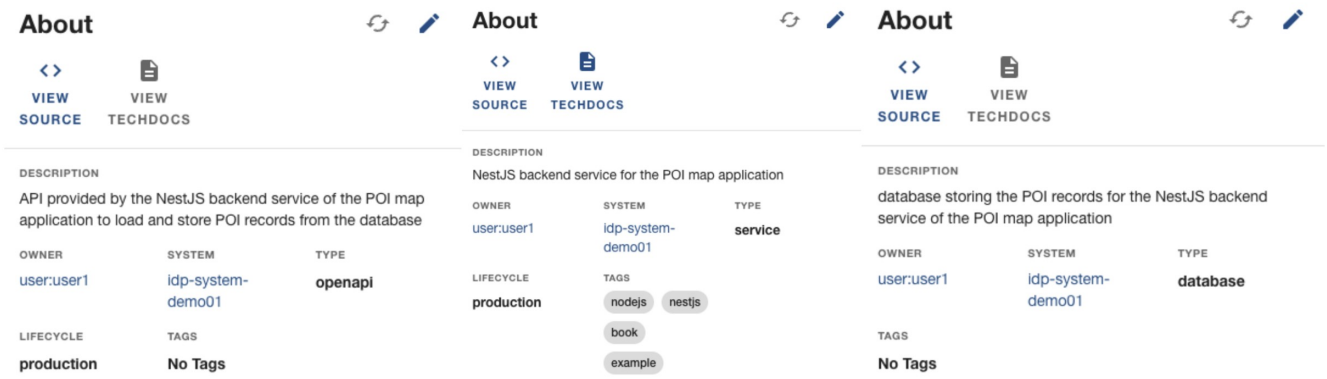


Figure 4-25: Component, API, and database resource descriptions for the POI backend.

Similar to these basic changes, more complex modifications can be performed whenever needed, such that the underlying metadata always reflects the current state based on your most recent engineering activities.

This concludes your RHDH journey for building the NestJS backend service of the POI map application based on the template you applied in Chapter 3 (see Backend template on page 25).

Next up, you will shift focus towards the proxy and frontend code base that has already been scaffolded (see Proxy and frontend template on page 42) into a monorepo using the `quarkus-with-angular` template.

Implement the proxy and the frontend

Remember that the `quarkus-with-angular` template you applied in one of the previous sections also scaffolded the monorepo that hosts both the proxy service (Quarkus) and the Angular SPA (frontend). In this section, you are going to add the necessary application code for the proxy service as well as the Angular SPA frontend to change the currently present "Hello, World" kind of code bases for each of the two applications.

Write the code

In RHDH, switch to the **Catalog View** and select the `demo01-poi-map-service` component. In contrast to a local development workflow that you followed for the backend service, you are taking a different approach here. In the component's details view, select the **Overview** tab and click the **OpenShift Dev Spaces** link in the About tile. This will launch a web-based developer workspace powered by Eclipse Che (Figure 4-26).

> **Note:** During the time it takes to launch your browser-based VS Code instance, you might be asked for a re-authentication along the way, potentially more than once depending on how your RHDH environment has been configured in that regard.

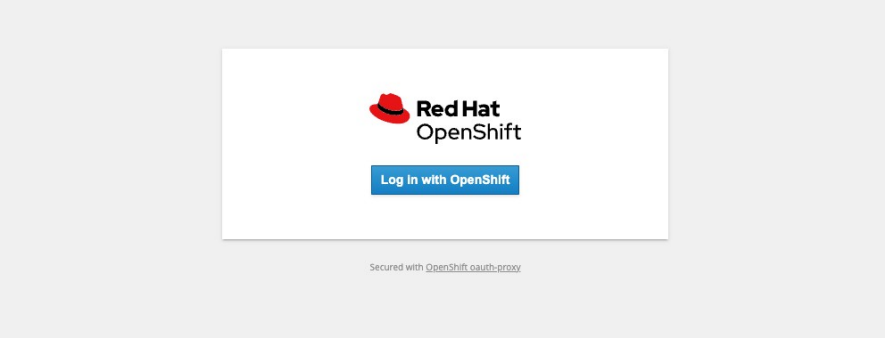


Figure 4-26a: OpenShift Dev Spaces login with OpenShift.

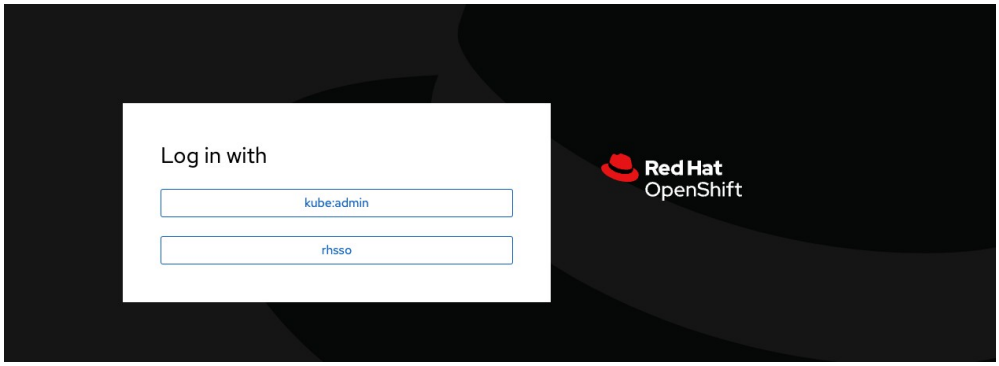


Figure 4-26b: OpenShift Dev Spaces login with Red Hat's single sign-on tool.

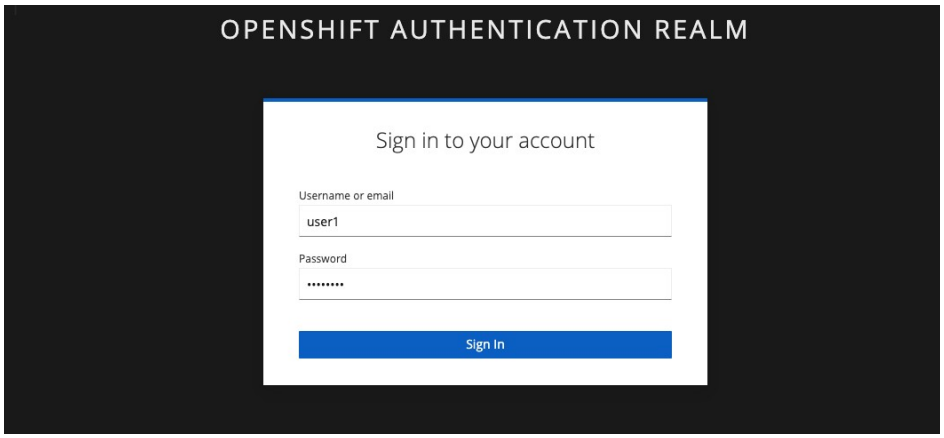


Figure 4-26c: Dev Spaces OpenShift Authentication Realm account sign in.

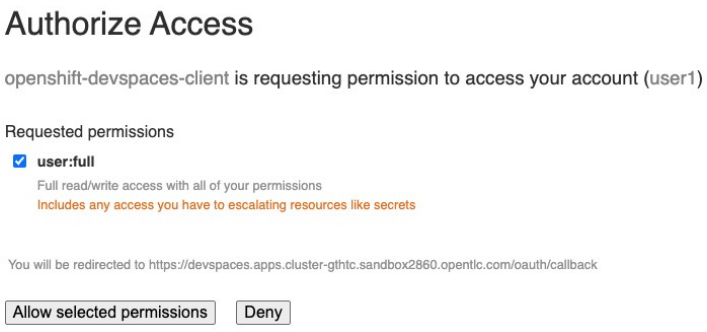


Figure 4-26d: Dev Spaces authorize access to grant full user permissions.

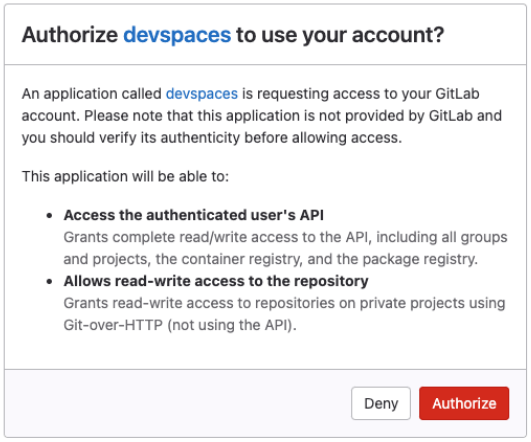


Figure 4-26e: GitLab authorize Dev Spaces.

What's really convenient when taking this route is that you eventually end up in your dedicated and fully-fledged VS Code instance with the proper Git repository already checked out (Figure 4-27). This means you can start right away with coding the application in question—all without going through any hassle of having to set up everything locally.

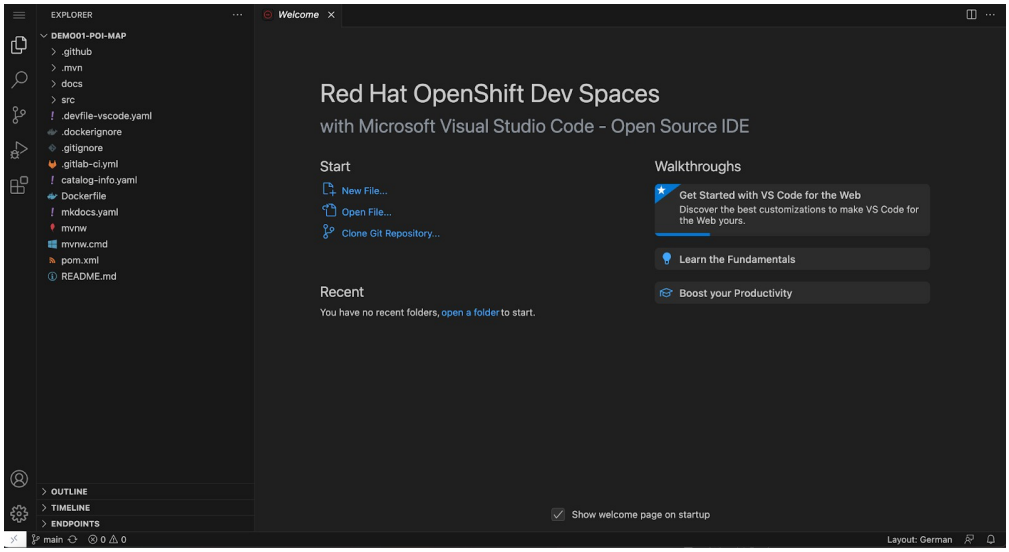


Figure 4-27: OpenShift Dev Spaces welcome screen.

Again, to speed things up, we provide a turnkey implementation for the frontend service in a dedicated repository that is part of this book's GitHub organization.

You are going to merge the sample code into the repository of your scaffolded backend application. In OpenShift Dev Spaces, your web VS Code instance, open a terminal session by selecting **Terminal → New Terminal** from the burger menu in the upper left corner of the UI (Figure 4-28).

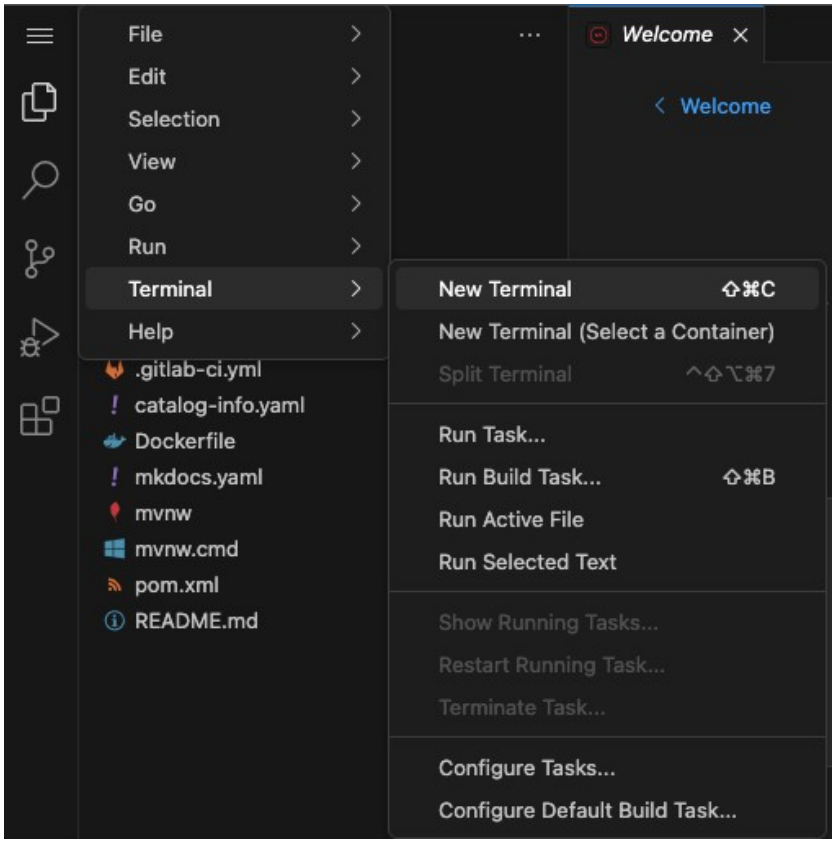


Figure 4-28: OpenShift Dev Spaces VS Code open new terminal.

Create a new branch in VS Code by switching to the Source Control view and then clicking the 3-dots menu in the upper-right of the left view pane to select **Branch → Create Branch** (Figure 4-29) and use `app-dev` as the branch's name (Figure 4-30).

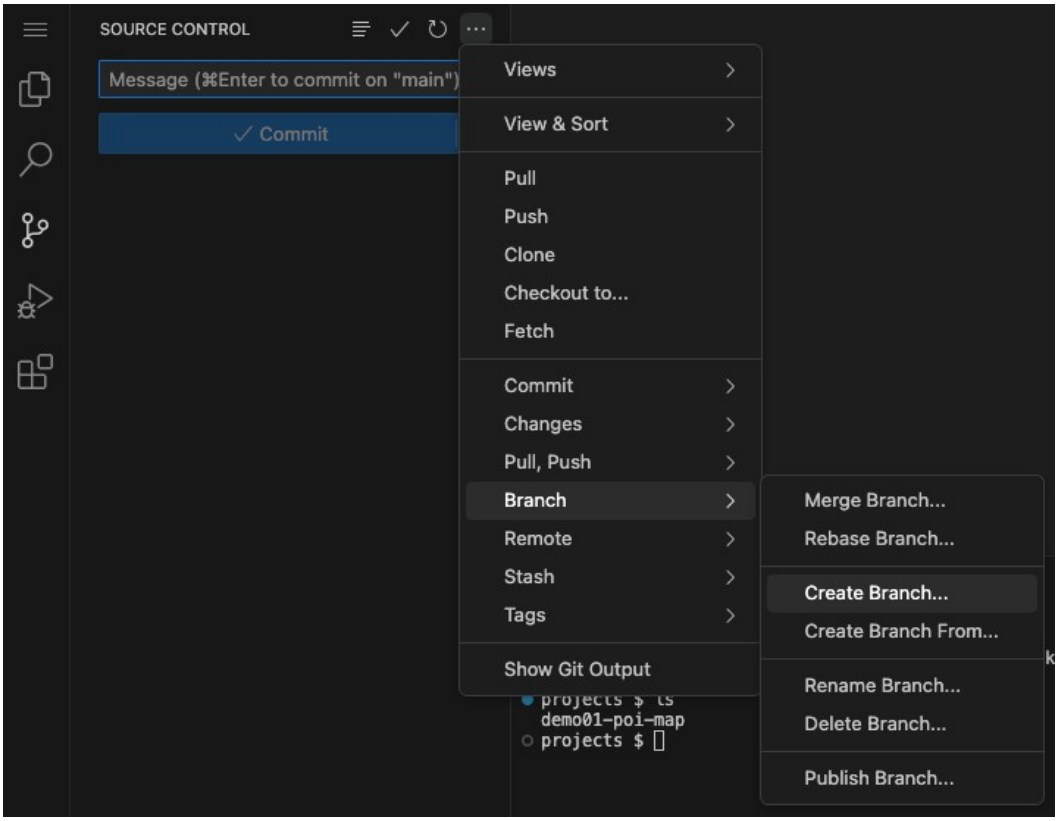


Figure 4-29: OpenShift Dev Spaces VS Code create new branch.

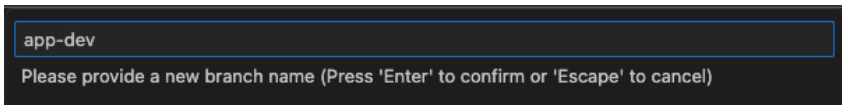


Figure 4-30: OpenShift Dev Spaces VS Code name branch.

After creating and selecting this new `app-dev` branch, click into the terminal window at the bottom right of the screen and proceed with the following steps in order to add the pre-created code necessary for the proxy and frontend applications to work together:

1. Add the sample code repository as another Git remote called `solution`:
   ```
   git remote add solution
   https://github.com/developer-hub-books/rhdh-book1-sample-code-poi-map.git
   ```
2. Fetch the contents from this new Git remote: `git fetch solution`
3. Merge the sample code into your `app-dev` Git branch:
   ```
   git merge --allow-unrelated-histories --strategy-option theirs -m "merge provided sample code" solution/main
   ```

4. Switch back to the file explorer view and open the file `src/main/angular/src/assets/env.js` from the files and folders view on the left. In that file, scroll down to line 12 and specify the REST API URL and the websocket endpoint (see TODO sections in the following example):

```
 //-------------------------------------------------------------------
  //TODO 1:
  //  Add the secure https route URL pointing to your proxy service into
the setting below.
  //  You can find the route URL in RHDH by inspecting the component's
details in the topology view.
  //  If you used the suggested namespace ('demo01') and component id
('poi-map') it should look like this https://demo01-poi-map-
demo01.<cluster_id_here>
  //  Beware that this must be a secure HTTP URL that starts with
'https://'
  window["env"]["gatewayApiUrl"] =
"<HTTPS_ROUTE_URL_TO_YOUR_PROXY_SERVICE_HERE>";
  //TODO 2:
  //  Add the secure websocket route URL pointing to your proxy service
into the setting below.
  //  You can find the route URL in RHDH by inspecting the component's
details in the topology view.
  //  If you used the suggested namespace ('demo01') and component id
('poi-map') it should look like this wss://demo01-poi-map-
demo01.<cluster_id_here>/ws-server-endpoint
  //  Beware that this must be a secure websocket URL that starts with
'wss://'
  window["env"]["websocketEndpoint"] =
     "<WSS_ROUTE_URL_TO_YOUR_PROXY_SERVICE_HERE>/ws-server-endpoint";
  //-------------------------------------------------------------------
```

It's important that both these (HTTPS and WSS) match the route's URL which has been created during the initial deployment of the proxy service. Save this file after applying the modification.

5. Next, open the file `src/main/resources/META-INF/openapi.yaml`, scroll down to line 12, and specify the proper server URL (see TODO section) for this OpenAPI spec.

```
servers:
  #-------------------------------------------------------------------
  # TODO:
  # Add the secure https route URL pointing to your proxy service into the
```

```
  # setting below. You can find the route URL in RHDH by inspecting the
  # component's details in the topology view. If you used the suggested
  # namespace ('demo01') and component id ('poi-map') it should look like
this
  # https://demo01-poi-map-demo01.<cluster_id_here>
  # Beware that this must be a secure HTTP URL that starts with 'https://'
  - url: <HTTPS_ROUTE_URL_TO_YOUR_PROXY_SERVICE_HERE>
    description: (RHDH component deployment)
  #----------------------------------------------------------------------
```

It's important that this server URL matches the respective HTTPS route URL which has been created during the initial deployment of the backend service. Save this file after applying the modification.

6. **Optional:** This step is only necessary if you used different settings for the namespace (`demo01`) and/or component ID of the backend (`poi-backend`). Open the file `src/main/resources/application.properties`. Scroll down to line 30 and specify the cluster internal service name (see TODO section in the next example).

```
#----------------------------------------------------------------------
# OPTIONAL TODO:
# In case you have been following the instructions given in the respective
# book chapters regarding the settings for namespace ('demo01') and
# component id ('poi-backend'), you are good.
# Otherwise please change the following config property and set it to the
# cluster internal Kubernetes service name which was generate during the
# initial RHDH deployment of the backend app component.
#
# http://<CLUSTER_INTERNAL_K8S_SERVICE_NAME>:3000
# http://demo01-poi-backend:3000

parks.backend.endpoint=http://demo01-poi-backend:3000

#----------------------------------------------------------------------
```

Save this file after applying the modification.

7. Stage and commit all changes which are reflected due to performing the manual changes as just explained. Your changeset should look similar to one in Figure 4-31.

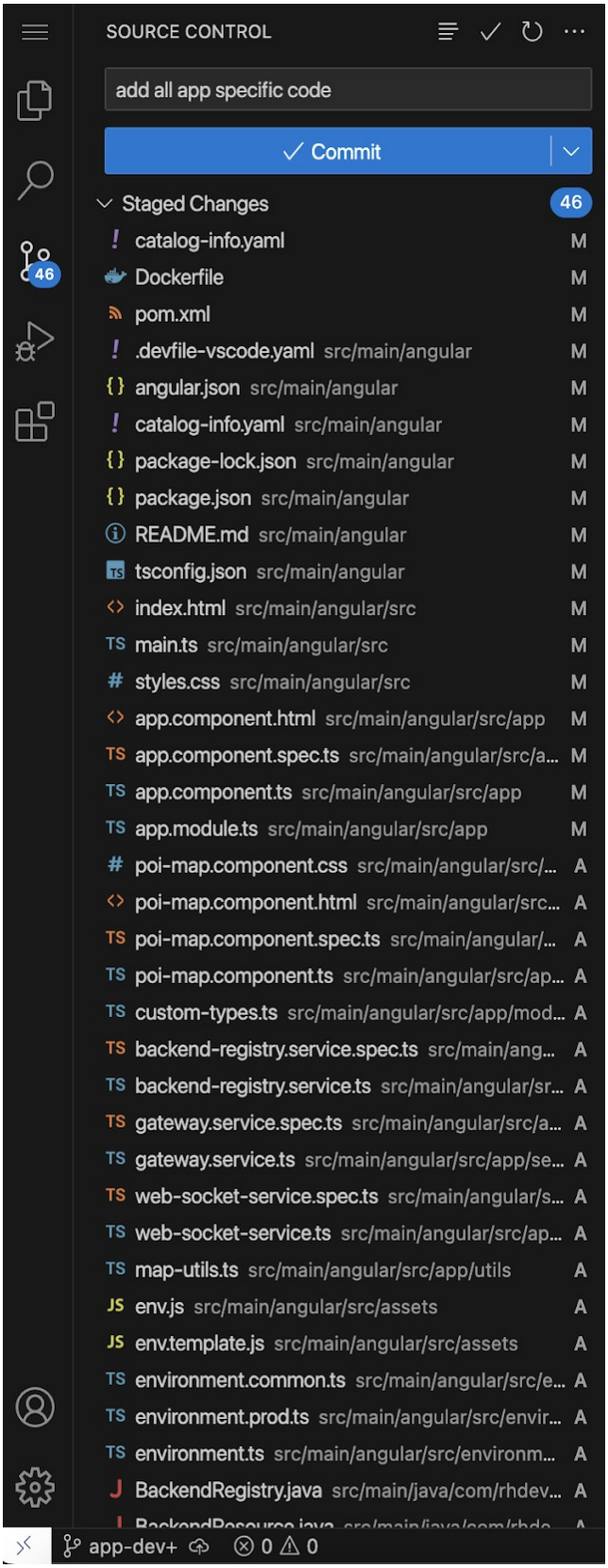


Figure 4-31: OpenShift Dev Spaces VS Code changeset view.

8. Finally, click the **Publish Branch** button to push this branch to the underlying GitLab repository (Figure 4-32a).

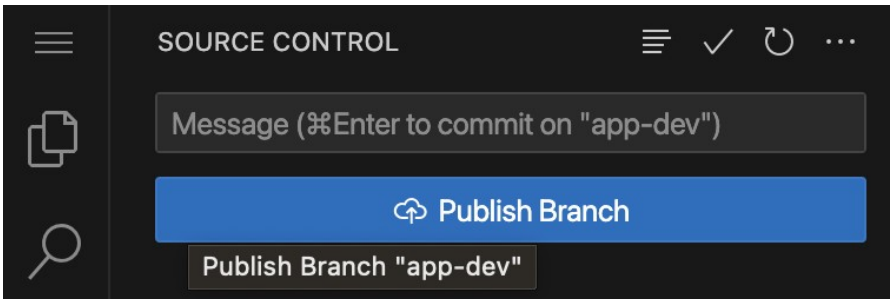


Figure 4-32a: OpenShift Dev Spaces VS Code publish branch.

9. When prompted to pick one of the two remotes for this repository, make sure to select the `origin` remote that points to your scaffolded application repository (Figure 4-32b).

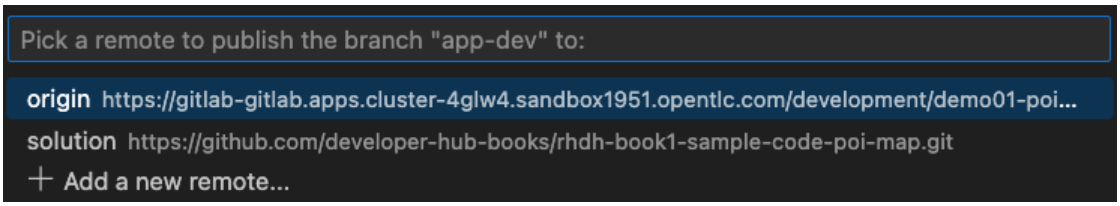


Figure 4-32b: OpenShift Dev Spaces VS Code pick remote.

10. Open the GitLab repository for the `demo01-poi-map` component and create a new merge request for this `app-dev` branch (Figure 4-33).

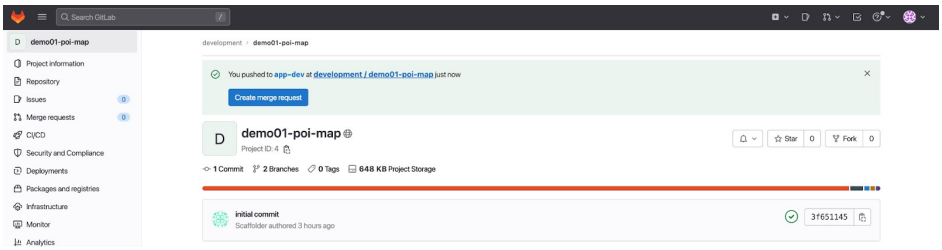


Figure 4-33a: GitLab create merge request for pushed branch.

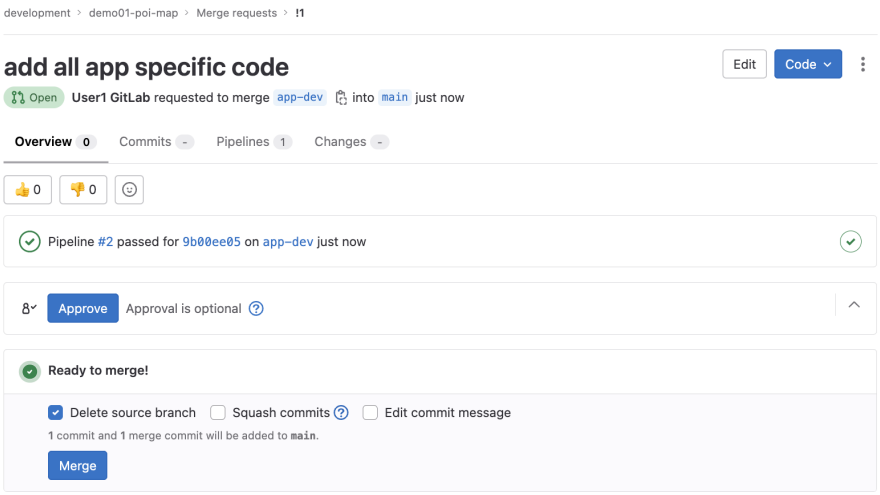


Figure 4-33b: GitLab open merge request for pushed branch.

11. Merge this new `app-dev` branch into the `main` branch right away (Figure 4-34).

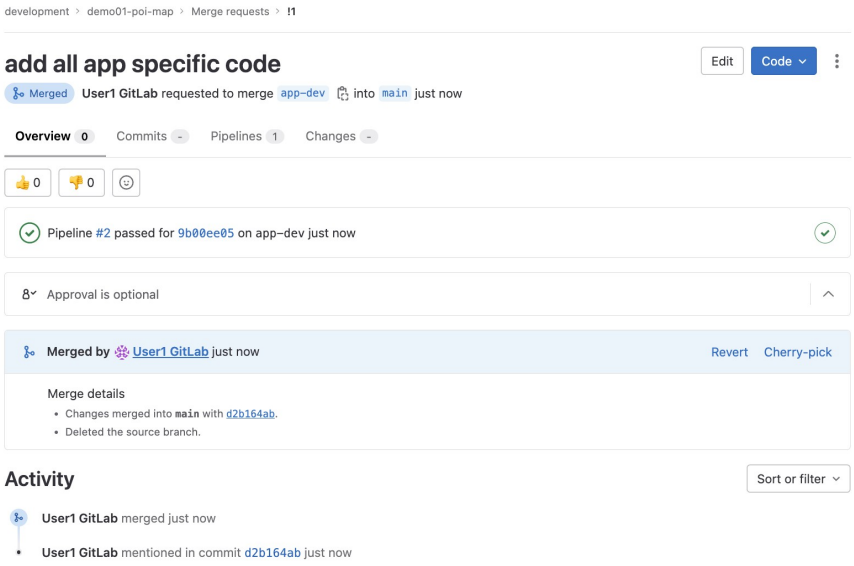


Figure 4-34: GitLab merged merge request for branch app-dev.

Check CI/CD

The merged code will trigger the preconfigured build pipeline via a webhook. After about two to three minutes, your code changes are available in the freshly built container image for the POI map service, which contains both the Quarkus proxy service and the Angular SPA frontend. See Figure 4-35.

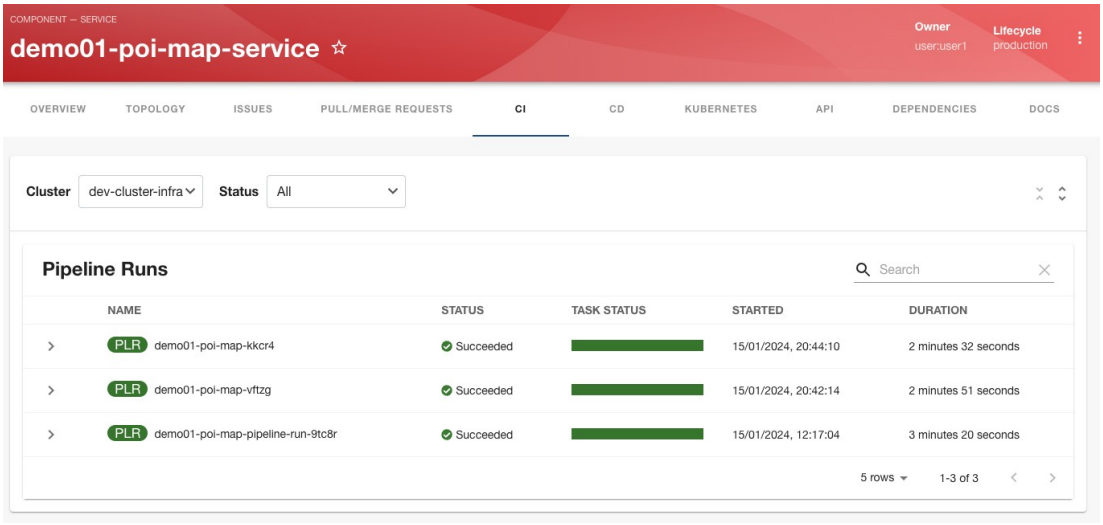


Figure 4-35: POI frontend's Tekton build pipeline triggered after merging code changes.

Check frontend app status

With the necessary application changes being done, let's figure out if your POI map proxy and frontend are running correctly. Open the `demo01-poi-map-service` in the RHDH catalog component view, switch to the **Topology** tab, click the deployment, and select **Resources** from the right-side pane (Figure 4-36).

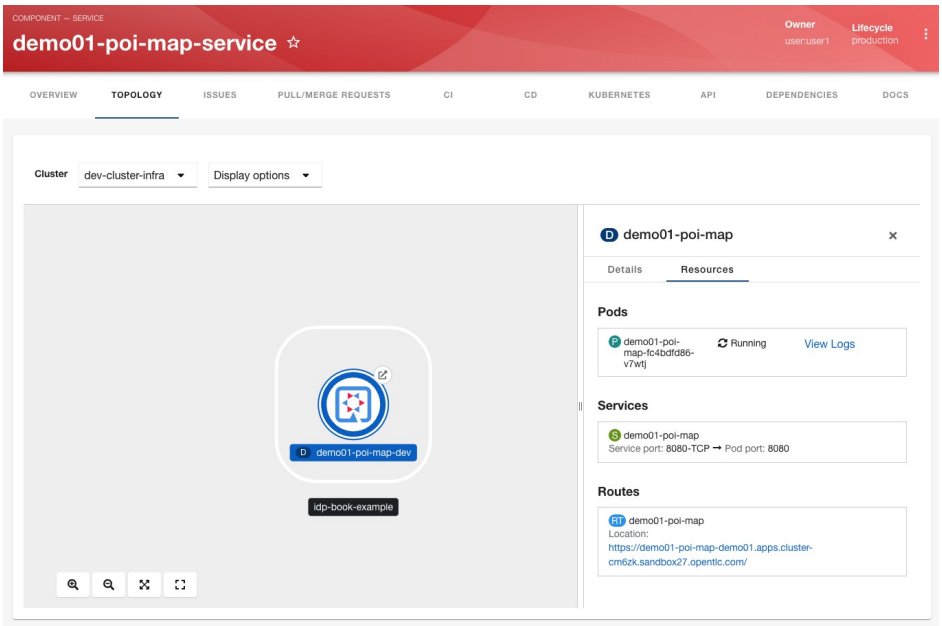


Figure 4-36: POI frontend component Topology tab deployment resources.

The pod seems to be running fine. Explore its logs by clicking **View Logs** next to the running pod information. You should see a log output similar to the one in Figure 4-37.

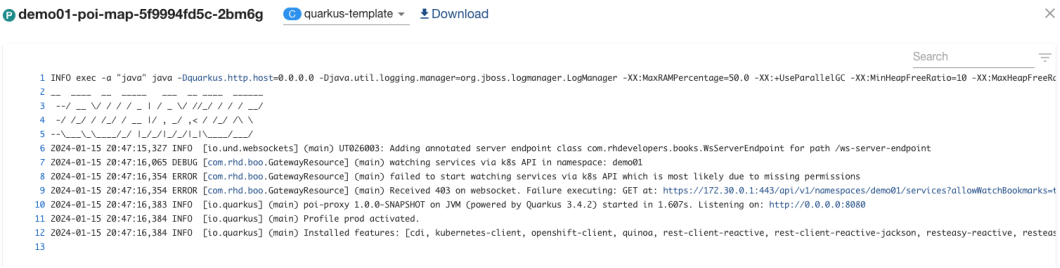


Figure 4-37: POI frontend component pod logs view.

This indicates that the proxy and frontend are up and running fine. After closing the logs, you can click on the **Routes** link to open a new browser tab where you should see the POI map application as shown in Figure 4-38.



Figure 4-38: POI map application fully working.

Add the documentation

Backstage upgrades technical documentation to first class. Making it relatively easy to create docs encourages their actual creation. Open `demo01-poi-map-service` in the catalog. Click on **View TechDocs** in the About tile in the component overview. Once again, the template scaffolder has left you a bit of boilerplate to start from.

Similarly to the quick edit you made to the backend component (`demo01-poi-backend`) earlier, you can perform smallish updates to the documentation by changing the Markdown file right in GitLab's file edit mode. For bigger documentation enhancements, you might want to work in a clone of the `demo01-poi-map-service` repo and in the editor or IDE you prefer.

Update the Software Catalog

After writing the code and the documentation, it's necessary to update relevant sections of the `catalog-info.yaml` files for both catalog components (`demo01-poi-map-service` and `demo01-poi-map-frontend`) so that they match the recent changes and are also tailored to reflect the POI map application rather than the "Hello, World" skeleton that was originally generated by the template. You've already learned how this is done while working on the backend component `demo01-poi-backend`. (See Update the Software Catalog on page 65.)

Summary

In this chapter you completed a common development task by replacing the skeleton provided for your language and framework with the first iteration of running code for your new project. You've triggered the process of building and deploying that code through a GitOps pipeline onto a cluster via the Kubernetes API. Your map application is online and displays the default set of Points of Interest. You've taken your project from scaffolded start to minimum viable product.

Next, you'll learn a technique for using Software Templates in the developer portal to manage essential maintenance tasks in a repeatable way.

Chapter 5: Template and component evolution

You've learned how to jump-start a new application with Software Templates, which scaffolded source repos for you and eventually constructed container images and deployed them to a cluster through the Kubernetes API. But software is never finished. Sooner or later, you'll need to update your templates and the components they generate. In this chapter, you'll see how to manage these maintenance and upgrade tasks in Red Hat Developer Hub.

Evolving Software Templates

Remember the second template you applied in Chapter 3 to scaffold the proxy and frontend parts of the POI map application created a Quarkus application that hosts an Angular SPA frontend. In order to have some concrete example to work on, assume your company decides that all templates for Quarkus applications should, as of now, adhere to the following standards:

- The container base image for Java applications should be `ubi8/openjdk-17:1.18`.
- The platform version for Quarkus should be `3.6.7`.
- Quarkus-based applications are supposed to include the `smallrye-health` extension.

Taking a closer look, it's quite clear that the software template `quarkus-with-angular` that is currently registered in the Red Hat Developer Hub catalog needs to be updated. While the responsibility for template-related maintenance typically lies with platform engineers, the

suggested updates in the context of this use case are relatively straightforward and serve as a simple example for developers in order to have a basic understanding of this procedure.

Locate the template and clone the repository

Go to **+ Create** and find the **Quarkus Service with hosted Angular Frontend** template. Click the **Links** (**<>**) button in the lower-left corner of the software template tile (Figure 5-1).

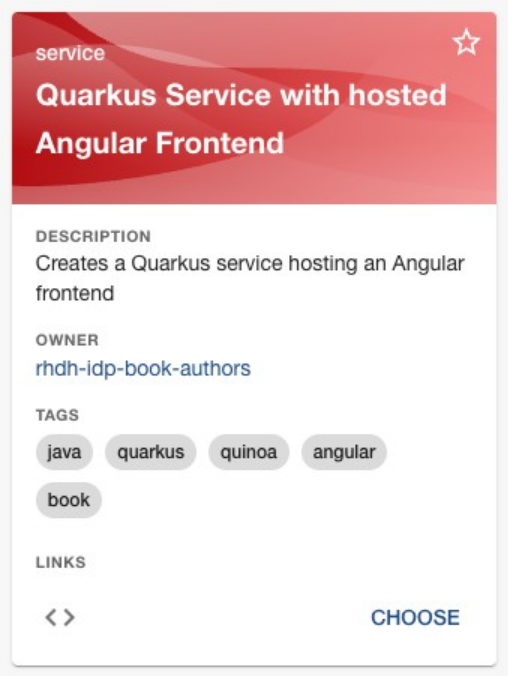


Figure 5-1: Quarkus Service with hosted Angular Frontend template.

Clicking this Links button brings you to the GitLab repository that hosts the template files. Over in GitLab, find the **Clone** button for the repository, then clone it to your local machine by using the provided clone URL (Figure 5-2).

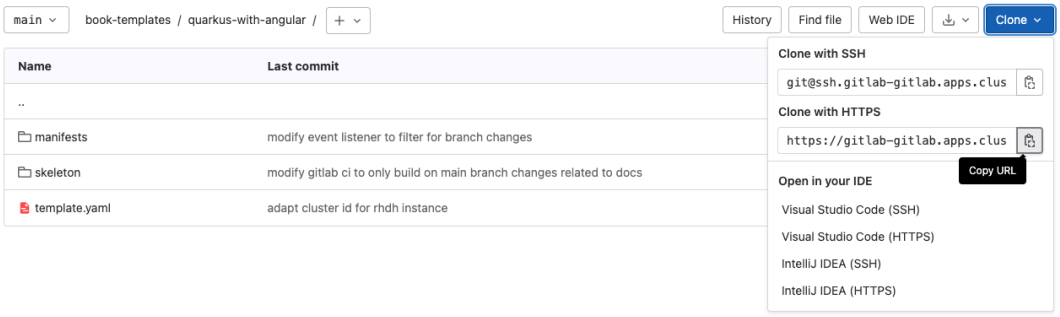


Figure 5-2: GitLab repository for the template to clone from.

Make the necessary template changes

After successfully cloning it, open the repository in the IDE of your choice, then start to perform the following necessary changes to the respective files residing in the `quarkus-with-angular` folder.

Patch the container file

Open the file `quarkus-with-angular/skeleton/Dockerfile`, scroll down to the `FROM` statement in line 80, and adapt the base image to `openjdk-17:1.18` as follows:

```
~~FROM registry.access.redhat.com/ubi8/openjdk-17:1.16~~
FROM registry.access.redhat.com/ubi8/openjdk-17:1.18
```

Save your changes.

Patch the pom.xml file

Open the file `quarkus-with-angular/skeleton/pom.xml`, scroll down to the Quarkus platform version in line 15, and adapt the version to `3.6.7` as follows:

```
~~<quarkus.platform.version>3.4.2</quarkus.platform.version>~~
<quarkus.platform.version>3.6.7</quarkus.platform.version>
```

Next, scroll further down to the dependencies section, which starts at line 31, and include the `smallrye-health` extension by adding the following XML snippet (e.g., right below the `quarkus-arc` dependency that is already there). It's important that you don't remove any of the existing ones:

```
<!-- ... -->
<dependencies>
    <dependency>
      <groupId>io.quarkus</groupId>
      <artifactId>quarkus-arc</artifactId>
```

```
    </dependency>
    <dependency>
      <groupId>io.quarkus</groupId>
      <artifactId>quarkus-smallrye-health</artifactId>
    </dependency>
<!-- ... -->
</dependencies>
```

Save the file.

Commit and push template updates

With these template updates in place, stage, commit, and push your changes. Then verify that your changes are reflected in the upstream repository on GitLab (Figure 5-3).

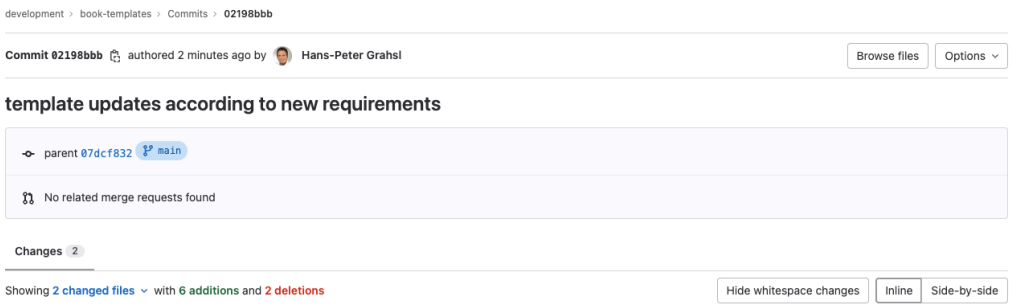


Figure 5-3: GitLab repository after updating the template.

With these updates in place, all new components scaffolded from this new version of the `quarkus-with-angular` template automatically follow the defined requirements related to the version of the container base image, the Quarkus platform version, and the inclusion of the `smallrye-health` extension. In the next section, you'll learn how to update existing software components that have already been created.

Updating existing components

It's crucial not to forget about the fact that despite changing a specific software template, existing applications previously built from this template do not yet adhere to any newly defined standards. So how should you go about updating them?

Unsurprisingly, the answer again is related to yet another template that will make the procedure of patching previously created software components a lot easier and less error prone. By making sure that the necessary changes are first suggested to existing source code repositories by means of a pull request, component owners can review everything and have a

final say before eventually merging the auto-generated changes originating from the applied "patch template."

Register the predefined patch template

In Chapter 3, you already prepared all templates for the book examples by forking the original template repository from the book's GitHub organization and customizing the RHDH cluster ID. You can always refer to the README in the original template repository if something is unclear. It explains all the preparation steps.

You'll now need to register the custom-component-patch template from within your forked template repository as follows:

1. Go to **+ Create → Register existing component** and copy the full HTTPS URL to the `template.yaml` file in the `custom-component-patch` folder of your forked Git repository hosting all book templates. Paste the URL into the form's (1) **Select URL** field, as shown in Figure 5-4.

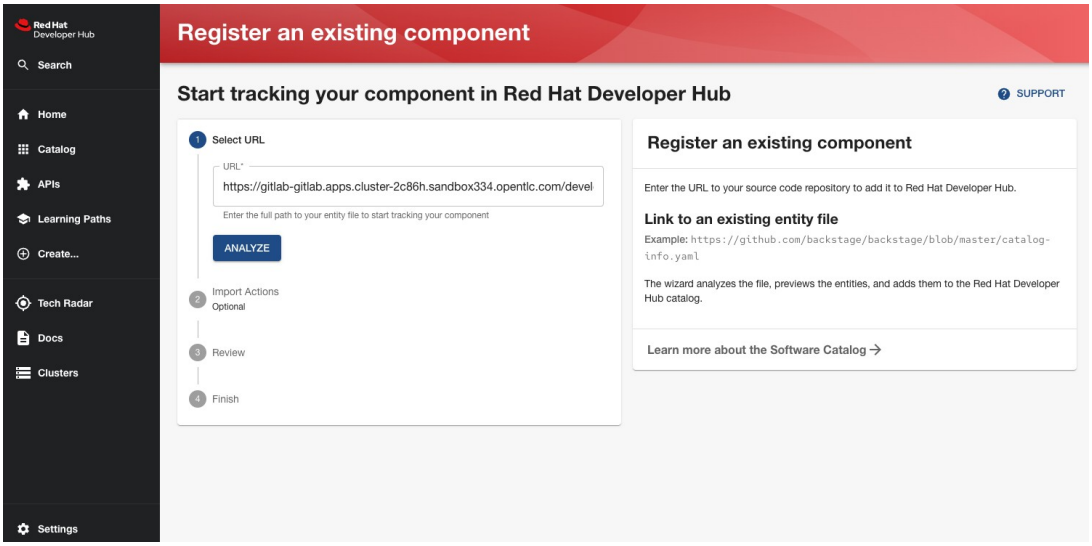


Figure 5-4: Template registration into the Software Catalog, step 1.

2. Clicking the **ANALYZE** button shows that two entities will be added into the software catalog (Figure 5-5). One entity is the location, pointing to the HTTPS URL from where the template was loaded, and the other entity is the template itself.

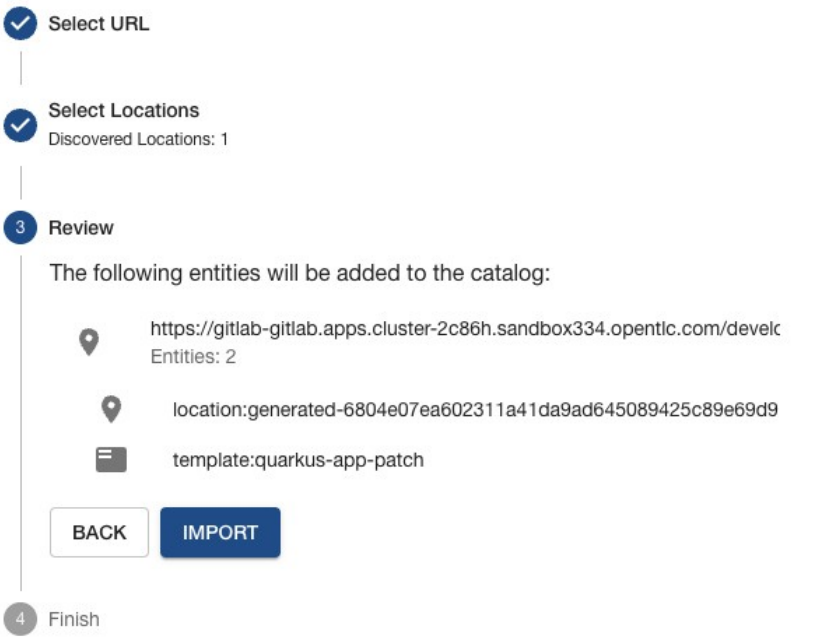


Figure 5-5: Template registration into the Software Catalog, step 2.

3. Click the **IMPORT** button to confirm. Figure 5-6 shows the entities have been added to the catalog.

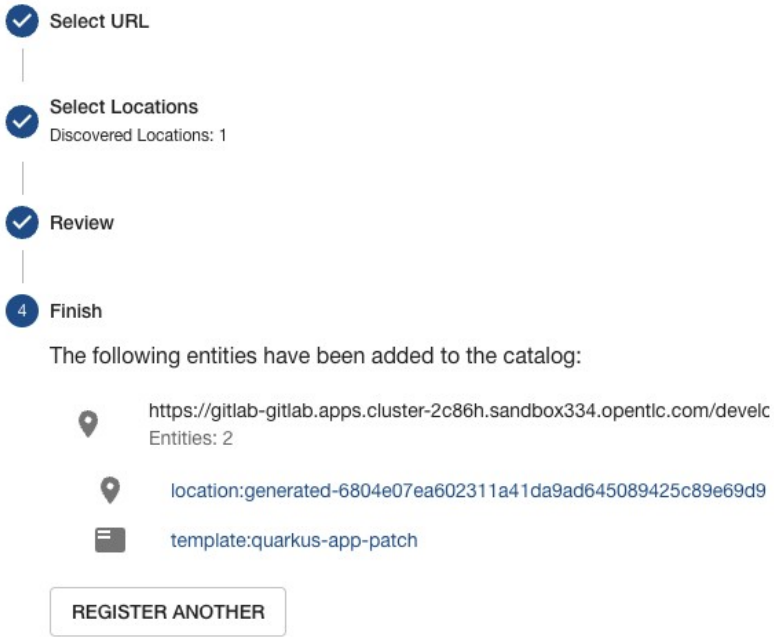


Figure 5-6: Template registration into the Software Catalog, step 3.

Apply the predefined patch template

Go to **+ Create** and select the **Quarkus Application Patch Template** by clicking the **CHOOSE** button in the lower-right corner of the template tile (Figure 5-7).

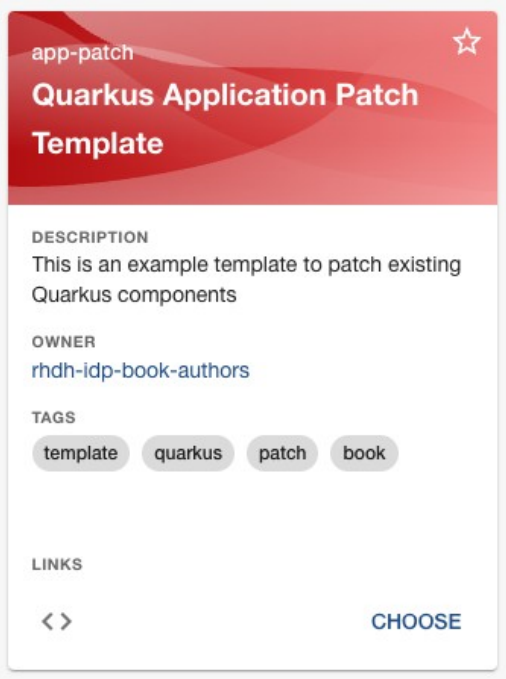


Figure 5-7: Quarkus Application Patch Template.

Template form wizard

This brings you to the template's form wizard where you can configure certain elements of the template. In the first section of this form (Figure 5-8), you provide information about the component that you want to apply this patch on. In this example, it's the cluster ID, the GitLab group/owner, and most importantly, the GitLab repo name(s) that should get patched. You are going to specify the repo name by clicking on the **+ ADD ITEM** button and entering the repo name of the proxy and frontend component, which is `demo01-poi-map` if you followed the naming conventions so far. You can optionally enter a particular name (default: `templated-app-patch`) for the repo branch that will store the code changes that should get applied by the template. This branch in turn becomes the source for the merge request that gets automatically opened.

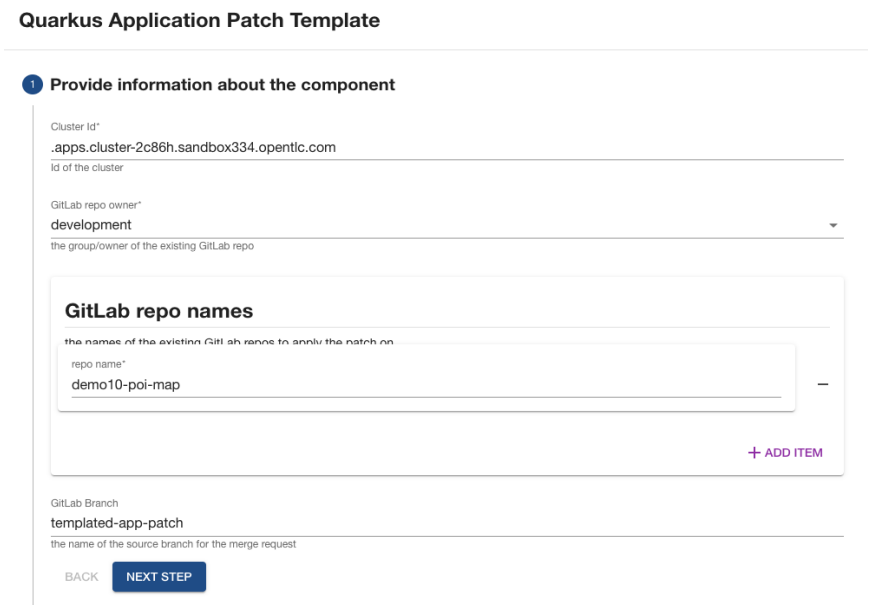


Figure 5-8: Quarkus Application Patch Template configuration form, first section.

In the second section (Figure 5-9), you can specify all information about the patch itself that is getting applied to the selected source code repositories. The template already provides the necessary defaults. In this example:

- the `pom.xml` file will get patched with the desired Quarkus version, and
- the Dockerfile to build the container image will get patched with the proper base image.

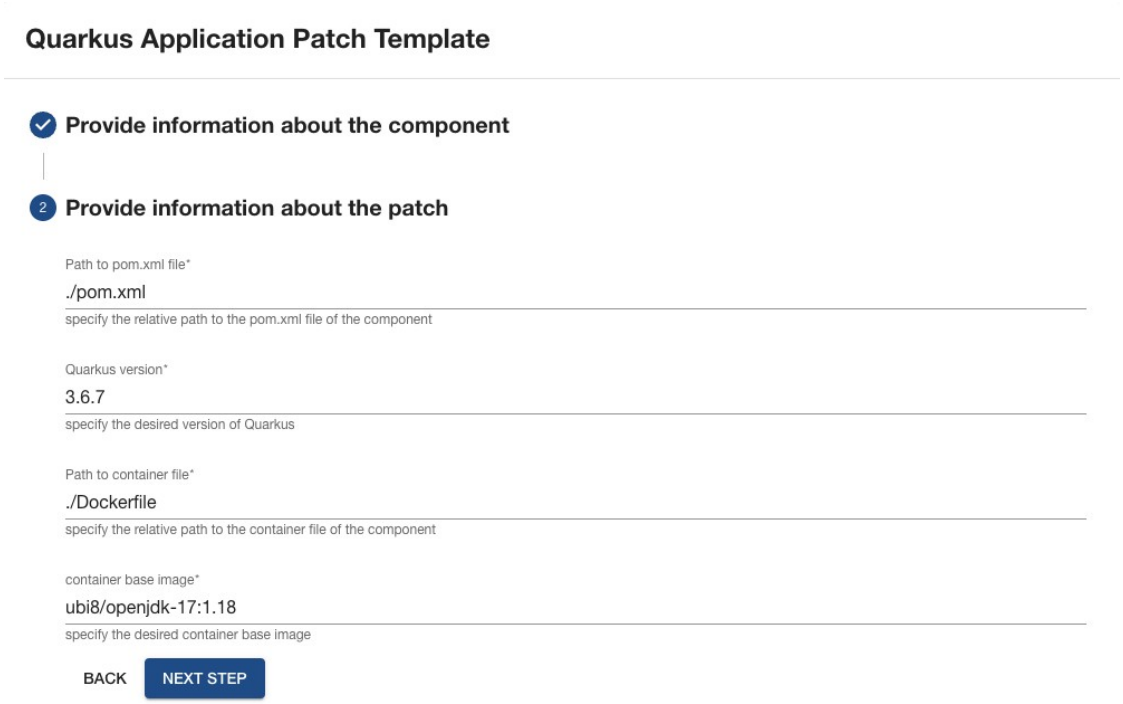


Figure 5-9: Quarkus Application Patch Template configuration form, second section.

Remember that another requirement was to include the `smallrye-health` extension. This is an additional change to the `pom.xml` file and happens without the need for explicit template configuration parameters.

Clicking the **NEXT STEP** button shows you a summary of all the entered form fields for a final review, as shown in Figure 5-10.



Figure 5-10: Quarkus Application Patch Template configuration form, final review.

Click **CREATE** to kick off the process of patching the component's source code repository.

Template patching procedure

The Quarkus Application Patch Template (Figure 5-11) is composed of four sequential steps, each of which represents either a built-in or a custom scaffolder action available in your running RHDH portal:

1. `fetch:plain`: This action fetches one or more source code repositories from its location and stores the repository contents into a working directory.
2. `roadiehq:utils:fs:replace`: This action will look into each of the working directories and process the specified files as follows:
    - `pom.xml`: Replace the Quarkus version and add the `smallrye-health` extension.
    - Dockerfile: Replace the base image.
3. `publish:gitlab:merge-request`: After applying the necessary changes to selected files in the working directory, this action will create a merge request in the respective GitLab repository.
4. `debug:log`: This action prints the full URL pointing to the merge request(s) that have been opened in one or more repositories.

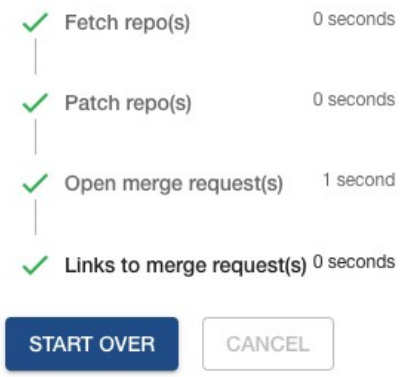


Figure 5-11: Quarkus Application Patch Template.

Review the opened merge request

In GitLab, go to the repository that has been patched by means of the template and inspect the opened merge request (Figure 5-12). The **Changes** tab shows the applied patches to the files as previously discussed.

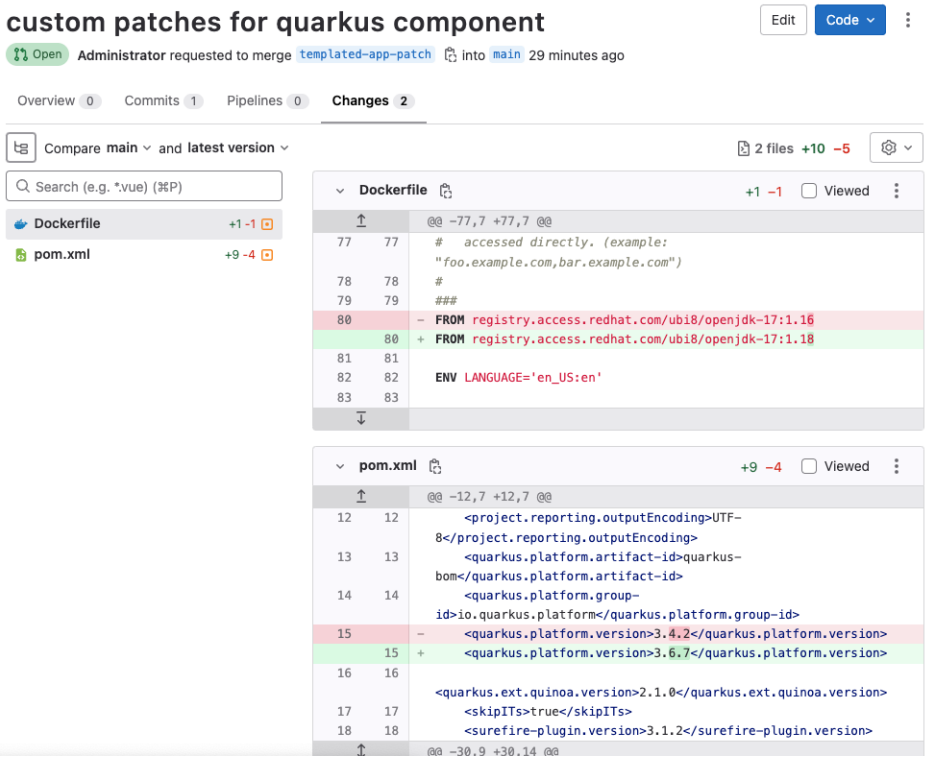


Figure 5-12: Inspect the suggested changes in the opened merge request.

Merge the auto-generated code changes

If everything went well and the review of the suggested code changes is positive, you as the component owner can merge this into the main branch. Go back to the **Overview** tab of the GitLab merge request, select the **Delete source branch** checkbox, and hit the **MERGE** button (Figure 5-13).

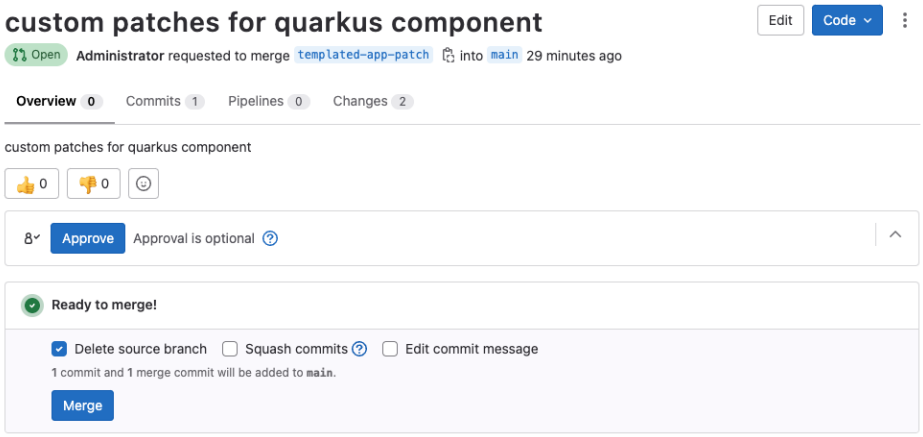


Figure 5-13: Merge the suggested changes.

Inspect the build pipeline

Now that the code changes are merged into the main branch, a new pipeline run is triggered via a webhook. After about two to three minutes, your code changes are available in the freshly built container image for the POI map service (Figure 5-14).

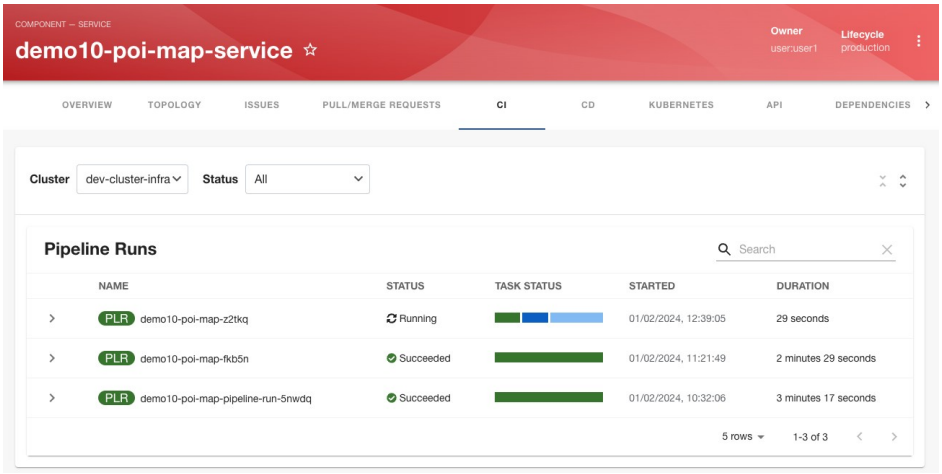


Figure 5-14: Tekton build pipeline triggered after merging code changes.

Verify all updates for the component

Let's verify if all the expected changes that have been applied by the patch template are indeed present. You can open the build pipeline details and then click on the second step named **package** to inspect the logs, as shown in Figure 5-15.

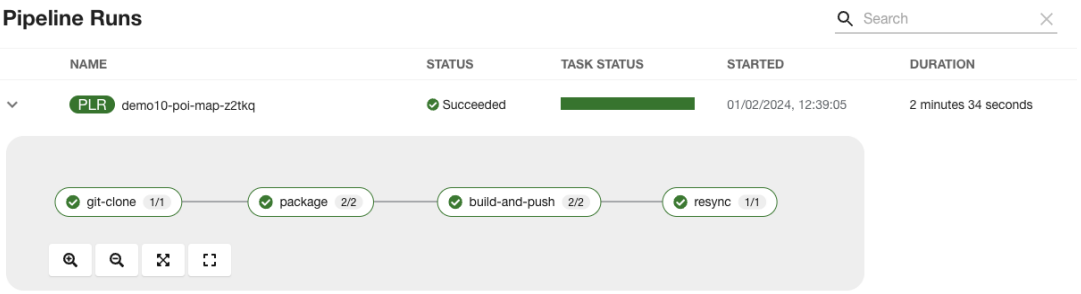


Figure 5-15: Check pipeline run details for the individual steps.

You can immediately see that the Quarkus version pulled down by Maven is now 3.6.7 instead of version 3.4.2 that was in use initially for this component according to the old template.

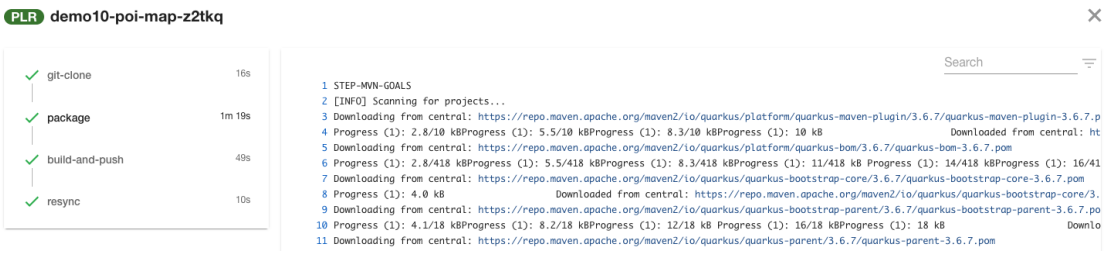


Figure 5-16: Inspect logs for pipeline step package.

Next, click the **build-and-push** step in order to check for the container base image that should now be `ubi8/openjdk-17:1.18` rather than `ubi8/openjdk-17:1.16` (Figure 5-17).



Figure 5-17: Inspect logs for pipeline step build-and-push.

Finally, open the **Topology** tab for the `demo01-poi-map-service` component. Click on the deployment and choose the **Resources** tab in the side pane that appears from the right to click **View Logs** (Figure 5-18).

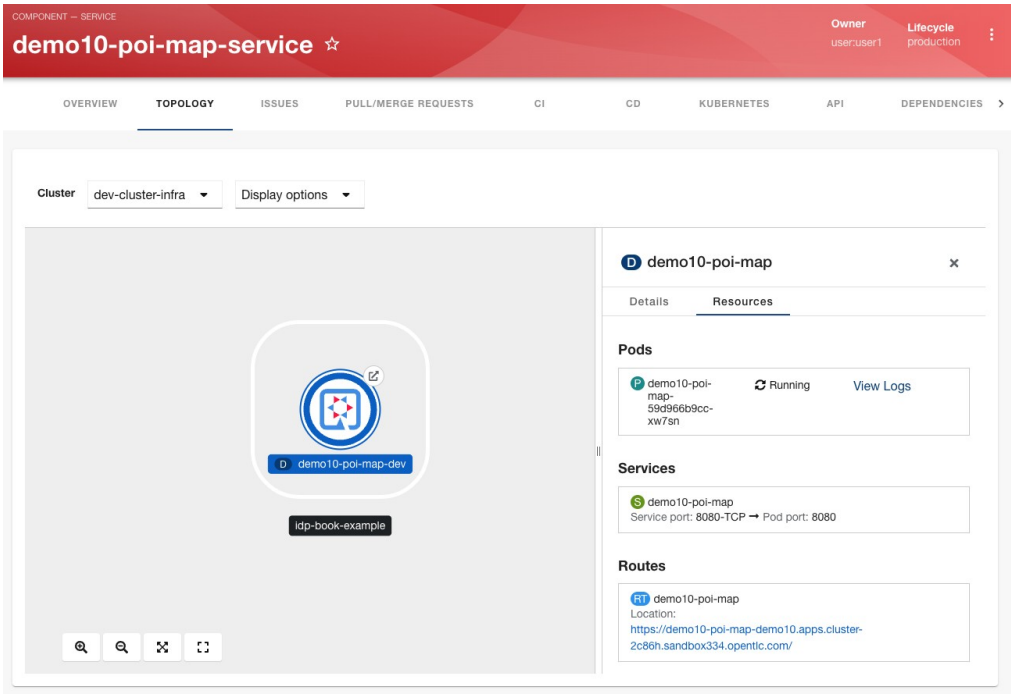


Figure 5-18: Component's Topology view with deployment resources selected.

If you scroll a bit further to the right within the logs window, you'll now also find the `smallrye-health` extension additionally included next to all the others that have already been there before applying the patch template (Figure 5-19).

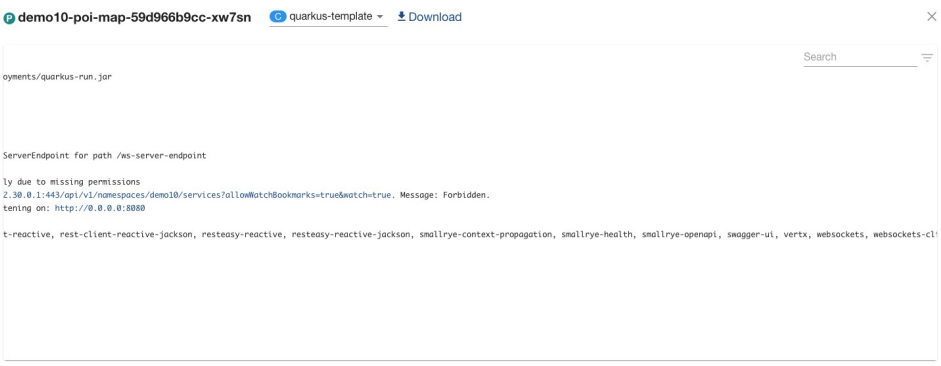


Figure 5-19: Pod's log view to inspect the application logs.

Updating existing deployments

When taking a closer look at the deployment for the POI backend application, you might wonder if you could switch from a standard deployment to a staged deployment. At the same time, it seems to be unclear and rather complicated for the typical application developer to understand what needs to be changed in a software component's underlying GitOps repository to achieve this. That's totally fine, because with a little help from a platform engineering team, all the complexity related to coming up with more sophisticated manifests that support a staged deployment can be baked into yet another software template. Let's find out how to benefit from such a patch template to get the job done.

Register the predefined patch template

In Chapter 3, you already prepared all templates for the book examples by forking the original template repository from the book's GitHub organization and customizing the RHDH cluster ID. You can always refer to the README of the original template repository if something is unclear. It explains all the preparation steps.

You'll now need to register the custom-deployment-patch template from within your forked template repository as follows:

1. Go to **+ Create → Register** existing component and copy the full HTTP(s) URL to the `template.yaml` file in the `custom-deployment-patch` folder of your forked Git repository hosting all book templates. Paste the URL into the form's (1) **Select URL** field, as shown in Figure 5-20.

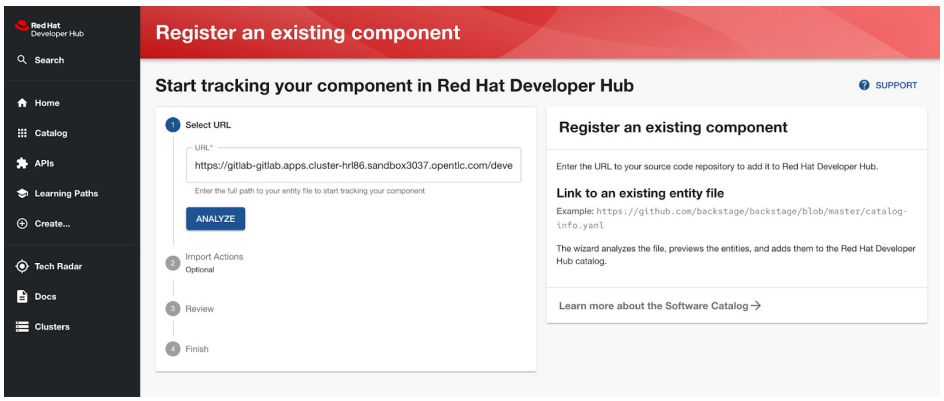


Figure 5-20: Template registration into the Software Catalog, step 1.

2. Clicking the **ANALYZE** button shows that two entities will be added into the software catalog (Figure 5-21). One entity is the location, the HTTP URL from where the template was loaded, and the other entity is the template itself.

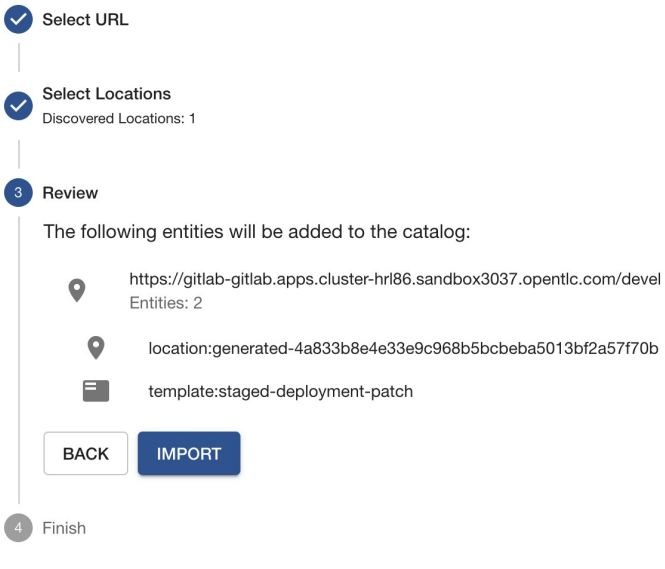


Figure 5-21: Template registration into the Software Catalog, step 2.

3. Click the **IMPORT** button to confirm. Figure 5-22 shows the entities have been added to the catalog.

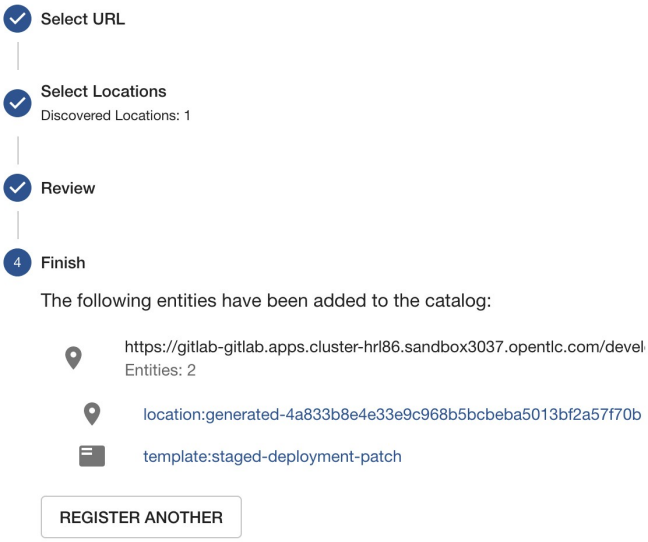


Figure 5-22: Template registration into the Software Catalog, step 3.

Apply the predefined patch template

Go to **+ Create** and select the **Staged Deployment Patch Template** by clicking the **CHOOSE** button in the lower-right corner of the template tile (Figure 5-23).

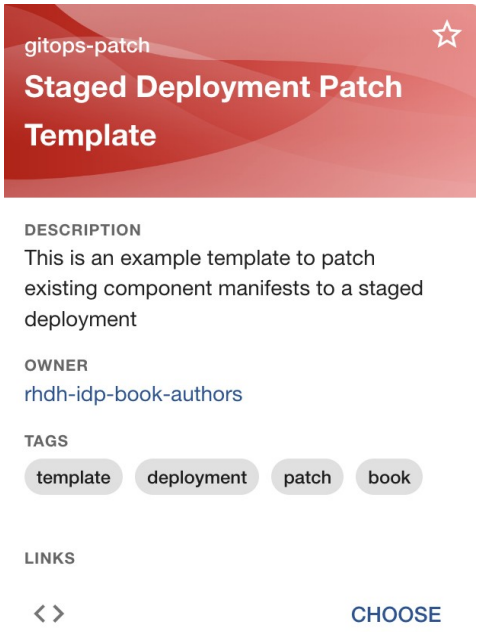


Figure 5-23: Staged Deployment Patch Template.

Template form wizard

This brings you to the template's form wizard where you can configure certain elements of the template (Figure 5-24). To keep things simple, this exemplary patch template only asks you to provide information about the component which you want to patch the GitOps manifests for: the cluster ID, the GitLab group/owner, the namespace, and the application ID. If you've been following the previously recommended naming conventions, you will specify `demo01` as namespace and `poi-backend` as application ID. You can optionally enter a branch name, which defaults to `templated-gitops-patch`, for the branch that will store the various manifest-related changes that get applied by the template. The auto-created branches will in turn become the sources for merge requests that get automatically opened in the GitOps repository of the component you referred to.

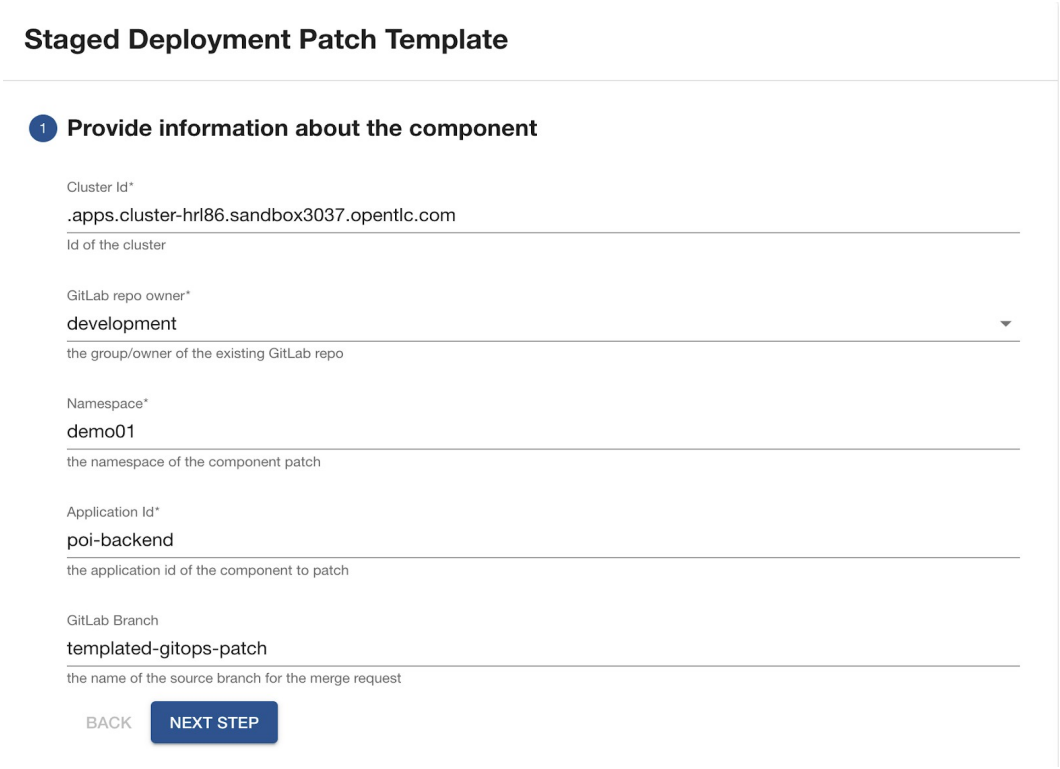


Figure 5-24: Staged Deployment Patch Template configuration form, input section.

Clicking the **NEXT STEP** button shows you a summary of all the entered form fields for a final review (Figure 5-25).

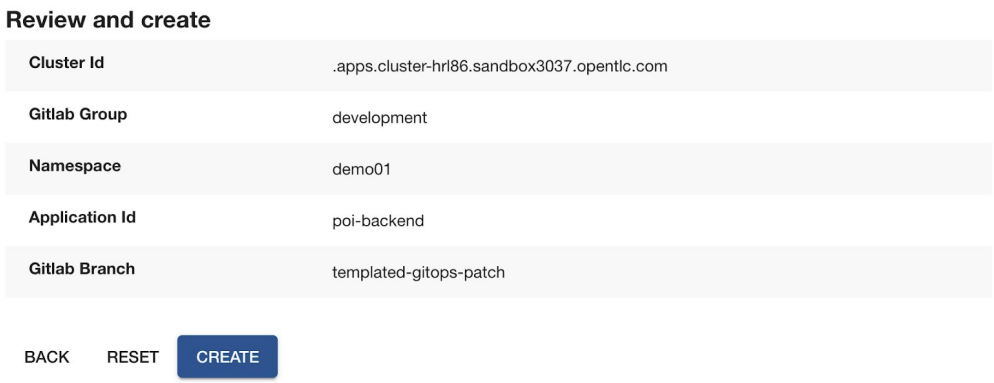


Figure 5-25: Staged Deployment Patch Template configuration form, final review.

Click **CREATE** to kick off the process of patching the component's GitOps repository.

Template patching procedure

The Staged Deployment Patch Template (Figure 5-26) is composed of multiple sequential steps, each of which represents either a built-in or a custom scaffolder action available in your running RHDH portal.

Note: Due to a peculiarity of how the currently available Backstage GitLab plug-in—more specifically, the scaffolder action for merge requests—has been implemented against the GitLab API, this template creates three separate merge requests while it ideally would only need to create a single one. The root cause for this is that any branch acting as the source for opened merge requests, is only allowed to contain homogenous Git changesets (i.e., either created, modified, or deleted files). Future implementations will hopefully do away with this limitation and allow to combine any file changes into a unified merge request.

1. `fetch:template`: This action fetches the template from its location and recursively walks through all source folders and files (see the `skeleton` subfolder at the origin). In each file, the scaffolder checks if it finds variables and needs to perform parameter replacements based on the settings which have been entered upfront in the form wizard.
2. `publish:gitlab:merge-request`: After creating new manifest-related files in a separate branch, this action will open the first merge request in the respective GitLab repository.
3. `debug:log`: This action prints the full URL pointing to the first merge request.
4. `publish:gitlab:merge-request`: After modifying existing manifest-related files in a separate branch, this action will open the second merge request in the respective GitLab repository.
5. `debug:log`: This action prints the full URL pointing to the second merge request.
6. `publish:gitlab:merge-request`: After deleting manifest-related files in a separate branch, this action will open the third merge request in the respective GitLab repository.
7. `debug:log`: This action prints the full URL pointing to the third merge request.

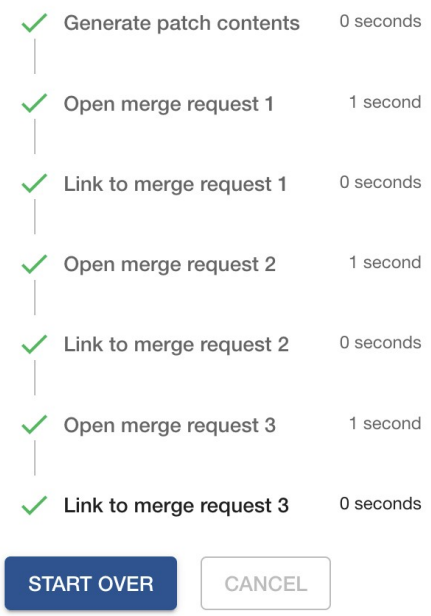


Figure 5-26: Staged Deployment Patch Template.

Review the opened merge requests

In GitLab, go to the `demo01-poi-backend-gitops` repository that has been patched by means of the template and inspect the opened merge requests (Figure 5-27). Remember the note from the previous section, which explains why you ended up with three separate ones.

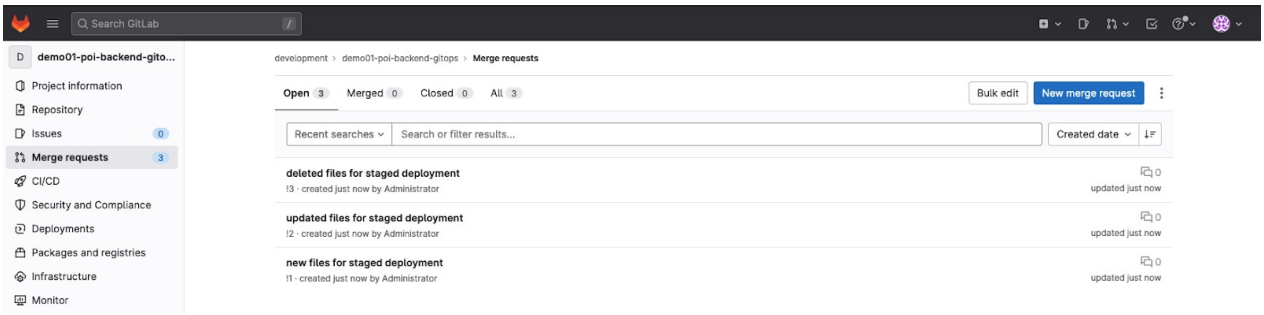


Figure 5-27: Inspect the opened merge requests after applying the template.

Clicking on one of the respective titles brings you to that merge request's overview. Figure 5-28 shows the one for newly created manifest files.

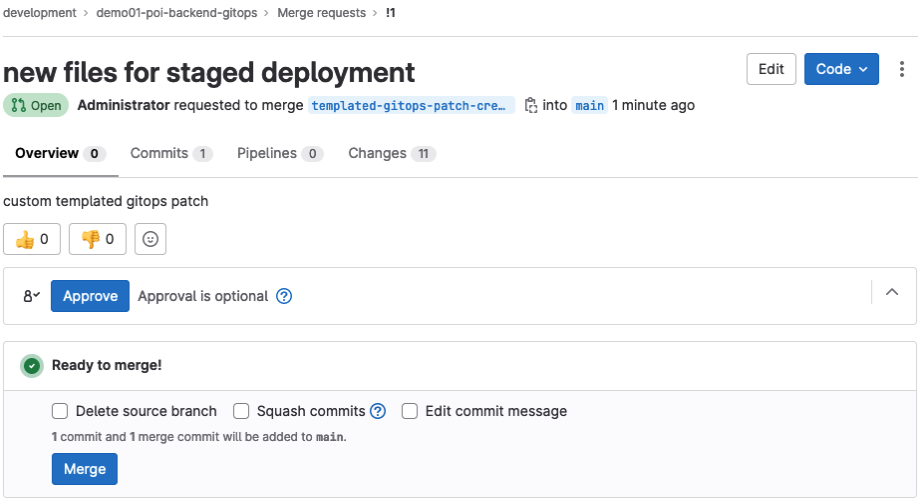


Figure 5-28: Merge request overview for newly created files.

If you want to take a closer look, feel free to inspect all individual changes for each of the merge requests by switching to the **Changes** tab. Figure 5-29 shows the changeset for this merge request.

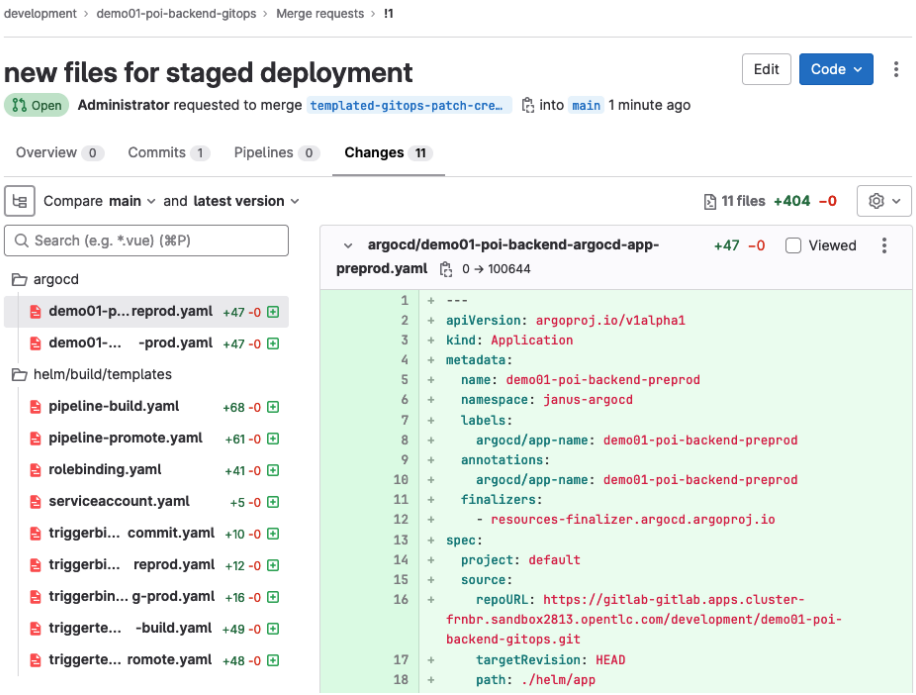


Figure 5-29: Merge request changeset details for newly created files.

Finally, go back to the **Overview** tab and confirm these auto-generated changes by clicking the **MERGE** button (Figure 5-30).

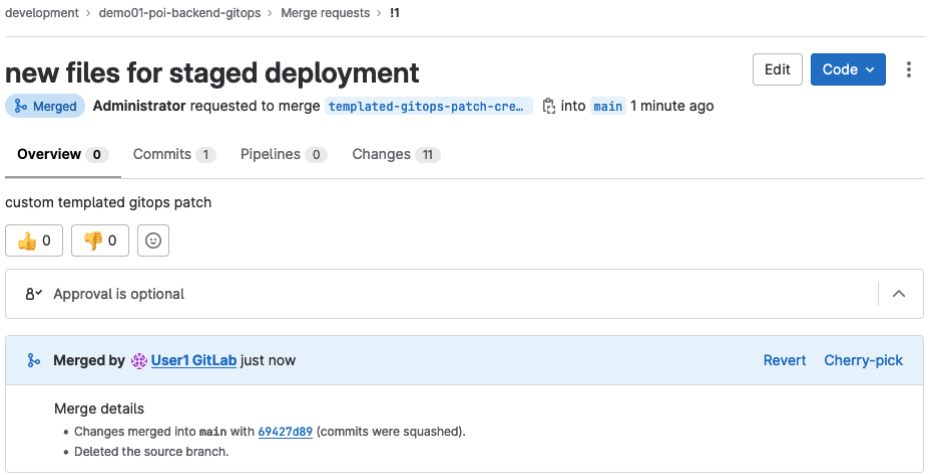


Figure 5-30: Merge request overview after merging the changes.

Make sure that in the end all three merge requests that have been opened by applying the template get successfully merged into the main branch of the `demo01-poi-backend-gitops` repository (Figure 5-31).

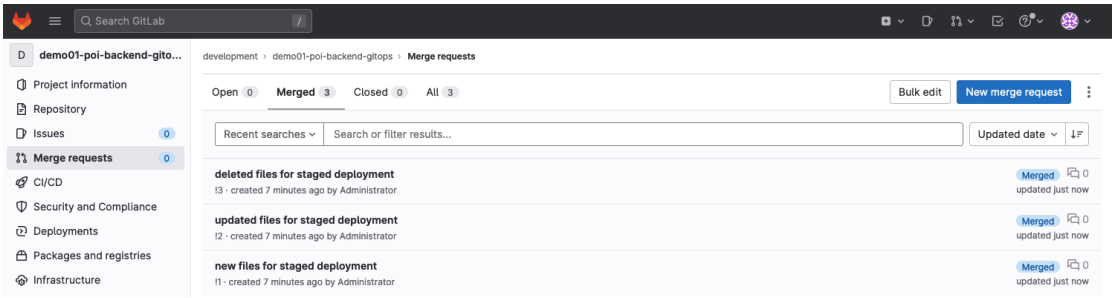


Figure 5-31: All merge requests merged in the GitOps repository.

Verify the patched deployment for the component

The changes you applied by means of the merge requests will eventually trigger Argo CD to take action based on the new desired state as defined by the `demo01-poi-backend-gitops` repository that hosts all the deployment manifests. If you don't want to wait for the next automatic sync interval, you can go into Argo CD's web UI, search for the `demo01-poi-backend-bootstrap` app, and manually hit the **SYNC** button to trigger it right away. This will lead to the reconciliation of all existing Kubernetes resources to match the new desired state of the patched deployment.

Check the namespaces

Most notably, in addition to the namespace `demo01` that already existed, you are supposed to see two new namespaces, namely `demo01-preprod` and `demo01-prod`. You can verify this

by going into the OpenShift web console and checking for these additional namespaces (Figure 5-32), which are used to perform a staged deployment and promote the POI backend application from the development stage (`demo01` namespace) to the pre-production stage (`demo01-preprod`) and finally to the production stage (`demo01-prod`).

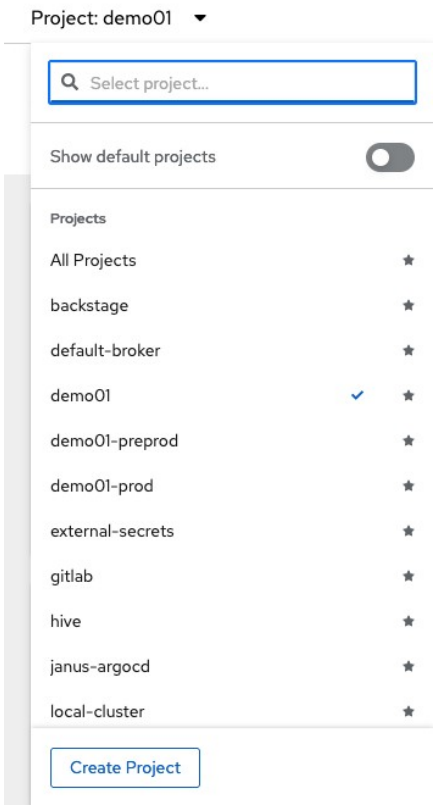


Figure 5-32: OpenShift web console projects/namespaces.

Check the database and backend deployments

In the OpenShift web console, click on the `demo01-preprod` namespace by selecting it from the **Projects** drop-down menu and open the **Topology** view (Figure 5-33). You should see the POI backend application (Deployment) together with its PostgreSQL database (StatefulSet). The same holds true if you look into the second namespace called `demo01-prod`. For now, don't worry too much that in both these namespaces only the database instances are successfully up and running while the backend applications are not healthy yet. The reason for this is an error (`ImagePullBackOff`), which stems from the fact that the needed container images aren't yet available for either of the two deployments.

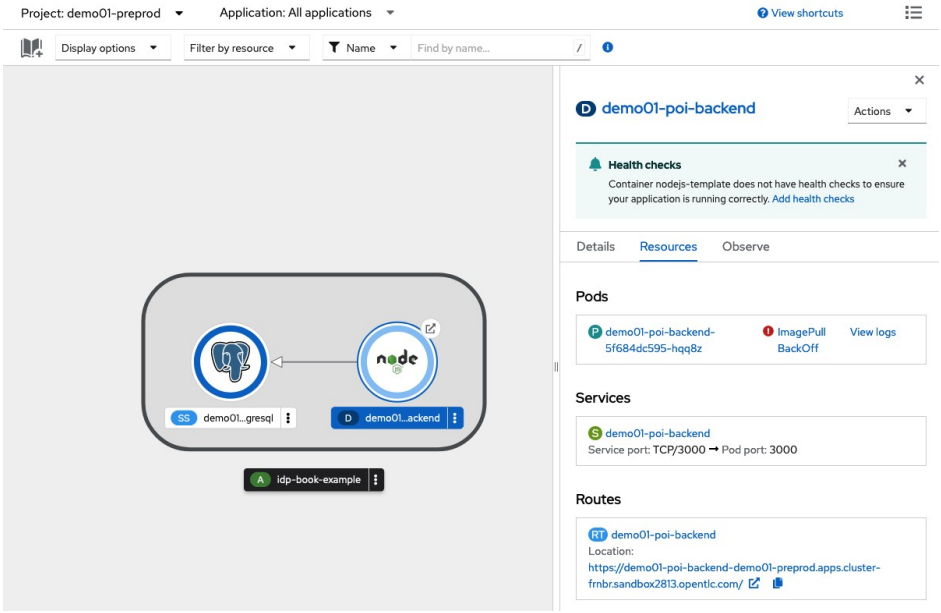


Figure 5-33: OpenShift web console topology view pre-prod project/namespace.

Promote the backend application

Thanks to the applied template patch, the POI backend now supports a staged deployment. In order to promote the application from the development stage (currently running in the `demo01` namespace) to the pre-production stage (`demo01-preprod` namespace), all you need to do is create a tag in the corresponding source code repository of the application. When the time comes to put this tagged version into production, you create a release based on this tag which leads to the promotion of the application from the pre-production stage (`demo01-preprod` namespace) to the production stage (`demo01-prod` namespace).

Tag the Git branch

Open the `demo01-poi-backend` repository in GitLab and choose **Repository → Tags** from the left navigation menu (Figure 5-34).

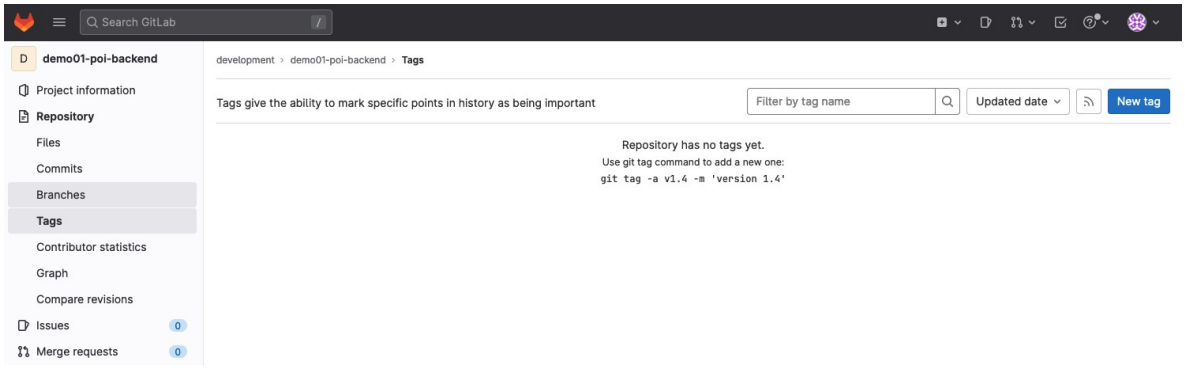


Figure 5-34: GitLab tags view in source code repository.

Click the **NEW TAG** button in the upper right and fill out the form with the details describing the tag to create, as shown in Figure 5-35.

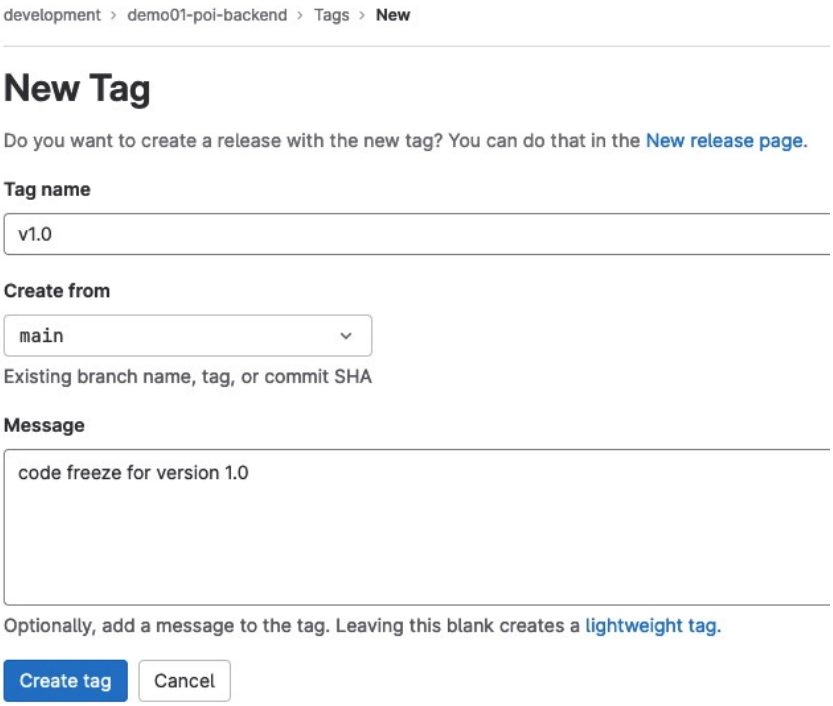


Figure 5-35: GitLab tag creation form in source code repository.

Finally, confirm by clicking the **CREATE TAG** button (Figure 5-36).

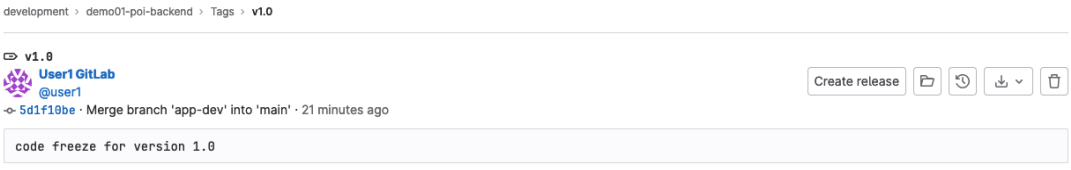


Figure 5-36: GitLab tag summary view in source code repository.

The creation of this new tag `v1.0` triggered the pipeline run of a new CI pipeline named `demo01-poi-backend-promote`, which (as its name implies) is intended to promote the application in question from one deployment stage to the next. In this case, your tag promotes the POI backend application from development to pre-production by copying the respective container image from the source image repository `demo01` to the target image repository `demo01-preprod`. While you might want to perform more sophisticated actions in a real-world setting, the approach shown here illustrates typical concepts and activities happening behind the scenes. You can inspect the pipeline run directly in Red Hat Developer Hub's web UI by opening the CI tab of the `demo01-poi-backend` component (Figure 5-37).

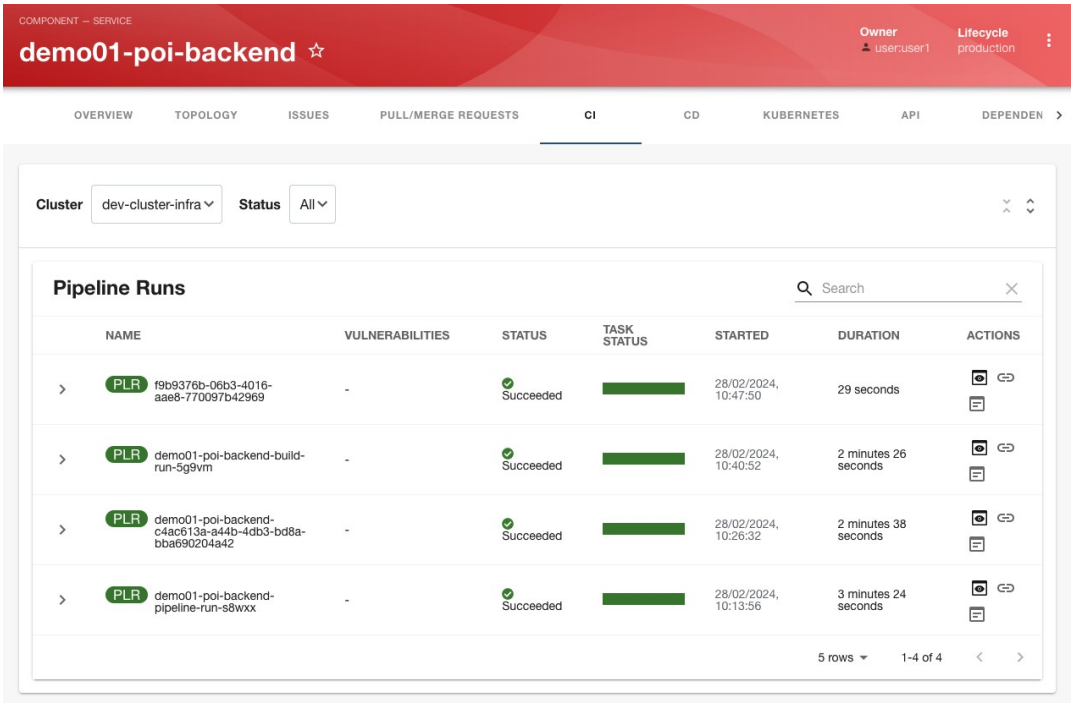


Figure 5-37: RHDH component view CI tab showing successful promote pipeline run.

Once this pipeline run successfully finishes, you can switch to the OpenShift web console and open the Topology view for the `demo01-preprod` namespace, where you should see that the promoted application is already running fine. The initial `ImagePullBackOff` error is now gone because the container image backing the deployment can finally be resolved correctly.

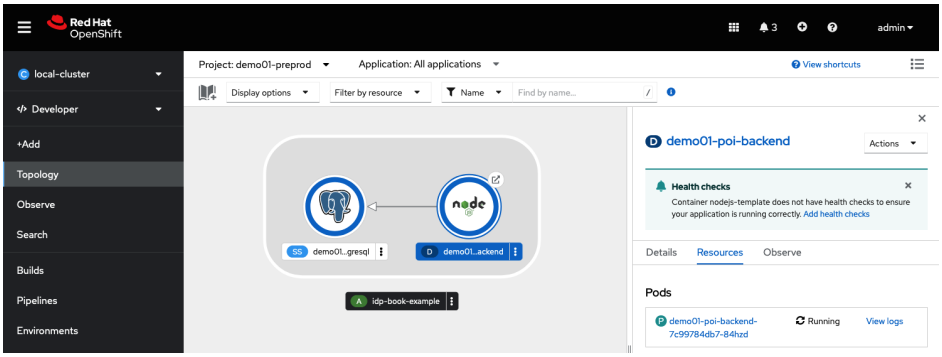


Figure 5-38: OpenShift Topology view in pre-prod project/namespace showing running deployment.

Create a new release

Open the `demo01-poi-backend` repository in GitLab and choose **Repository → Tags** from the left navigation menu. You should find the previously created tag named `v1.0` if you followed the naming conventions (Figure 5-39).

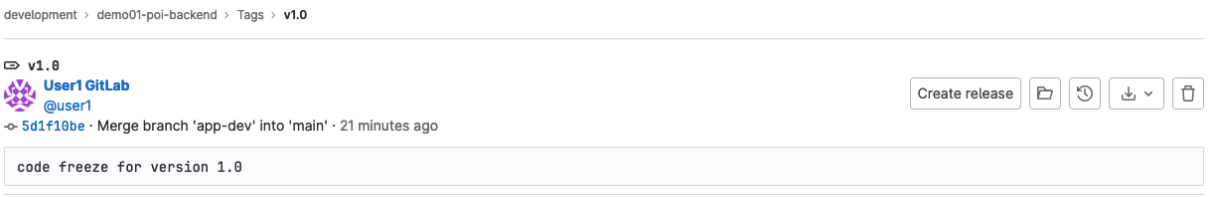


Figure 5-39: GitLab tags overview showing the previously created v1.0 tag.

Click the **CREATE RELEASE** button on the right, which opens a form to describe the upcoming release (Figure 5-40). Fill out the main fields as you see fit.

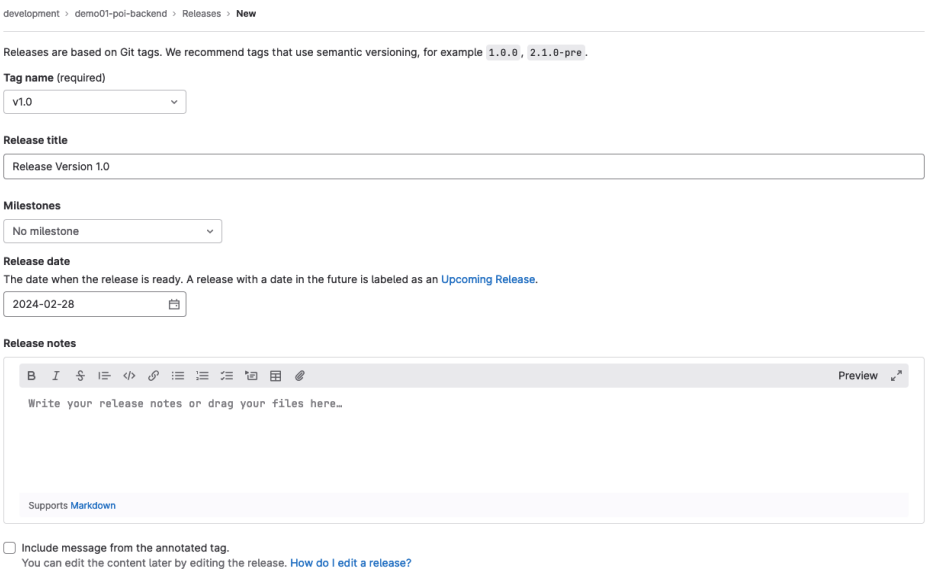


Figure 5-40a: GitLab release creation form wizard.

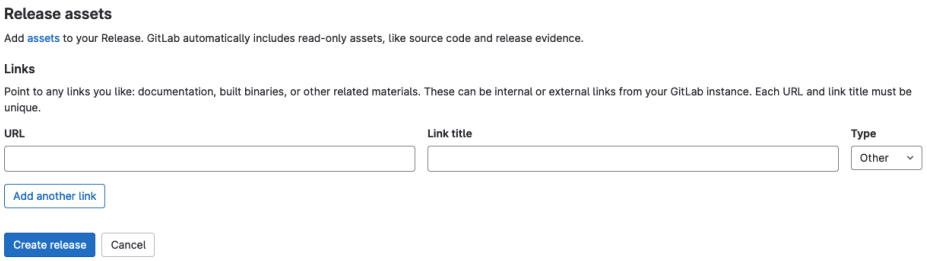


Figure 5-40b: GitLab release creation form wizard.

When you are done, scroll down to the end of the page and click the **CREATE RELEASE** button. You'll be presented with a summary for the newly created release (Figure 5-41).

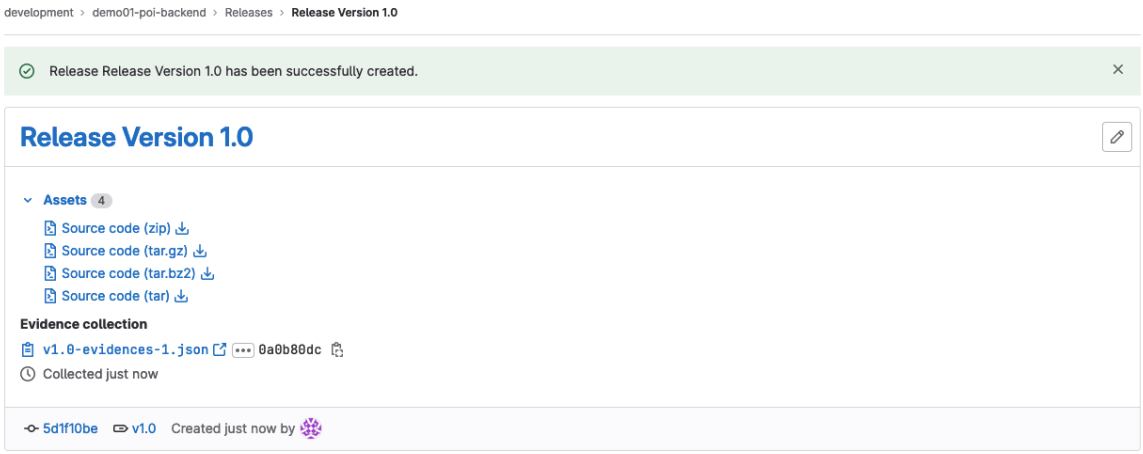


Figure 5-41: GitLab release summary view.

The creation of this new release triggered another run of the new CI pipeline `demo01-poi-backend-promote`, which is used to promote the application in question from one deployment stage to the next. In this case, the created release on GitLab promotes the POI backend application from pre-production to production by copying the respective container image from the source image repository `demo01-preprod` to the target image repository `demo01-prod`. While you might want to perform more sophisticated actions in a real-world setting, the approach shown here illustrates typical concepts and activities happening behind the scenes. You can inspect the pipeline run directly in the Red Hat Developer Hub's web UI by opening the CI tab of the `demo01-poi-backend` component (Figure 5-42).

Once this pipeline run successfully finishes, you can switch to the OpenShift web console and open the Topology view for the `demo01-prod` namespace, where you should see that the promoted application is already running fine (Figure 5-43). Remember the initial `ImagePullBackOff` error, which is now gone because the container image backing the deployment could finally be resolved correctly.

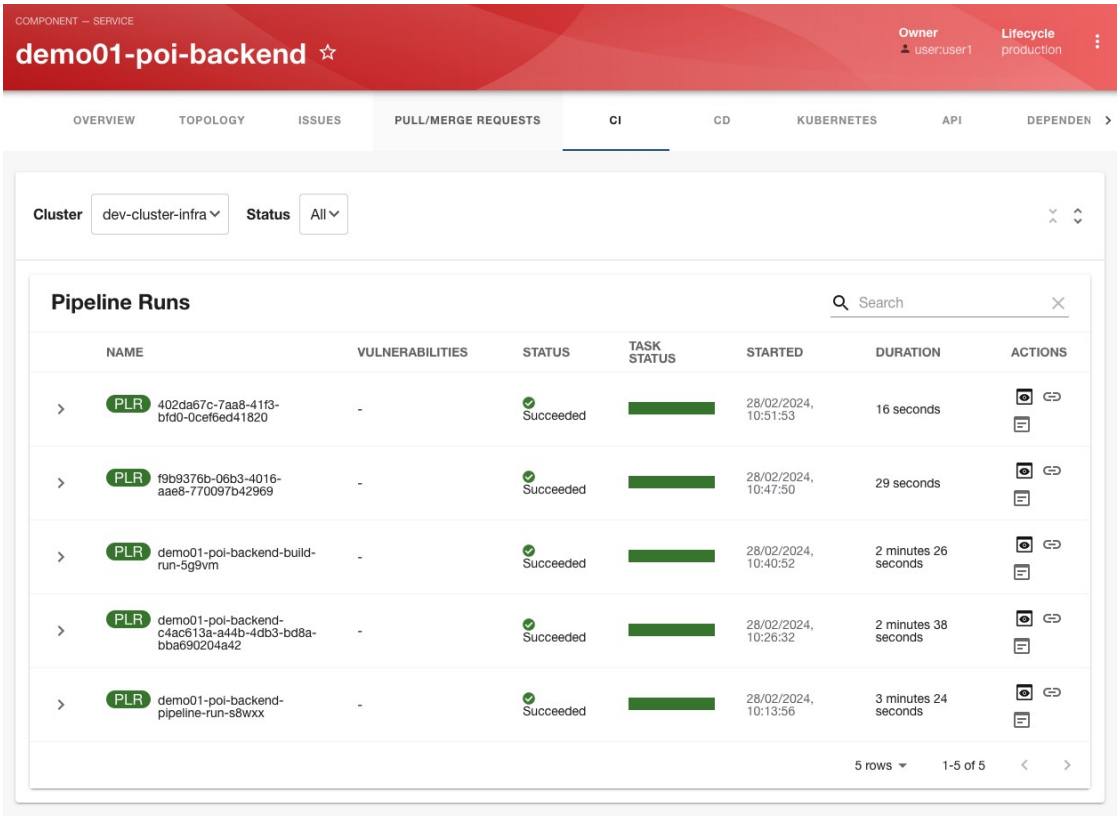


Figure 5-42: RHDH component view CI tab showing successful promote pipeline run.

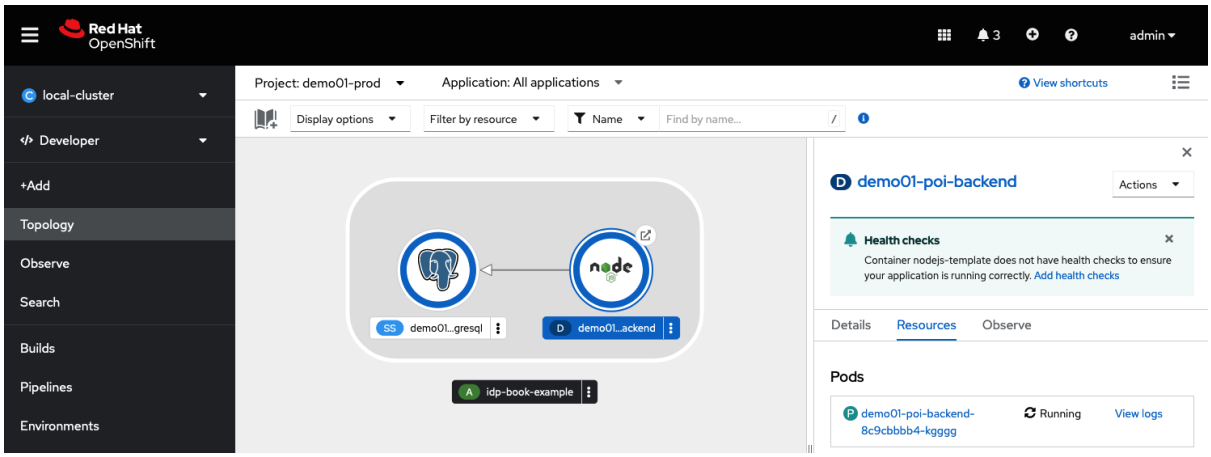


Figure 5-43: OpenShift Topology view prod project / namespace showing running deployment.

And there you have it! By means of another turnkey-ready patch template, you were able to introduce a staged deployment mechanism for an existing software component that has been originally scaffolded by an application template without support for multi-stage deployments.

Summary

In this chapter, you learned how Red Hat Developer Hub supports two crucial maintenance activities, namely template evolution and component updates. While the continuous evolution of software templates may be among the tasks of platform engineers, maintaining existing components tends to be the responsibility of application developers. This notwithstanding, platform engineers can make developers' lives considerably easier by baking certain application-specific update operations themselves into Software Templates. Using this kind of maintenance template, developers can follow a guided and mostly automated process to confidently patch existing components in a standard way.

In this chapter's exercises, you saw the process of using maintenance templates to both incorporate new requirements into an existing software template and to patch a previously scaffolded software component. You also worked on slightly more involved updates that go beyond the mere application scope and deal with operational and deployment-related aspects of components. In those cases, you as a developer could reuse handy patch templates crafted by platform engineers that allow for the same convenience when applying them onto an underlying GitOps repository rather than acting on a component's source code only.

Chapter 6: Embracing the future of development with IDPs

You've reached the end of this exploration of internal developer platforms through the lens of Red Hat Developer Hub (RHDH) and its foundations in the Backstage project. Software development is in a time of change. The journey from understanding the basic constructs of an IDP to navigating the complexities of software templates, the Software Catalog, and the evolutionary processes involved in maintaining and updating these elements, underscores a significant shift towards more streamlined, efficient, and collaborative development practices.

Paradigm shift

The adoption of IDPs like Backstage and RHDH represents more than just a shift in tools or processes; it embodies a paradigm shift in how organizations approach software development. By centralizing access to tools, services, and information, IDPs reduce cognitive load on developers, enabling them to focus on their core task of creating value through code. This shift is not just about efficiency; it's about empowering developers with the right tools and information at the right time, fostering a culture of collaboration, and driving innovation.

Journey ahead

As the digital landscape continues to evolve, the role of IDPs will become increasingly critical. The ability to quickly adapt to changing technologies, methodologies, and business requirements will be a key determinant of success. The journey through RHDH and the

exploration of Backstage has provided a glimpse into the potential of IDPs to meet these challenges by:

- Simplifying the complexity of modern software development
- Enhancing visibility and collaboration across teams
- Automating repetitive tasks to accelerate development cycles
- Providing a scalable framework to manage growing software ecosystems

Tomorrow and tomorrow

For organizations and developers navigating software development, the future is in platforms that simplify some of the complexity of distributed systems and new techniques to amplify the capabilities of developers. RHDH and Backstage are scaffolding, or starting points. The real journey is in the continuous evolution of these platforms and the ecosystems around them. Success with these platforms like any others depends on developers, platform engineers, and organizational leaders embracing these changes, contributing to the open source communities that drive them, and shaping the future of development in their own organizations.

The evolution of developer portals like Backstage and RHDH is a collaborative effort, reflecting the collective experience of the developer community. By participating in this ecosystem, sharing knowledge, and contributing to its growth, you can play a part in making development for today's advanced cloud architectures more accessible, enjoyable, and effective.

Using platforms like RHDH and Backstage to create portals fitted to their specific needs, developers, teams, and organizations can unlock new levels of efficient collaboration and the innovation that results.

Today

You should have a good idea of IDP terms, techniques, and certainly Software Templates. To continue exploring Red Hat Developer Hub, you can check out resources from documentation to hands-on time with a running RHDH on OpenShift.

Red Hat Developer Hub resources

- Download the latest electronic version of this book, continuously updated as errata and software advances require: https://red.ht/developer-hub-book
- Find repos holding the code and templates for the exercises and guidance on infrastructure requirements for running them in this book's GitHub org: https://github.com/developer-hub-books
- Explore hands-on learning paths on Red Hat Developer:
    - Install and configure Red Hat Developer Hub and explore templating basics
    - Application development with Red Hat Developer Hub
- Visit the Red Hat Developer Hub product page.

About the authors

Hans-Peter Grahsl is a Red Hat Developer Advocate and open source enthusiast who loves helping developer communities improve their productivity. He is passionate about event-driven architectures, distributed stream processing, and data engineering. Grahsl lives with his family in Graz, Austria, and travels often to speak at international developer conferences.

Joshua Wood is a Principal Developer Advocate at Red Hat. Co-author of *Kubernetes Operators* (O'Reilly, 2020) and *OpenShift for Developers, 2nd Edition* (O'Reilly, 2021), he was formerly responsible for documentation at CoreOS. He likes fast cars, slow boats, and short autobiographies.

Ryan Jarvinen is a Red Hat Principal Developer Advocate and noted speaker living and working in Sacramento, California. Jarvinen enjoys learning about best practices for developer experience and usability in the cloud-native ecosystem and helping teams develop strategies for maximizing collaboration using open source technologies.